Practical - 1

Problem Definition

• Create a React application using Vite for a student portfolio page.

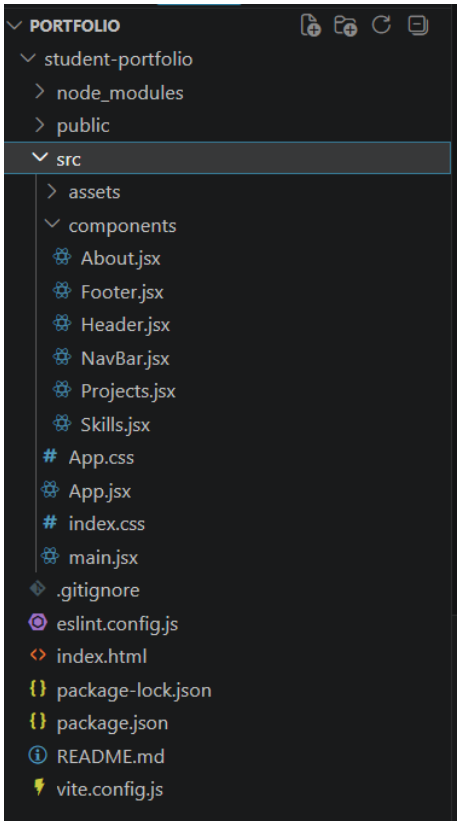
• The application must include at least 4 reusable components: Header, About, Skills, and Footer.

• Each component must be independently structured and composed into a single-page layout.

• No logic or JSX should be duplicated across components.

• Props must be used to pass at least one piece of data into a minimum of 2 components.

Folder Structure:



Steps:

1. Verify Node.js and npm are installed:

```
PS C:\Users\Bhumi> node -v
v24.18.0
PS C:\Users\Bhumi> npm -v
11.18.0
PS C:\Users\Bhumi>
```

2. Scaffold the React app using Vite:

```
PS C:\Users\Bhumi>  npm create vite@latest student-portfolio
Need to install the following packages:
create-vite@9.1.2
Ok to proceed? (y) y

> hello@1.0.0 npx
> create-vite student-portfolio
```

Main.jsx

```jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

3. Inside src/components, create four files: Header.jsx, About.jsx, Skills.jsx, Footer.jsx.

Header.jsx

```jsx
function Header({ name, tagline }) {
  return (
    <header className="hero">
      <p className="hero-eyebrow">{name} &mdash; Portfolio</p>
      <h1 className="hero-title">Hi, I'm {name}. {tagline}</h1>
      <p className="hero-subtitle">
        IT student building full-stack web applications
        with React, Node.js, and MongoDB.
      </p>
      <div className="hero-actions">
        <a className="btn btn-primary" href="#projects">View My Work</a>
        <a className="btn btn-secondary" href="#contact">Contact Me</a>
      </div>
    </header>
  )
}

```

```
export default Header
```

About.jsx

```
function About() {
  return (
    <div className="bento-card span-3" id="about">
      <p className="card-number">01</p>
      <h2 className="card-title">About Me</h2>
      <p className="card-text">
        I am an Information Technology student passionate about creating
        modern, responsive, and user-friendly web applications. I enjoy
        learning new technologies, solving real-world problems, and
        developing projects that enhance my skills in web development.
      </p>
    </div>
  )
}

export default About
```

Skills.jsx

```
function Skills({ skillList }) {
  return (
    <div className="bento-card span-3" id="skills">
      <p className="card-number">05</p>
      <h2 className="card-title">Skills</h2>
      <div className="skills-groups">
        {skillList.map((group) => (
          <div key={group.category}>
            <p className="skill-group-title">
              <span className="skill-dot" style={{ background: group.color }}></span>
              {group.category}
            </p>
            <div className="skill-tags">
              {group.skills.map((skill) => <span className="skill-tag" key={skill}>{skill}</span>)}
            </div>
          </div>
        ))}
      </div>
    </div>
  )
}
```

```
24.
25.export default Skills
```

Footer.jsx

```
10.function Footer() {
11.  return (
12.    <div className="bento-card span-3 contact-card" id="contact">
13.      <p className="card-number">06</p>
14.      <h2 className="card-title">Let's build something together</h2>
15.      <p className="card-text">Open to internships, freelance work, and
   collaboration.</p>
16.      <a className="btn btn-secondary"
   href="mailto:student@example.com">Email Me</a>
17.    </div>
18.  )
19.}
20.
21.export default Footer
```

4. Import and compose all four components inside App.jsx in a single layout.

App.jsx

```
5. import Header from './components/Header'
6. import NavBar from './components/NavBar'
7. import About from './components/About'
8. import Projects from './components/Projects'
9. import Skills from './components/Skills'
10.import Footer from './components/Footer'
11.
12.function App() {
13.  const projectList = [
14.    {
15.      title: 'Personal Expense Tracker (UI/UX)',
16.      description: 'Designed a user-friendly interface and wireframes
   focused on a clean and intuitive user experience.',
17.      tech: ['Figma'],
18.    },
19.
20.    {
21.      title: 'Beauties & Bows \u2013 E-commerce Website',
22.      description: 'Developed a responsive front-end website for a
   fashion brand, implementing product display and user-friendly
   navigation.',
23.      tech: ['HTML', 'CSS', 'JavaScript'],
```

```
    },
    {
      title: 'Note-Taking Web Application',
      description: 'Created a web app to add, edit, and delete notes, with a backend integrated with a database for data management.',
      tech: ['PHP', 'MySQL'],
    },
    {
      title: 'Shake & Bake Website',
      description: 'Developed and customized a business website using WordPress, managing layout, design, and content.',
      tech: ['WordPress'],
    },
  ]

  const skillList = [
    { category: 'Frontend', color: '#1d3557', skills: ['React', 'JavaScript', 'HTML', 'CSS'] },
    { category: 'Backend', color: '#457b9d', skills: ['Node.js', 'Express', 'REST APIs'] },
    { category: 'Database', color: '#2a9d8f', skills: ['MongoDB', 'Mongoose', 'MySQL'] },
    { category: 'Tools & Practices', color: '#6c757d', skills: ['Git', 'GitHub'] },
  ]

  return (
    <div className="page">
      <NavBar />
      <Header name="Bhumi Vora" tagline="I build web apps." />
      <div className="bento-grid">
        <About />
        <Projects projectList={projectList} />
        <Skills skillList={skillList} />
        <Footer />
      </div>
      <p className="footer-note">&copy; {new Date().getFullYear()} &middot; Built with React &amp; Vite</p>
    </div>
  )
}

export default App
```

Supplementary Problem:

1. **Add a NavBar component that highlights the active section.**

NavBar.jsx

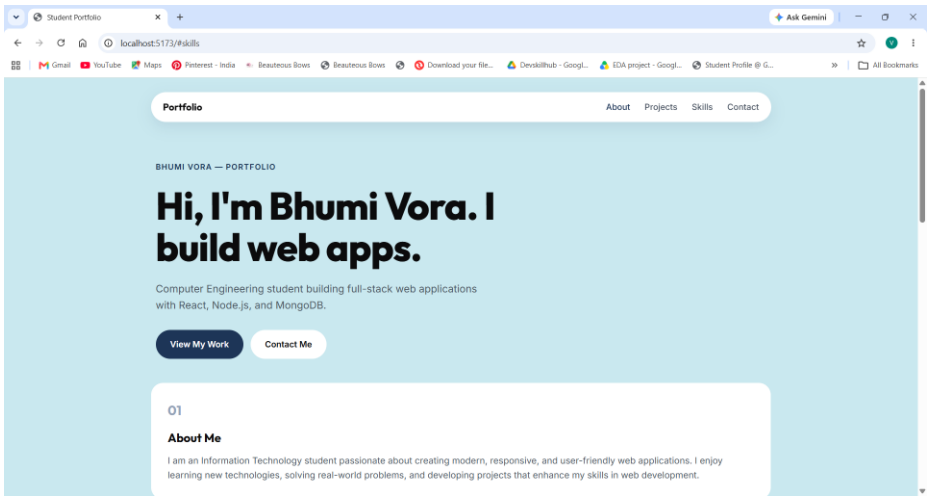
```
4. import { useState } from 'react'
5.
6. function NavBar() {
7.   const sections = ['About', 'Projects', 'Skills', 'Contact']
8.   const [active, setActive] = useState('About')
9.
10.  return (
11.    <nav className="navbar">
12.      <span className="navbar-brand">Portfolio</span>
13.      <div className="navbar-links">
14.        {sections.map((section) => <a key={section} href={'#' +
   section.toLowerCase()} onClick={() => setActive(section)}
   className={active === section ? 'active' : ''}>{section}</a>)}
15.      </div>
16.    </nav>
17.  )
18.}
19.
20.export default NavBar
```

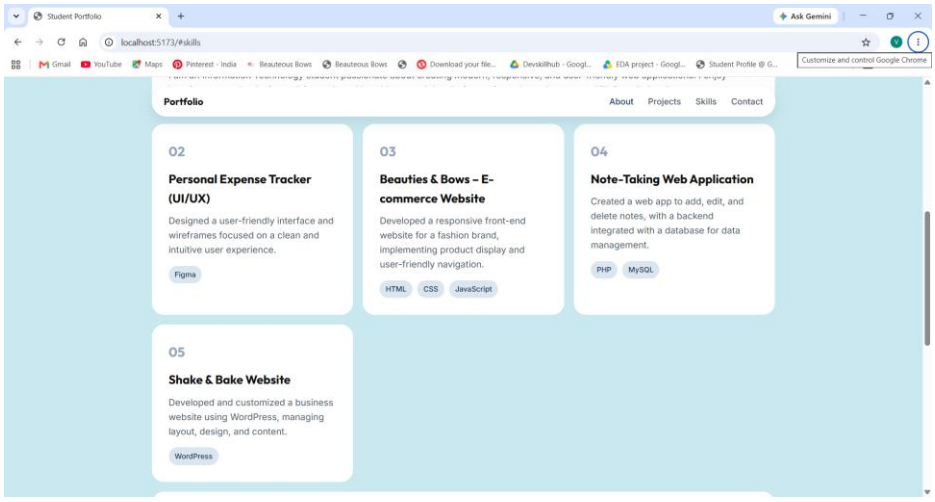
Projects.jsx

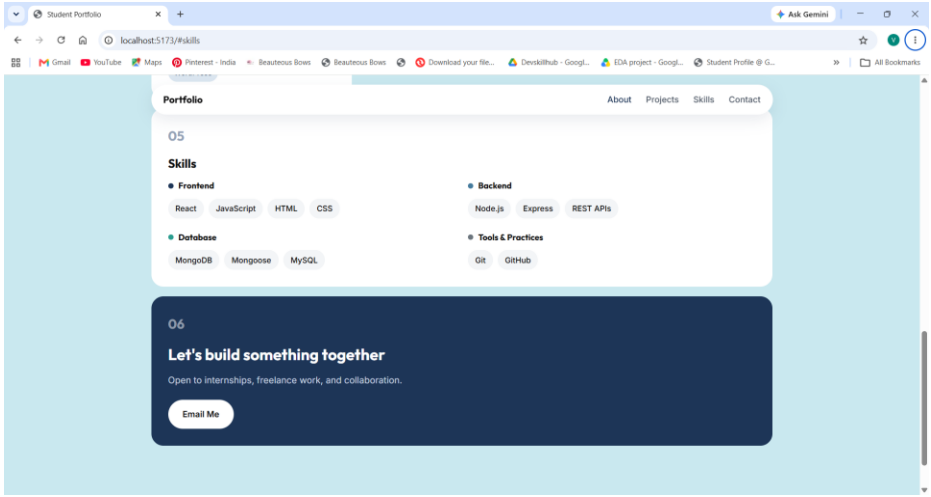
```
7. function Projects({ projectList }) {
8.   return (
9.     <>
10.      {projectList.map((project, index) => (
11.        <div className="bento-card" key={project.title} id={index === 0
   ? 'projects' : undefined}>
12.          <p className="card-number">{String(index + 2).padStart(2,
   '0')}</p>
13.          <h3 className="card-title">{project.title}</h3>
14.          <p className="card-text">{project.description}</p>
15.          <div className="card-tech">
16.            {project.tech.map((t) => <span className="tech-tag"
   key={t}>{t}</span>)}
17.          </div>
18.        </div>
19.      ))}
20.    </>
21.  )
22.}
23.
24.export default Projects
```

7. Run the app and verify all components render correctly without console errors.

Output:







Practical -2

Problem Definition:

• Extend the portfolio application from Practical 1 by adding React Router.

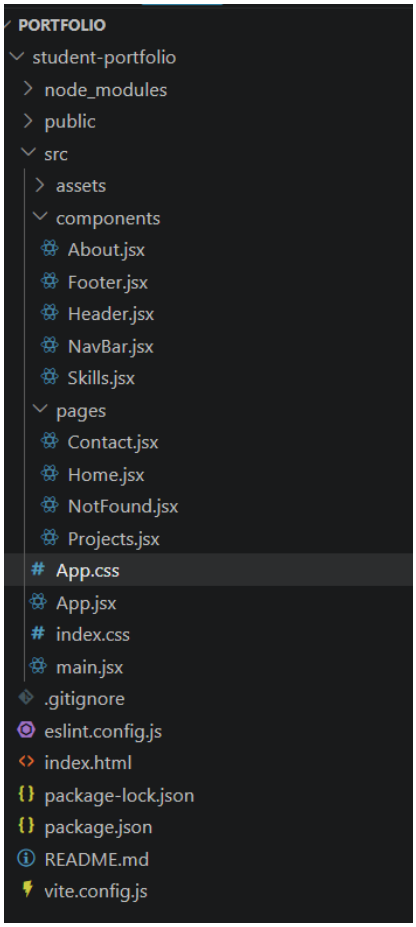
• Implement a navigation bar with at least 3 routes: Home, Projects, and Contact.

• Each route must render a distinct component without a full page reload.

• Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page.

• The Contact page must include a controlled input that captures and displays user input in real time.

Folder Structure:



Steps:

1. **Install React Router in the existing portfolio project: npm install react-router-dom**

```
PS C:\Users\Bhumi> npm install react-router-dom
```

2. **Wrap the App component with BrowserRouter in main.jsx: import { BrowserRouter } from 'react-router-dom';**

main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)
```

3. **Define 3 routes inside App.jsx using Routes and Route: } /> } /> } />**

App.jsx

```jsx
import { useState, useEffect } from 'react'
import { Routes, Route } from 'react-router-dom'
import NavBar from './components/NavBar'
import Home from './pages/Home'
import Projects from './pages/Projects'
import Contact from './pages/Contact'
import NotFound from './pages/NotFound'
import './App.css'

function App() {
  const [darkMode, setDarkMode] = useState(true)

  useEffect(() => {
    document.documentElement.className = darkMode ? 'dark' : 'light'
  }, [darkMode])

  return (
    <div className="app-shell">
      <NavBar darkMode={darkMode} setDarkMode={setDarkMode} />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/projects" element={<Projects />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
```

```
    </div>
  )
}

export default App
```

4. Build the NavBar using Link (not tags) to avoid full page reloads.

NavBar.jsx

```
import { NavLink, Link } from 'react-router-dom'

function NavBar({ darkMode, setDarkMode }) {
  return (
    <nav className="navbar">
      <div className="navbar-brand">
        <span className="brand-mark">BD</span>
        <span>Portfolio</span>
      </div>

      <div className="navbar-links">
        <NavLink to="/" end className={({ isActive }) => isActive ? 'active' : ''}>Home</NavLink>
        <NavLink to="/projects" className={({ isActive }) => isActive ? 'active' : ''}>Projects</NavLink>
        <a href="/#skills">Skills</a>
        <NavLink to="/contact" className={({ isActive }) => isActive ? 'active' : ''}>Contact</NavLink>
      </div>

      <div className="navbar-actions">
        <button className="theme-toggle" onClick={() => setDarkMode(!darkMode)}>
          {darkMode ? '☀' : '🌙'}
        </button>
        <Link to="/contact" className="navbar-cta">Hire Me</Link>
      </div>
    </nav>
  )
}

export default NavBar
```

5. Inside Contact.jsx, create a controlled input tied to useState: const [message, setMessage] = useState(''); setMessage(e.target.value)} />

Contact.jsx

```
import { useState } from 'react'
```

```
3.
4.  function Contact() {
5.    const [name, setName] = useState('')
6.    const [email, setEmail] = useState('')
7.    const [message, setMessage] = useState('')
8.    const maxLength = 250
9.
10.   const handleSubmit = (e) => {
11.     e.preventDefault()
12.     alert(`Thanks ${name || 'there'}! Your message has been noted.`)
13.   }
14.
15.   return (
16.     <div className="page">
17.       <div className="bento-card span-3 contact-card">
18.         <h2 className="card-title">Contact Me</h2>
19.         <form className="contact-form" onSubmit={handleSubmit}>
20.           <label>
21.             Name
22.             <input type="text" value={name} onChange={(e) =>
    setName(e.target.value)} placeholder="Your name" />
23.           </label>
24.           <label>
25.             Email
26.             <input type="email" value={email} onChange={(e) =>
    setEmail(e.target.value)} placeholder="you@example.com" />
27.           </label>
28.           <label>
29.             Message
30.             <textarea
31.               value={message}
32.               maxLength={maxLength}
33.               onChange={(e) => setMessage(e.target.value)}
34.               placeholder="Write your message..."
35.               rows={5}
36.             />
37.           </label>
38.           <p className="char-count">{message.length} / {maxLength}
    characters</p>
39.           {message.length > 0 && (
40.             <div className="live-preview"><strong>Live
    preview:</strong> {message}</div>
41.           )}
42.           <button type="submit" className="btn btn-secondary">Send
    Message</button>
43.         </form>
44.       </div>
45.     </div>
```

```
  )
}

export default Contact
```

Home.jsx

```
import { useState } from 'react'
import Header from '../components/Header'
import About from '../components/About'
import Skills from '../components/Skills'

const skillList = [
  { category: 'Frontend', color: '#ff6a3d', skills: ['React', 'JavaScript', 'HTML', 'CSS'] },
  { category: 'Backend', color: '#1d3557', skills: ['Node.js', 'Express'] },
  { category: 'Database', color: '#2a9d8f', skills: ['MongoDB', 'MySQL'] },
  { category: 'Tools & Practices', color: '#e76f51', skills: ['Git', 'VS Code', 'Vite'] },
]

function Home() {
  const [showFact, setShowFact] = useState(false)

  return (
    <div className="page">
      <Header name="Bhumi" tagline=" I turn ideas into websites." />

      <button className="btn btn-secondary fact-toggle" onClick={() => setShowFact(!showFact)}>
        {showFact ? 'Hide quick fact' : 'Show a quick fact about me'}
      </button>
      {showFact && (
        <p className="quick-fact">
          I'm currently in Semester 5 at CHARUSAT, focused on full-stack web development.
        </p>
      )}

      <div className="bento-grid">
        <About />
        <Skills skillList={skillList} />
      </div>
    </div>
  )
}
```

```
36.
37.export default Home
```

Supplementary Problem:

1. **Add a 404 Not Found route that renders a custom error component for unknown paths.**

NotFound.jsx

```
4. import { Link } from 'react-router-dom'
5.
6. function NotFound() {
7.   return (
8.     <div className="page not-found-page">
9.       <h1>404</h1>
10.      <p className="card-text">Oops — this page doesn't exist.</p>
11.      <Link to="/" className="btn btn-primary">Back to Home</Link>
12.    </div>
13.  )
14.}
15.
16.export default NotFound
```

2. **Store the contact form input in state and display a live character count below the input. in this**

Projects.jsx

```
50.const projects = [
51.  {
52.    id: 1,
53.    title: 'Personal Expense Tracker (UI/UX)',
54.    tech: ['Figma'],
55.    points: [
56.      'Designed user-friendly interface and wireframes',
57.      'Focused on clean and intuitive user experience',
58.    ],
59.  },
60.  {
61.
62.    title: 'Beauties & Bows – E-commerce Website',
63.    tech: ['HTML', 'CSS', 'JavaScript'],
64.    points: [
65.      'Developed a responsive front-end website for a fashion brand',
66.      'Implemented product display and user-friendly navigation',
67.    ],
```

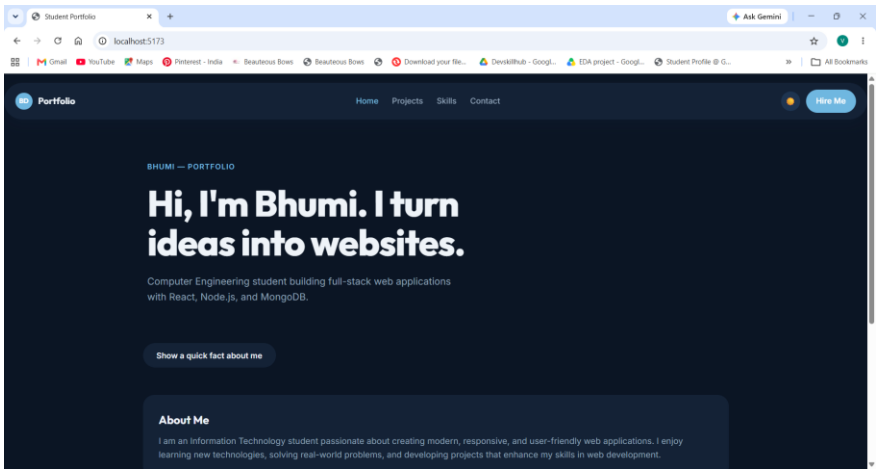
```
  },
  {

    title: 'Note-Taking Web Application',
    tech: ['PHP', 'MySQL'],
    points: [
      'Created a web app to add, edit, and delete notes',
      'Integrated backend with database for data management',
    ],
  },
  {
    title: 'Shake & Bake Website',
    tech: ['WordPress'],
    points: [
      'Developed and customized a business website using WordPress',
      'Managed layout, design, and content',
    ],
  },
]

function Projects() {
  return (
    <div className="page">
      <div className="bento-grid">
        {projects.map((p, index) => (
          <div key={p.id} className="bento-card span-3">
            <p className="card-number">{String(index + 1).padStart(2,
'0')}</p>
            <h2 className="card-title">{p.title}</h2>
            <ul className="project-points">
              {p.points.map((point, i) => (
                <li key={i}>{point}</li>
              ))}
                  </ul>
                  <div className="card-tech">
                    {p.tech.map((t) => (
                      <span key={t} className="tech-tag">{t}</span>
                    ))}
                  </div>
                </div>
              ))}
            </div>
          </div>
        )
      }

      export default Projects
```

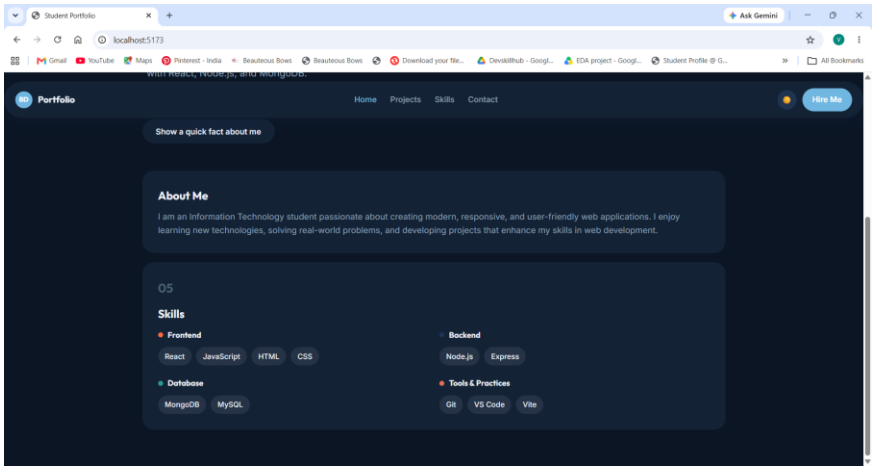
Header.jsx

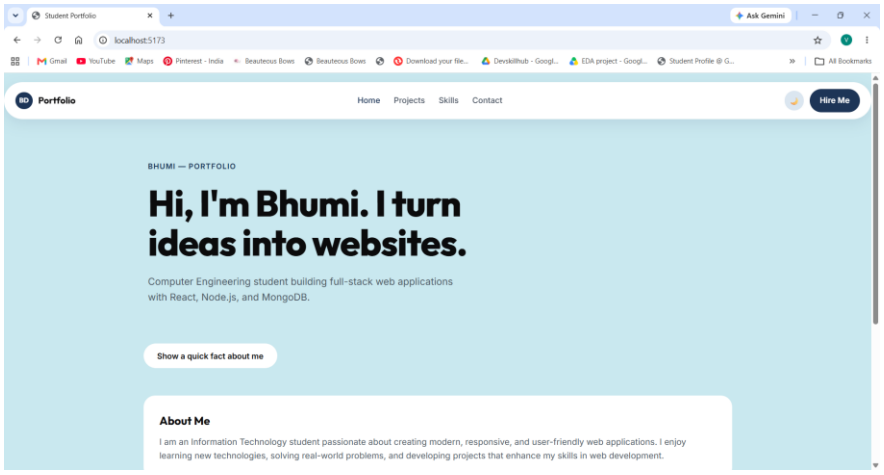
```jsx
function Header({ name, tagline }) {
  return (
    <header className="hero">
      <p className="hero-eyebrow">{name} &mdash; Portfolio</p>
      <h1 className="hero-title">Hi, I'm {name}. {tagline}</h1>
      <p className="hero-subtitle">
        Information Technology student building full-stack web applications
        with React, Node.js, and MongoDB.
      </p>
    </header>
  )
}

export default Header
```

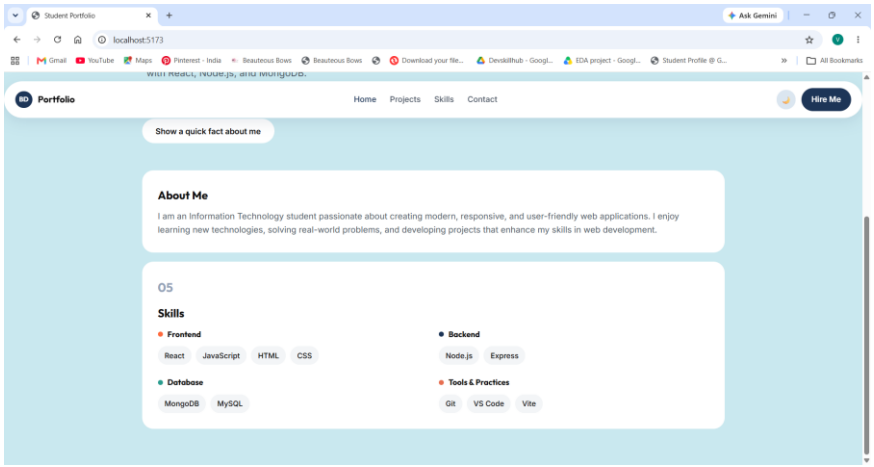
7. Test navigation across all 3 routes and confirm no page reload occurs

Output:






Student Portfolio
localhost:5173
Ask Gemini
Portfolio
Home
Projects
Skills
Contact
Hire Me
BHUMI — PORTFOLIO
Hi, I'm Bhumi. I turn ideas into websites.
Computer Engineering student building full-stack web applications with React, Node.js, and MongoDB.
Show a quick fact about me
About Me
I am an Information Technology student passionate about creating modern, responsive, and user-friendly web applications. I enjoy learning new technologies, solving real-world problems, and developing projects that enhance my skills in web development.


Student Portfolio
localhost:5173
Ask Gemini
Portfolio
Home
Projects
Skills
Contact
Hire Me
Show a quick fact about me
About Me
I am an Information Technology student passionate about creating modern, responsive, and user-friendly web applications. I enjoy learning new technologies, solving real-world problems, and developing projects that enhance my skills in web development.
05
Skills
Frontend
React
JavaScript
HTML
CSS
Backend
Node.js
Express
Database
MongoDB
MySQL
Tools & Practices
Git
VS Code
Vite

Practical -3

Problem Definition

• Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.

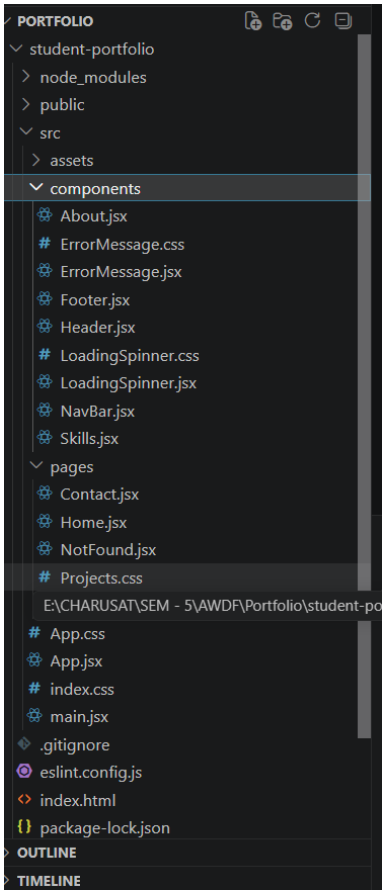
• Use the Projects page built in Practical 2 as the integration point.

• Implement a loading spinner component shown while the request is in progress.

• Implement an error message component shown if the API call fails.

• Render at least the repository name and URL for each item returned.

Folder Structure:



Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.

Projects.jsx

```
import { useState, useEffect } from "react";
import LoadingSpinner from "../components/LoadingSpinner";
import ErrorMessage from "../components/ErrorMessage";

const GITHUB_USERNAME = "https://github.com/Bhumi3009";

function Projects() {
  const [repos, setRepos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [search, setSearch] = useState("");

  const fetchRepos = () => {
    setLoading(true);
    setError(null);

    fetch(`https://api.github.com/users/Bhumi3009/repos`)
      .then((res) => {
        if (!res.ok) {
          throw new Error(`Request failed with status ${res.status}`);
        }
        return res.json();
      })
      .then((data) => setRepos(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
```

```
};

useEffect(() => {
  fetchRepos();
}, []);

const filteredRepos = repos.filter((repo) =>
  repo.name.toLowerCase().includes(search.toLowerCase())
);

if (loading) return <LoadingSpinner />;
if (error) return <ErrorMessage message={error} onRetry={fetchRepos} />;

return (
  <div className="page">
    <input
      type="text"
      placeholder="Search repositories..."
      value={search}
      onChange={(e) => setSearch(e.target.value)}
      className="projects-search"
    />

    <div className="bento-grid">
      {filteredRepos.map((repo, index) => (
        <div key={repo.id} className="bento-card span-3">
```

```
            <p className="card-number">{String(index + 1).padStart(2, "0")}</p>
            <h2 className="card-title">
              <a href={repo.html_url} target="_blank" rel="noreferrer">
                {repo.name}
              </a>
            </h2>
            <p className="project-points">
              {repo.description || "No description provided."}
            </p>
            <div className="card-tech">
              <span className="tech-tag">⭐ {repo.stargazers_count}</span>
              {repo.language && (
                <span className="tech-tag">{repo.language}</span>
              )}
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}

export default Projects;
```

LoadingSpinner.jsx

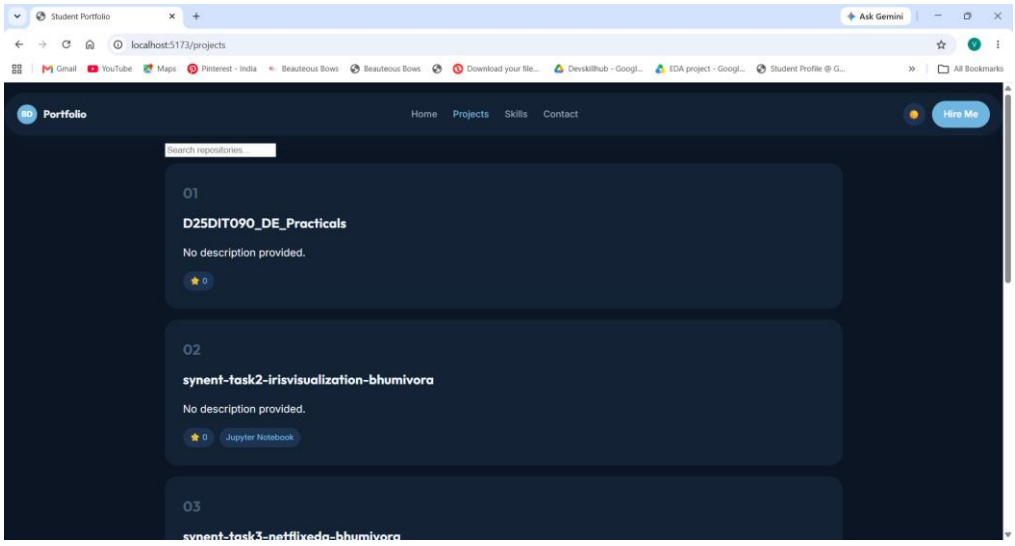
```
1. import "./LoadingSpinner.css";
2.
3. export default function LoadingSpinner({ label = "Loading
   repositories..." }) {
```

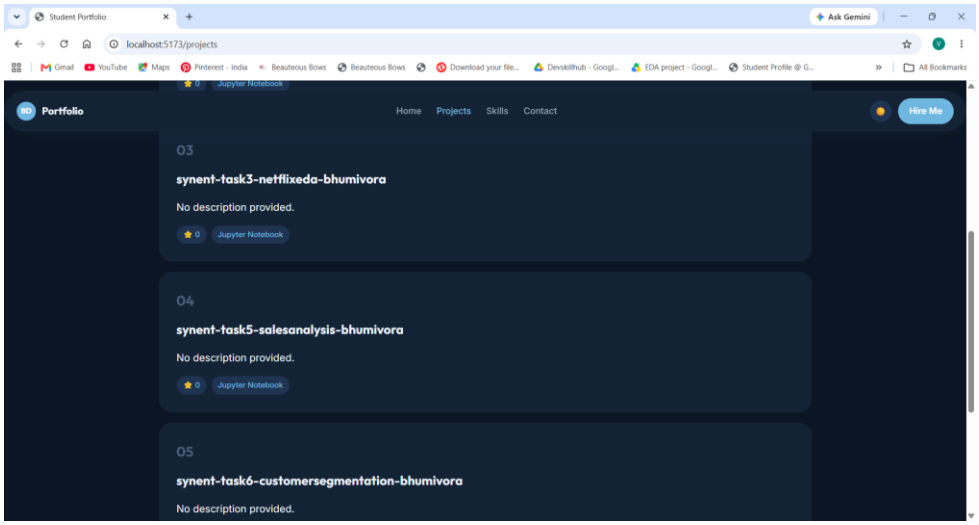
```
  return (
    <div className="spinner-wrap">
      <div className="spinner"></div>
      <p className="spinner-label">{label}</p>
    </div>
  );
}
```

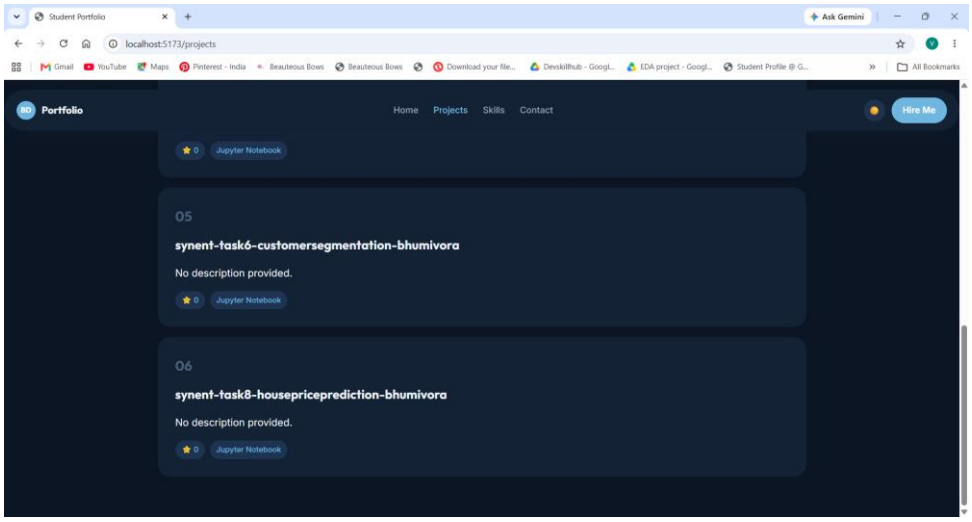
ErrorMessage.jsx

```
import "./ErrorMessage.css";

export default function ErrorMessage({ message, onRetry }) {
  return (
    <div className="error-wrap">
      <p className="error-title">Something went wrong</p>
      <p className="error-detail">{message}</p>
      {onRetry && (
        <button className="retry-btn" onClick={onRetry}>
          Retry
        </button>
      )}
    </div>
  );
}
```

Output:




Student Portfolio
Ask Gemini
localhost:5173/projects
Portfolio
Home
Projects
Skills
Contact
Hire Me
03
synent-task3-netflixeda-bhumivora
No description provided.
Jupyter Notebook
04
synent-task5-salesanalysis-bhumivora
No description provided.
Jupyter Notebook
05
synent-task6-customersegmentation-bhumivora
No description provided.


Student Portfolio
Ask Gemini
localhost:5173/projects
Portfolio
Home
Projects
Skills
Contact
Hire Me
Jupyter Notebook
05
synent-task6-customersegmentation-bhumivora
No description provided.
Jupyter Notebook
06
synent-task8-housepriceprediction-bhumivora
No description provided.
Jupyter Notebook

Practical - 4

Problem Definition

• Build a Node/Express backend for a Task Management system.

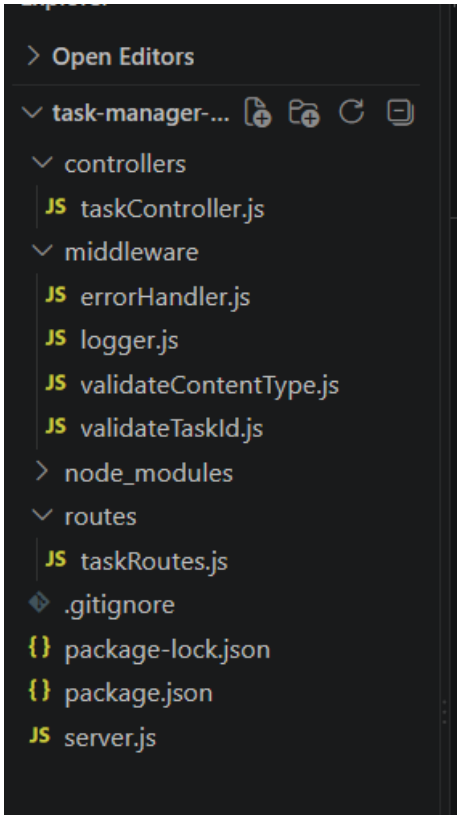
• Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).

• Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.

• Apply a global error handling middleware as the last middleware in the pipeline.

• Use correct HTTP status codes (200, 201, 404, 500) for each response.

Folder Structure:



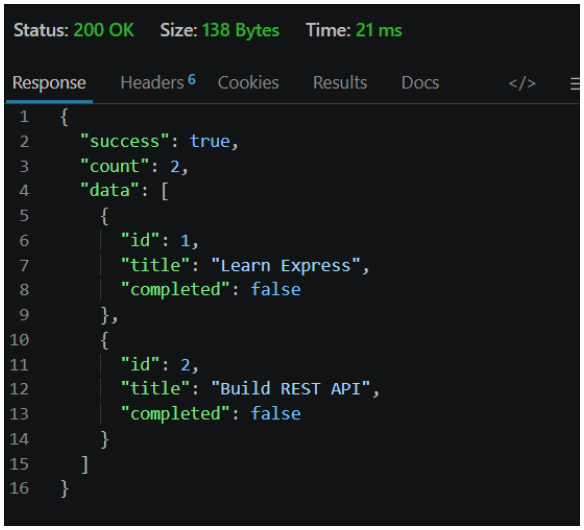
controllers/taskController.js

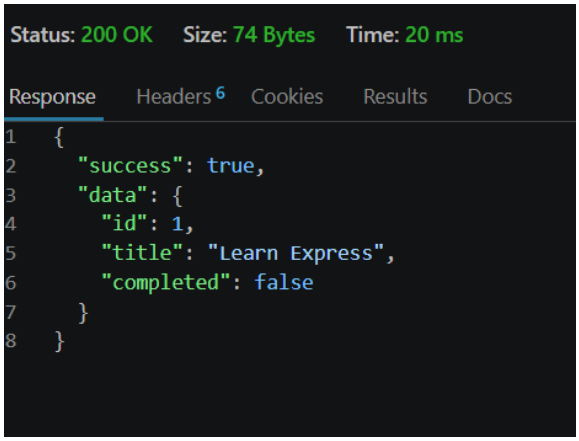
```
let tasks = [
  { id: 1, title: 'Learn Express', completed: false },
```

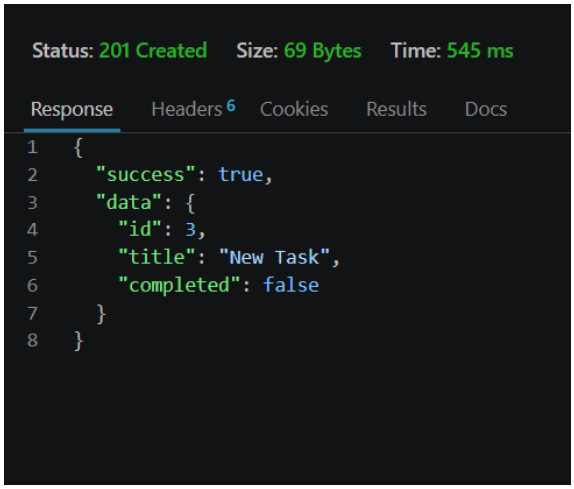
```
  { id: 2, title: 'Build REST API', completed: false }
];
let nextId = 3;

const getAllTasks = (req, res) => {
  res.status(200).json({ success: true, count: tasks.length, data: tasks });
};

const getTaskById = (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) {
    return res.status(404).json({ success: false, error: 'Task not found' });
  }
  res.status(200).json({ success: true, data: task });
};

const createTask = (req, res) => {
  const { title, completed } = req.body;
  if (!title) {
    return res.status(400).json({ success: false, error: 'Title is required' });
  }
  const newTask = { id: nextId++, title, completed: completed || false };
  tasks.push(newTask);
  res.status(201).json({ success: true, data: newTask });
};
```

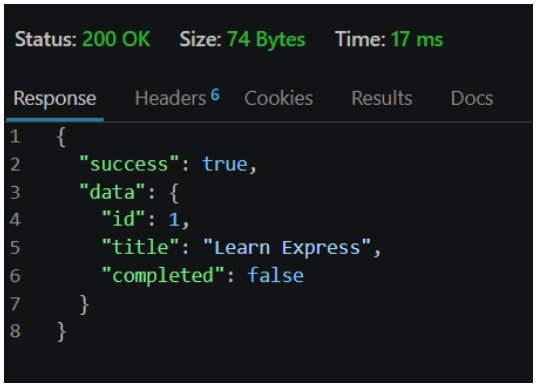
```
const updateTask = (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) {
    return res.status(404).json({ success: false, error: 'Task not found' });
  }
  const { title, completed } = req.body;
  if (title !== undefined) task.title = title;
  if (completed !== undefined) task.completed = completed;
  res.status(200).json({ success: true, data: task });
};

const deleteTask = (req, res) => {
  const index = tasks.findIndex(t => t.id === parseInt(req.params.id));
  if (index === -1) {
    return res.status(404).json({ success: false, error: 'Task not found' });
  }
  tasks.splice(index, 1);
  res.status(200).json({ success: true, message: 'Task deleted successfully' });
};

module.exports = { getAllTasks, getTaskById, createTask, updateTask, deleteTask
};
```

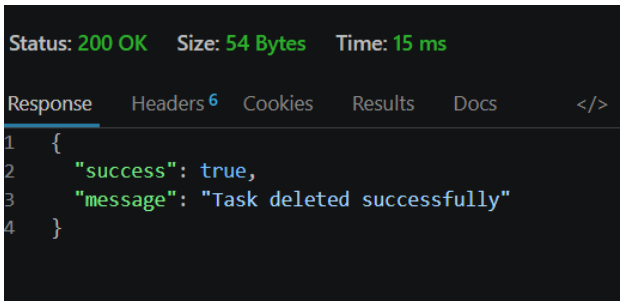
Output:


Status: 200 OK Size: 138 Bytes Time: 21 ms
Response Headers 6 Cookies Results Docs
{
"success": true,
"count": 2,
"data": [
{
"id": 1,
"title": "Learn Express",
"completed": false
},
{
"id": 2,
"title": "Build REST API",
"completed": false
}
]
}


Status: 200 OK Size: 74 Bytes Time: 20 ms
Response Headers 6 Cookies Results Docs
{
"success": true,
"data": {
"id": 1,
"title": "Learn Express",
"completed": false
}
}


Status: 201 Created Size: 69 Bytes Time: 545 ms
Response Headers 6 Cookies Results Docs
{
"success": true,
"data": {
"id": 3,
"title": "New Task",
"completed": false
}
}





middleware/logger.js

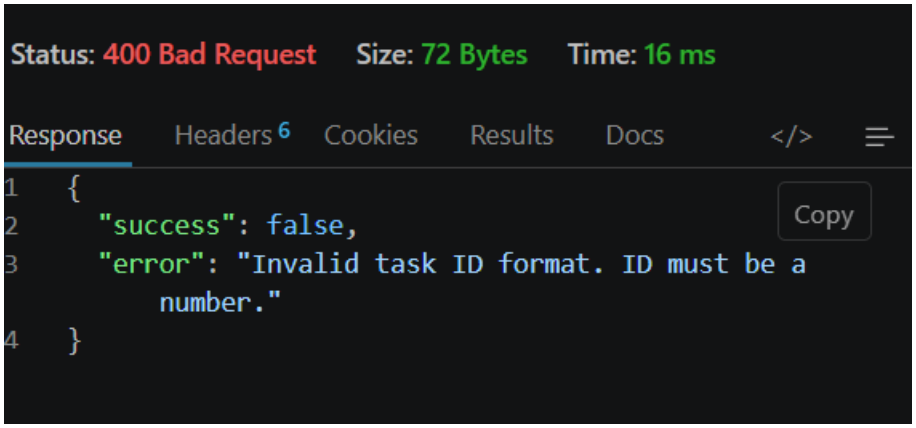
```
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
  next();
};

module.exports = logger;
```

middleware/validateTaskId.js

```
const validateTaskId = (req, res, next) => {
  const { id } = req.params;
  if (!/^\d+$/.test(id)) {
    return res.status(400).json({
      success: false,
```

```
    error: 'Invalid task ID format. ID must be a number.'
   });
  }
  next();
};
module.exports = validateTaskId;
```

Output:



middleware/errorHandler.js

```
const errorHandler = (err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    error: 'Something went wrong'
  });
};

module.exports = errorHandler;
```

routes/taskRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  getAllTasks,
  getTaskById,
  createTask,
  updateTask,
  deleteTask
} = require('../controllers/taskController');
const validateContentType = require('../middleware/validateContentType');
const validateTaskId = require('../middleware/validateTaskId');

router.get('/', getAllTasks);
router.get('/:id', validateTaskId, getTaskById);
router.post('/', validateContentType, createTask);
router.put('/:id', validateContentType, validateTaskId, updateTask);
router.delete('/:id', validateTaskId, deleteTask);

module.exports = router;
```

server.js

```
const express = require('express');
const taskRoutes = require('./routes/taskRoutes');
const logger = require('./middleware/logger');
const errorHandler = require('./middleware/errorHandler');
```

```
const app = express();
const PORT = 5000;

app.use(express.json());
app.use(logger);

app.use('/tasks', taskRoutes);

app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: 'Route not found',
    path: req.originalUrl
  });
});
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Output:

```
PS C:\Users\Bhumi> cd "E:\CHARUSAT\SEM - 5\AWDF\task-manager-api"
PS E:\CHARUSAT\SEM - 5\AWDF\task-manager-api> node server.js
Server running on port 5000
```

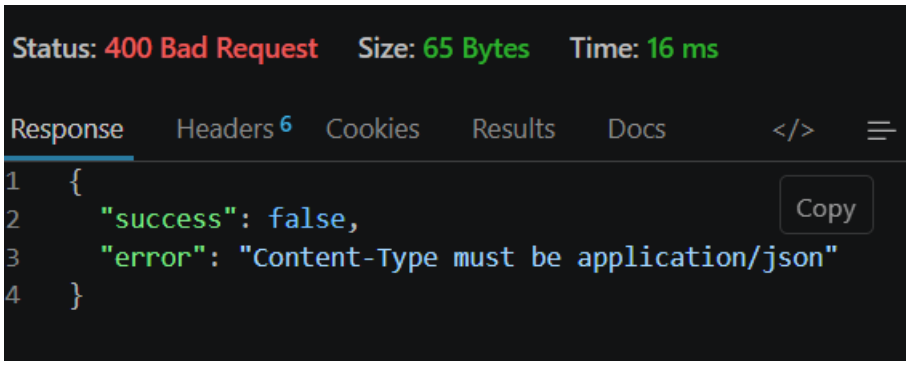
Supplementary Problem :

Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT.

middleware/validateContentType.js

```
const validateContentType = (req, res, next) => {
  if (req.method === 'POST' || req.method === 'PUT') {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      return res.status(400).json({
        success: false,
        error: 'Content-Type must be application/json'
      });
    }
  }
  next();
};

module.exports = validateContentType;
```

Output:

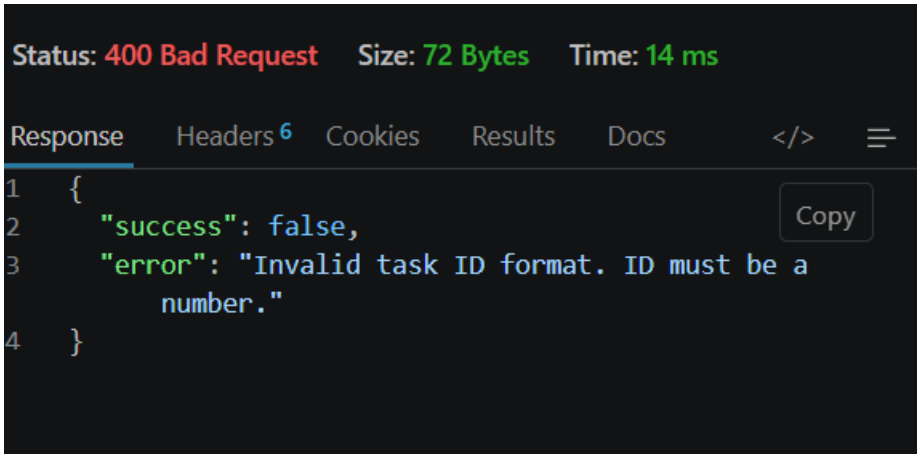


2. Add a route-specific middleware that validates the task ID format before reaching the controller.

middleware/validateTaskId.js

```
const validateTaskId = (req, res, next) => {
  const { id } = req.params;
  if (!/^\d+$/.test(id)) {
    return res.status(400).json({
      success: false,
      error: 'Invalid task ID format. ID must be a number.'
    });
  }
  next();
};

module.exports = validateTaskId;
```

Output:



3. Add a 404 handler for undefined routes that returns a structured JSON response.

server.js

```
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: 'Route not found',
    path: req.originalUrl
  });
});
```

Output:

Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

Practical 5

AIM:

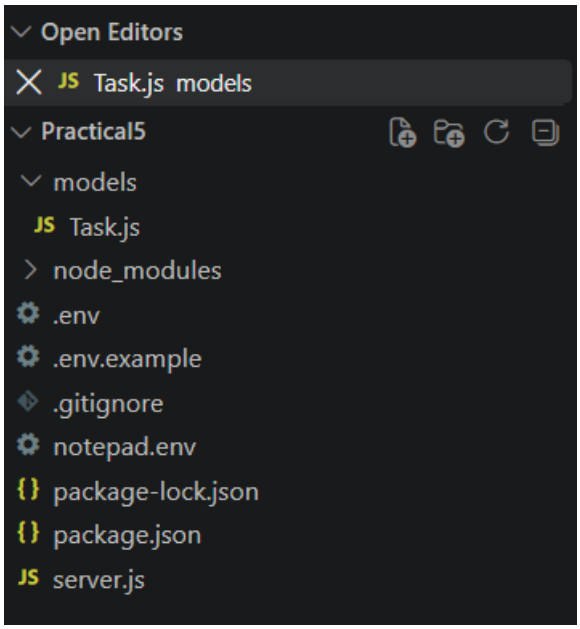
• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.

• Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).

• Replace the in-memory array from Practical 4 with real Mongoose model operations.

• Test all CRUD operations against the live database using Postman. • Ensure validation errors

Folder Structure:



server.js

```
/**
 * Practical 5 – MongoDB Integration and Schema Design with Mongoose
 *
 * Objective: Replace the in-memory array from Practical 4 with a real
```

```
 * MongoDB database accessed through the Mongoose ODM.
 *
 * ── Architecture ──────────────────────────────────────────────────────
 *   Express App
 *     └── Mongoose ODM
 *           └── MongoDB Database
 *                 └── tasks collection
 *                       { title, description, completed, priority, createdAt }
 *
 * ── What changed from Practical 4 ─────────────────────────────────────
 *   - Added dotenv to keep MONGO_URI out of source code
 *   - Added mongoose.connect() before starting the server
 *   - Replaced in-memory array operations with Mongoose model methods:
 *       Task.find()            ← was: tasks array
 *       Task.create()          ← was: tasks.push()
 *       Task.findByIdAndUpdate() ← was: tasks.find() + mutate
 *       Task.findByIdAndDelete() ← was: tasks.splice()
 *   - Added GET /tasks/:id endpoint with proper 404 handling (supplementary)
 *   - Validation errors from Mongoose are caught and returned as clean JSON
 *     (not the raw Mongoose ValidationError object)
 *   - Every route is wrapped in try/catch and passes errors to next(err)
 *
 * ── Setup ─────────────────────────────────────────────────────────────
 –
```

```
 *   1. Copy .env.example to .env and set MONGO_URI
 *   2. npm install
 *   3. node server.js  (or: npm run dev)
 *   4. Test with Postman on http://localhost:5000
 *
 * ── Routes ──────────────────────────────────────────────────────────
 *   GET    /tasks        → all tasks
 *   GET    /tasks/:id    → single task (404 if not found)
 *   POST   /tasks        → create task  (validates title)
 *   PUT    /tasks/:id    → update task  (404 if not found)
 *   DELETE /tasks/:id    → delete task  (404 if not found)
 */

const express  = require('express')
const mongoose = require('mongoose')
require('dotenv').config()         // loads .env into process.env

const Task = require('./models/Task')

const app  = express()
const PORT = process.env.PORT || 5000

/* ─────────────────────────────────────────────────
   Middleware Pipeline
──────────────────────────────────────────────────── */
```

```
// 1. Parse JSON bodies
app.use(express.json())

// 2. Request logger
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`)
  next()
})

// 3. Content-Type check for write operations
app.use((req, res, next) => {
  if (
    (req.method === 'POST' || req.method === 'PUT') &&
    req.headers['content-type'] !== 'application/json'
  ) {
    return res.status(400).json({
      error: 'Content-Type must be application/json for POST and PUT requests',
    })
  }
  next()
})

/* ————————————————————————————————————
   Helper: format Mongoose validation errors
   into a clean object instead of sending the
   raw ValidationError to the client.
```

```
──────────────────────────────────────────────────────── */
function formatValidationError(err) {
  const errors = {}
  Object.keys(err.errors).forEach((field) => {
    errors[field] = err.errors[field].message
  })
  return errors
}

/* ──────────────────────────────────────────────────────
   CRUD Routes
──────────────────────────────────────────────────────── */

// GET /tasks – return all tasks
app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 })
    res.status(200).json(tasks)
  } catch (err) {
    next(err)
  }
})

// GET /tasks/:id – return a single task (supplementary)
app.get('/tasks/:id', async (req, res, next) => {
  try {
```

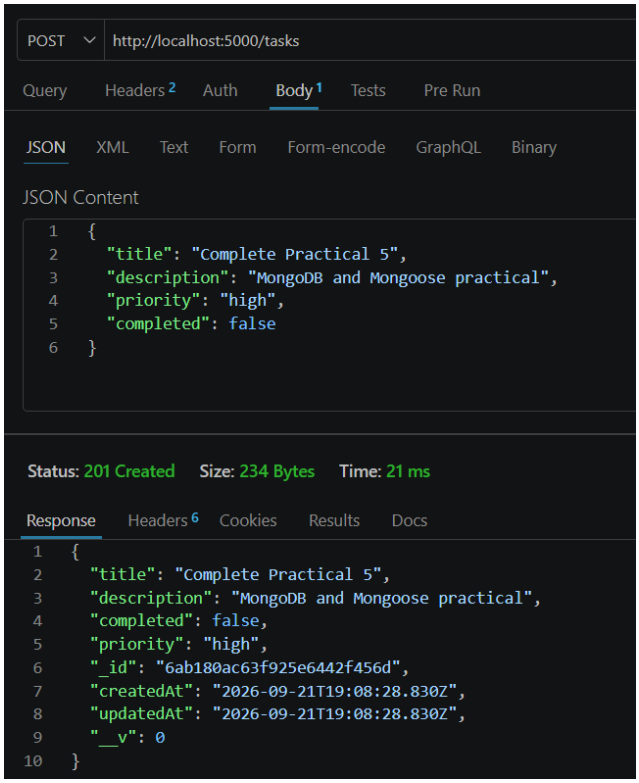
```
    const task = await Task.findById(req.params.id)

    if (!task) {
      return res.status(404).json({ error: 'Task not found' })
    }

    res.status(200).json(task)
  } catch (err) {
    // CastError means the :id was not a valid MongoDB ObjectId
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' })
    }
    next(err)
  }
})

// POST /tasks – create a new task
app.post('/tasks', async (req, res, next) => {
  try {
    const task = await Task.create({
      title:       req.body.title,
      description: req.body.description,
      completed:   req.body.completed,
      priority:    req.body.priority,
    })
```

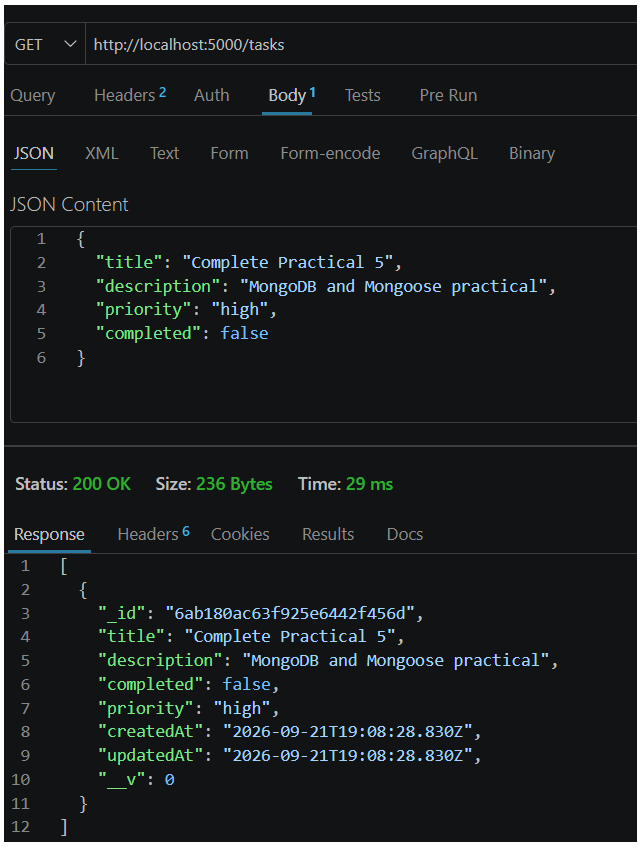
```
    res.status(201).json(task)
  } catch (err) {
    // Mongoose validation error → return structured field-level messages
    if (err.name === 'ValidationError') {
      return res.status(400).json({ errors: formatValidationError(err) })
    }
    next(err)
  }
})

// PUT /tasks/:id – update an existing task
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      {
        title:       req.body.title,
        description: req.body.description,
        completed:   req.body.completed,
        priority:    req.body.priority,
      },
      {
        new:         true,   // return the updated document, not the old one
        runValidators: true,  // run schema validators on update as well
      }
    )
```

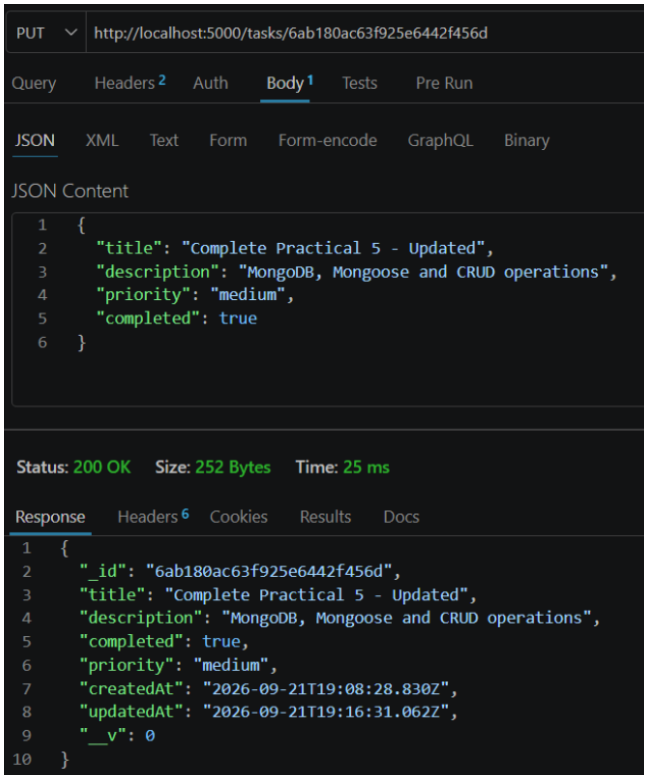
```
    if (!task) {
      return res.status(404).json({ error: 'Task not found' })
    }

    res.status(200).json(task)
  } catch (err) {
    if (err.name === 'ValidationError') {
      return res.status(400).json({ errors: formatValidationError(err) })
    }
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' })
    }
    next(err)
  }
})

// DELETE /tasks/:id – delete a task
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id)

    if (!task) {
      return res.status(404).json({ error: 'Task not found' })
    }
```

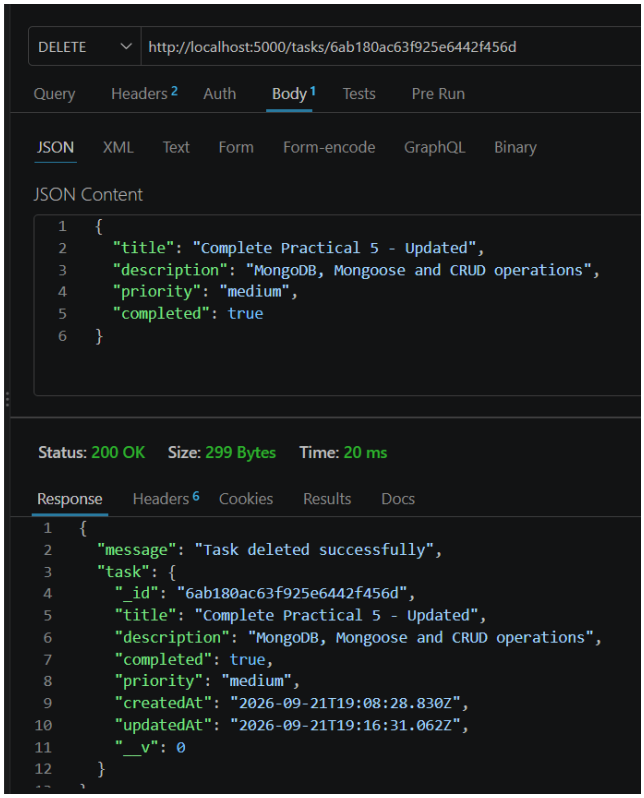
```
    res.status(200).json({ message: 'Task deleted successfully', task })
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' })
    }
    next(err)
  }
})

/* ————————————————————————————————————
   404 Handler – undefined routes
———————————————————————————————————— */
app.use((req, res) => {
  res.status(404).json({ error: `Route ${req.method} ${req.url} not found` })
})

/* ————————————————————————————————————
   Global Error Handler (must be last, 4 params)
———————————————————————————————————— */
app.use((err, req, res, next) => {
  console.error(`[ERROR] ${err.stack}`)
  res.status(500).json({ error: 'Something went wrong on the server' })
})

/* ————————————————————————————————————
   Connect to MongoDB, then start the server.
```

```
   Server only starts AFTER a successful DB
   connection — avoids accepting requests before
   the database is ready.
──────────────────────────────────────────────────── */
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully')
    app.listen(PORT, () => {
      console.log(`Server running on http://localhost:${PORT}`)
      console.log('Available routes:')
      console.log('  GET    /tasks')
      console.log('  GET    /tasks/:id')
      console.log('  POST   /tasks')
      console.log('  PUT    /tasks/:id')
      console.log('  DELETE /tasks/:id')
    })
  })
  .catch((err) => {
    console.error('MongoDB connection failed:', err.message)
    console.error('Make sure your MONGO_URI in .env is correct.')
    process.exit(1)
  })
```


POST
http://localhost:5000/tasks
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
{
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"priority": "high",
"completed": false
}
Status: 201 Created
Size: 234 Bytes
Time: 21 ms
Response
Headers 6
Cookies
Results
Docs
{
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"completed": false,
"priority": "high",
"_id": "6ab180ac63f925e6442f456d",
"createdAt": "2026-09-21T19:08:28.830Z",
"updatedAt": "2026-09-21T19:08:28.830Z",
"__v": 0
}


GET
http://localhost:5000/tasks
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
{
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"priority": "high",
"completed": false
}
Status: 200 OK
Size: 236 Bytes
Time: 29 ms
Response
Headers 6
Cookies
Results
Docs
[
{
"_id": "6ab180ac63f925e6442f456d",
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"completed": false,
"priority": "high",
"createdAt": "2026-09-21T19:08:28.830Z",
"updatedAt": "2026-09-21T19:08:28.830Z",
"__v": 0
}
]





.env

MONGO_URI=mongodb://127.0.0.1:27017/Practical5

.env.example

MONGO_URI=mongodb://localhost:27017/taskdb

PORT=5000

package.json

```
{
  "name": "practical-5-mongodb-mongoose",
  "version": "1.0.0",
  "description": "Practical 5 – MongoDB Integration and Schema Design with Mongoose",
  "main": "server.js",
  "type": "commonjs",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "mongoose": "^8.0.3"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

.gitignore

```
node_modules/
.env
```

Supplementary Problem:

• Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high'].

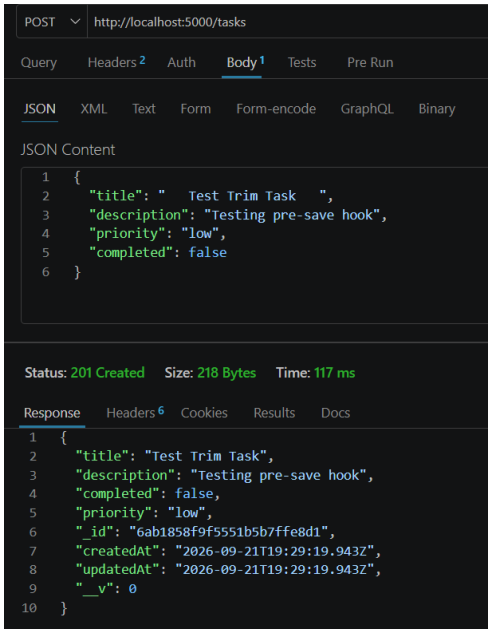
```
priority: {
   type: String,
   enum: ['low', 'medium', 'high'],
   default: 'medium'
}
```

• Add a pre-save hook that automatically trims whitespace from the title field.

Task.js

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
   {
      title: {
         type: String,
         required: [true, 'Title is required'],
         trim: true
      },

      description: {
         type: String,
         trim: true
      },

      completed: {
```

```
        type: Boolean,
        default: false
      },

      priority: {
        type: String,
        enum: ['low', 'medium', 'high'],
        default: 'medium'
      }
    },
    {
      timestamps: true
    }
);

// Pre-save hook: trim whitespace from title
taskSchema.pre('save', function (next) {
    if (this.isModified('title') && this.title) {
      this.title = this.title.trim();
    }
    next();
});

const Task = mongoose.model('Task', taskSchema);

module.exports = Task;
```

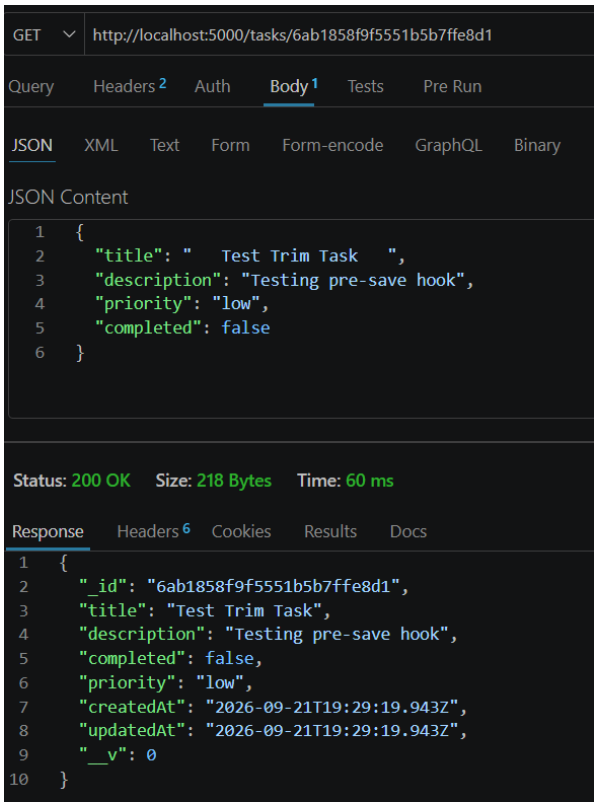


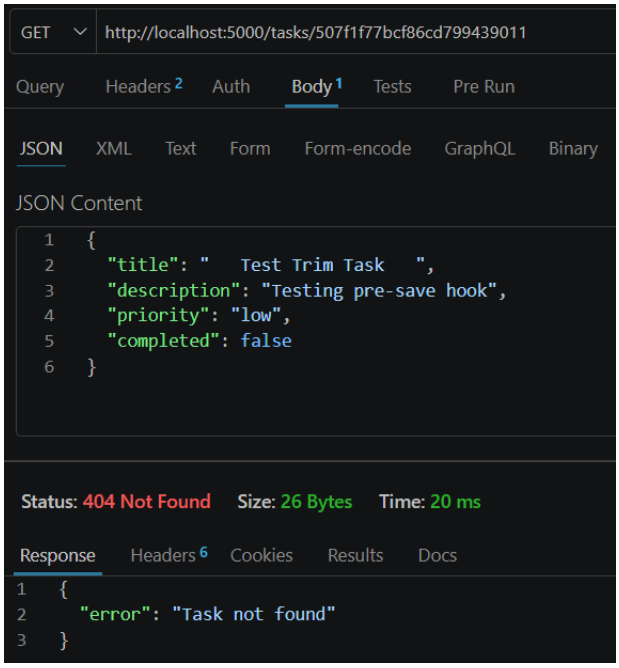
• Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist.

Server.js

```
// GET /tasks/:id - Get a single task by ID
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;

    // Check whether the ID is a valid MongoDB ObjectId
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        error: 'Invalid task ID'
      });
    }

    const task = await Task.findById(id);
```

```
        // Task ID is valid but task does not exist
        if (!task) {
            return res.status(404).json({
                error: 'Task not found'
            });
        }

        // Task found
        res.status(200).json(task);

    } catch (err) {
        next(err);
    }
});
```





Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

Practical 6

Problem Definition

Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.

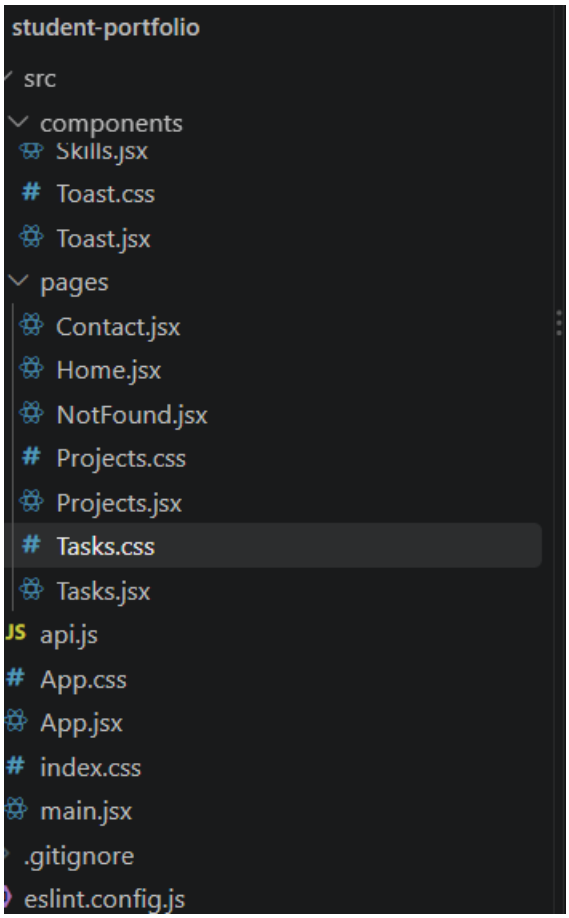
• Replace the GitHub repo data from Practical 3 with task data from your own backend.

• The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.

• All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.

• Handle loading and error states for every API interaction, not just the initial fetch.

Folder Structure:



backend/Practical5/server.js

```
/**
 * Practical 5 – MongoDB Integration and Schema Design with Mongoose
 *
 * Objective: Replace the in-memory array from Practical 4 with a real
 * MongoDB database accessed through the Mongoose ODM.
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

// Mongoose model
const Task = require('./models/Task');

const app = express();
const PORT = process.env.PORT || 5000;

/* ─────────────────────────────────────────────────
   Middleware Pipeline
──────────────────────────────────────────────────── */

// 1. Allow React frontend to access this backend
app.use(cors());
```

```
// 2. Parse JSON request bodies
app.use(express.json());

// 3. Request logger
app.use((req, res, next) => {
  console.log(
    `[${new Date().toISOString()}] ${req.method} ${req.url}`
  );
  next();
});

// 4. Content-Type check for write operations
app.use((req, res, next) => {
  if (
    (req.method === 'POST' || req.method === 'PUT') &&
    req.headers['content-type'] !== 'application/json'
  ) {
    return res.status(400).json({
      error: 'Content-Type must be application/json for POST and PUT
requests'
    });
  }

  next();
});

/* ————————————————————————————————————
```

```
   Helper: Format Mongoose validation errors
_____________________________________________________ */

function formatValidationError(err) {
   const errors = {};

   Object.keys(err.errors).forEach((field) => {
      errors[field] = err.errors[field].message;
   });

   return errors;
}

/* _____________________________________________________
   CRUD Routes
_____________________________________________________ */

// GET /tasks – return all tasks
app.get('/tasks', async (req, res, next) => {
   try {
      const tasks = await Task.find().sort({ createdAt: -1 });

      res.status(200).json(tasks);
   } catch (err) {
      next(err);
   }
```

```
});

// GET /tasks/:id – return a single task
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);

    if (!task) {
      return res.status(404).json({
        error: 'Task not found'
      });
    }

    res.status(200).json(task);
  } catch (err) {

    // Invalid MongoDB ObjectId
    if (err.name === 'CastError') {
      return res.status(400).json({
        error: 'Invalid task ID format'
      });
    }

    next(err);
  }
});
```

```
// POST /tasks – create a new task
app.post('/tasks', async (req, res, next) => {
  try {
    const task = await Task.create({
      title: req.body.title,
      description: req.body.description,
      completed: req.body.completed,
      priority: req.body.priority
    });

    res.status(201).json(task);
  } catch (err) {

    // Mongoose validation error
    if (err.name === 'ValidationError') {
      return res.status(400).json({
        errors: formatValidationError(err)
      });
    }

    next(err);
  }
});

// PUT /tasks/:id – update an existing task
```

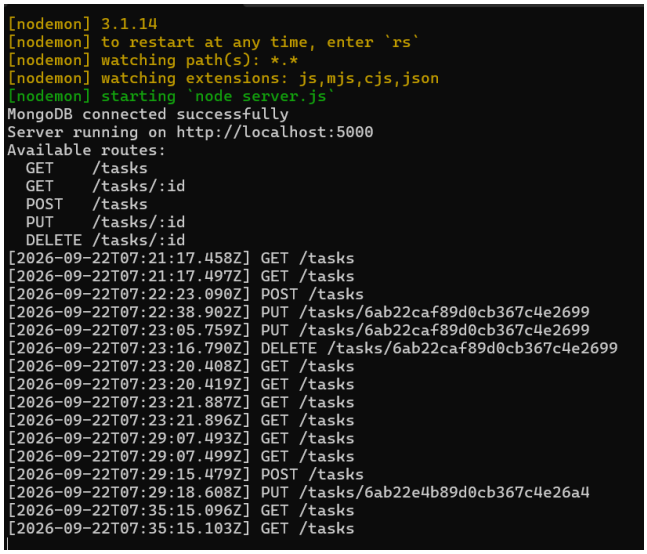
```
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      {
        title: req.body.title,
        description: req.body.description,
        completed: req.body.completed,
        priority: req.body.priority
      },
      {
        new: true,
        runValidators: true
      }
    );

    if (!task) {
      return res.status(404).json({
        error: 'Task not found'
      });
    }

    res.status(200).json(task);
  } catch (err) {

    // Mongoose validation error
```

```
      if (err.name === 'ValidationError') {
        return res.status(400).json({
          errors: formatValidationError(err)
        });
      }

      // Invalid MongoDB ObjectId
      if (err.name === 'CastError') {
        return res.status(400).json({
          error: 'Invalid task ID format'
        });
      }

      next(err);
    }
});

// DELETE /tasks/:id – delete a task
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);

    if (!task) {
      return res.status(404).json({
        error: 'Task not found'
      });
```

```
        }

        res.status(200).json({
            message: 'Task deleted successfully',
            task
        });
    } catch (err) {

        // Invalid MongoDB ObjectId
        if (err.name === 'CastError') {
            return res.status(400).json({
                error: 'Invalid task ID format'
            });
        }

        next(err);
    }
});

/* ————————————————————————————————————
   404 Handler – undefined routes
———————————————————————————————————— */

app.use((req, res) => {
    res.status(404).json({
        error: `Route ${req.method} ${req.url} not found`
```

```
    });
});

/* ————————————————————————————————————
   Global Error Handler
   Must be the last middleware
———————————————————————————————————— */

app.use((err, req, res, next) => {
    console.error(`[ERROR] ${err.stack}`);

    res.status(500).json({
        error: 'Something went wrong on the server'
    });
});

/* ————————————————————————————————————
   Connect to MongoDB, then start the server
———————————————————————————————————— */

mongoose
    .connect(process.env.MONGO_URI)
    .then(() => {
        console.log('MongoDB connected successfully');

        app.listen(PORT, () => {
```

```
        console.log(`Server running on http://localhost:${PORT}`);

        console.log('Available routes:');
        console.log('  GET    /tasks');
        console.log('  GET    /tasks/:id');
        console.log('  POST   /tasks');
        console.log('  PUT    /tasks/:id');
        console.log('  DELETE /tasks/:id');
    });
})
.catch((err) => {
    console.error(
        'MongoDB connection failed:',
        err.message
    );

    console.error(
        'Make sure your MONGO_URI in .env is correct.'
    );

    process.exit(1);
});
```



frontend/src/api.js

```
const BASE_URL = "http://localhost:5000";


async function handleResponse(res) {
  if (!res.ok) {
    let message = `Request failed with status ${res.status}`;
    try {
      const data = await res.json();
      message = data.error || (data.errors && Object.values(data.errors).join(",
")) || message;
    } catch {
      // response had no JSON body — keep the default message
    }
    throw new Error(message);
  }
  return res.json();
}
```

```
export const getTasks = () =>
  fetch(`${BASE_URL}/tasks`).then(handleResponse);

export const createTask = (task) =>
  fetch(`${BASE_URL}/tasks`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(task),
  }).then(handleResponse);

export const updateTask = (id, task) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(task),
  }).then(handleResponse);

export const deleteTask = (id) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "DELETE",
  }).then(handleResponse);
```

frontend/src/pages/Tasks.jsx

```
import { useState, useEffect } from "react";
import { getTasks, createTask, updateTask, deleteTask } from "../api";
import LoadingSpinner from "../components/LoadingSpinner";
import ErrorMessage from "../components/ErrorMessage";
```

```
import Toast from "../components/Toast";
import "./Tasks.css";

const emptyForm = { title: "", description: "", priority: "medium" };

function Tasks() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [loadError, setLoadError] = useState(null);

  const [form, setForm] = useState(emptyForm);
  const [editingId, setEditingId] = useState(null);
  const [submitting, setSubmitting] = useState(false);
  const [actionError, setActionError] = useState(null);
  const [deletingId, setDeletingId] = useState(null);

  const [toasts, setToasts] = useState([]);

  const addToast = (message, type = "success") => {
    const id = Date.now() + Math.random();
    setToasts((prev) => [...prev, { id, message, type }]);
  };

  const dismissToast = (id) => {
    setToasts((prev) => prev.filter((t) => t.id !== id));
  };
```

```
  const loadTasks = () => {
    setLoading(true);
    setLoadError(null);
    getTasks()
      .then(setTasks)
      .catch((err) => setLoadError(err.message))
      .finally(() => setLoading(false));
  };

  useEffect(() => {
    loadTasks();
  }, []);

  const handleFormChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const startEdit = (task) => {
    setEditingId(task._id);
    setForm({
      title: task.title,
      description: task.description || "",
      priority: task.priority,
    });
    setActionError(null);
```

```
  };

  const cancelEdit = () => {
    setEditingId(null);
    setForm(emptyForm);
    setActionError(null);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.title.trim()) {
      setActionError("Title is required");
      return;
    }

    setActionError(null);

    // — Editing an existing task: wait for the server, same as before —
    if (editingId) {
      setSubmitting(true);
      const currentTask = tasks.find((t) => t._id === editingId);
      updateTask(editingId, { ...form, completed: currentTask?.completed ?? false
})
        .then((savedTask) => {
          setTasks((prev) => prev.map((t) => (t._id === editingId ? savedTask : t)));
          setForm(emptyForm);
          setEditingId(null);
```

```
        addToast("Task updated");
      })
      .catch((err) => {
        setActionError(err.message);
        addToast(`Update failed: ${err.message}`, "error");
      })
      .finally(() => setSubmitting(false));
    return;
  }

  // ── Creating a new task: show it immediately (optimistic), reconcile after ──
  const tempId = `temp-${Date.now()}`;
  const optimisticTask = {
    _id: tempId,
    title: form.title,
    description: form.description,
    priority: form.priority,
    completed: false,
    isPending: true,
  };

  setTasks((prev) => [optimisticTask, ...prev]);
  setForm(emptyForm);
  setSubmitting(true);

  createTask({ title: optimisticTask.title, description: optimisticTask.description,
priority: optimisticTask.priority })
```

```
    .then((savedTask) => {
      setTasks((prev) => prev.map((t) => (t._id === tempId ? savedTask : t)));
      addToast("Task added");
    })
    .catch((err) => {
      // Roll back the optimistic card since the server rejected it
      setTasks((prev) => prev.filter((t) => t._id !== tempId));
      setActionError(err.message);
      addToast(`Add failed: ${err.message}`, "error");
    })
    .finally(() => setSubmitting(false));
};

const toggleCompleted = (task) => {
  setActionError(null);
  updateTask(task._id, { ...task, completed: !task.completed })
    .then((updated) => {
      setTasks((prev) => prev.map((t) => (t._id === task._id ? updated : t)));
    })
    .catch((err) => {
      setActionError(err.message);
      addToast(`Update failed: ${err.message}`, "error");
    });
};

const handleDelete = (task) => {
```

```
  const confirmed = window.confirm(`Delete "${task.title}"? This can't be
undone.`);
  if (!confirmed) return;


  setDeletingId(task._id);
  setActionError(null);
  deleteTask(task._id)
    .then(() => {
      setTasks((prev) => prev.filter((t) => t._id !== task._id));
      if (editingId === task._id) cancelEdit();
      addToast("Task deleted");
    })
    .catch((err) => {
      setActionError(err.message);
      addToast(`Delete failed: ${err.message}`, "error");
    })
    .finally(() => setDeletingId(null));
 };


 if (loading) return <LoadingSpinner label="Loading tasks..." />;
 if (loadError) return <ErrorMessage message={loadError} onRetry={loadTasks}
/>;


 return (
  <div className="page">
    <Toast toasts={toasts} onDismiss={dismissToast} />
```

```
    <div className="bento-card span-3 contact-card">
      <h2 className="card-title">{editingId ? "Update Task" : "New Task"}</h2>
      <form className="contact-form" onSubmit={handleSubmit}>
        <label>
          Title
          <input
            type="text"
            name="title"
            value={form.title}
            onChange={handleFormChange}
            placeholder="Task title"
          />
        </label>
        <label>
          Description
          <textarea
            name="description"
            value={form.description}
            onChange={handleFormChange}
            placeholder="Details (optional)"
            rows={3}
          />
        </label>
        <label>
          Priority
          <select name="priority" value={form.priority}
onChange={handleFormChange}>
```

```
            <option value="low">Low</option>
            <option value="medium">Medium</option>
            <option value="high">High</option>
          </select>
        </label>

        {actionError && <p className="task-error">{actionError}</p>}

        <div className="task-form-actions">
          <button type="submit" className="btn btn-primary"
disabled={submitting}>
            {submitting ? "Saving..." : editingId ? "Update Task" : "Add Task"}
          </button>
          {editingId && (
            <button type="button" className="btn btn-secondary"
onClick={cancelEdit}>
              Cancel
            </button>
          )}
        </div>
      </form>
    </div>

    <div className="bento-grid task-grid">
      {tasks.length === 0 && <p className="tasks-empty">No tasks yet — add
one above.</p>}
```

```
      {tasks.map((task) => (
        <div
          key={task._id}
          className={`bento-card span-3 task-card ${task.completed ? "task-completed" : ""} ${task.isPending ? "task-pending" : ""}`}
        >
          <div className="task-card-header">
            <label className="task-checkbox">
              <input
                type="checkbox"
                checked={task.completed}
                disabled={task.isPending}
                onChange={() => toggleCompleted(task)}
              />
              <h2 className="card-title">{task.title}</h2>
            </label>
            <span className={`tech-tag priority-${task.priority}`}>
              {task.isPending ? "saving..." : task.priority}
            </span>
          </div>

          {task.description && <p className="project-points">{task.description}</p>}

          {!task.isPending && (
            <div className="task-actions">
```

```
            <button className="btn btn-secondary" onClick={() =>
startEdit(task)}>
              Edit
            </button>
            <button
              className="btn btn-secondary task-delete"
              onClick={() => handleDelete(task)}
              disabled={deletingId === task._id}
            >
              {deletingId === task._id ? "Deleting..." : "Delete"}
            </button>
          </div>
        )}
      </div>
    ))}
  </div>
 </div>
 );
}

export default Tasks;
```

frontend/src/components/Toast.jsx

```
import { useEffect } from "react";
import "./Toast.css";

export default function Toast({ toasts, onDismiss }) {
 return (
```

```
    <div className="toast-stack">
      {toasts.map((toast) => (
        <ToastItem key={toast.id} toast={toast} onDismiss={onDismiss} />
      ))}
    </div>
  );
}

function ToastItem({ toast, onDismiss }) {
  useEffect(() => {
    const timer = setTimeout(() => onDismiss(toast.id), 3000);
    return () => clearTimeout(timer);
  }, [toast.id, onDismiss]);

  return (
    <div className={`toast toast-${toast.type}`}>
      <span>{toast.message}</span>
      <button className="toast-close" onClick={() => onDismiss(toast.id)}>
        ×
      </button>
    </div>
  );
}
```

frontend/src/App.jsx

```
import { useState, useEffect } from 'react'
import { Routes, Route } from 'react-router-dom'
```

```
import NavBar from './components/NavBar'
import Home from './pages/Home'
import Projects from './pages/Projects'
import Tasks from './pages/Tasks'
import Contact from './pages/Contact'
import NotFound from './pages/NotFound'
import './App.css'

function App() {
  const [darkMode, setDarkMode] = useState(true)

  useEffect(() => {
    document.documentElement.className = darkMode ? 'dark' : 'light'
  }, [darkMode])

  return (
    <div className="app-shell">
      <NavBar darkMode={darkMode} setDarkMode={setDarkMode} />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/projects" element={<Projects />} />
        <Route path="/tasks" element={<Tasks />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
```

```
  )
}

export default App
```

frontend/src/components/NavBar.jsx

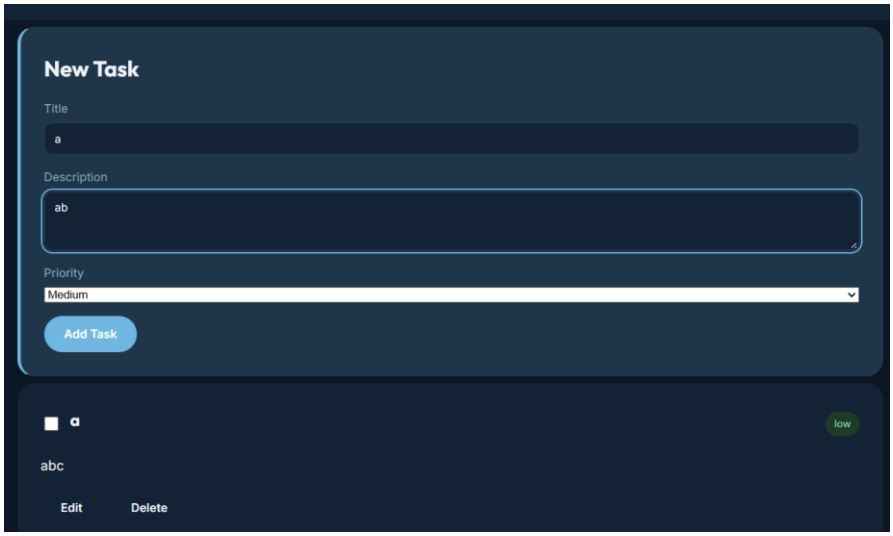
```
<NavLink to="/tasks" className={({ isActive }) => isActive ? 'active' :
''}>Tasks</NavLink>
```

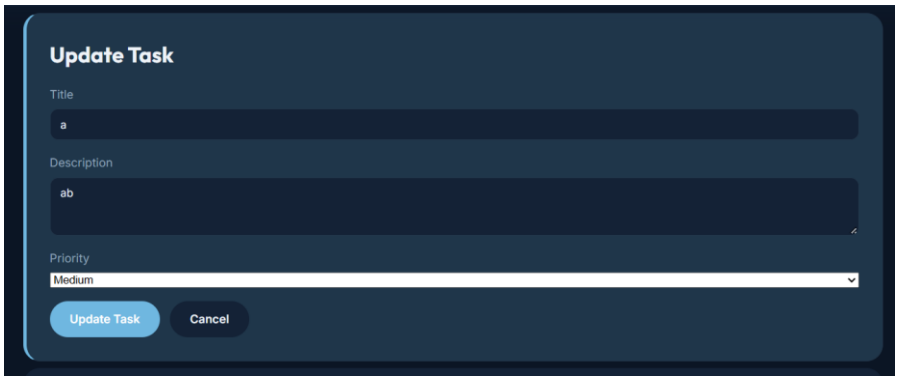
Supplementary Problems

• Add an optimistic UI update for task creation show the new task in the list immediately, before the server confirms.

Tasks.jsx:

```
const handleDelete = (task) => {
  const confirmed = window.confirm(`Delete "${task.title}"? This can't be
undone.`);
  if (!confirmed) return;

  setDeletingId(task._id);
  deleteTask(task._id)
    .then(() => {
      setTasks((prev) => prev.filter((t) => t._id !== task._id));
      addToast("Task deleted");
    })
    .catch((err) => addToast(`Delete failed: ${err.message}`, "error"))
    .finally(() => setDeletingId(null));
};
```

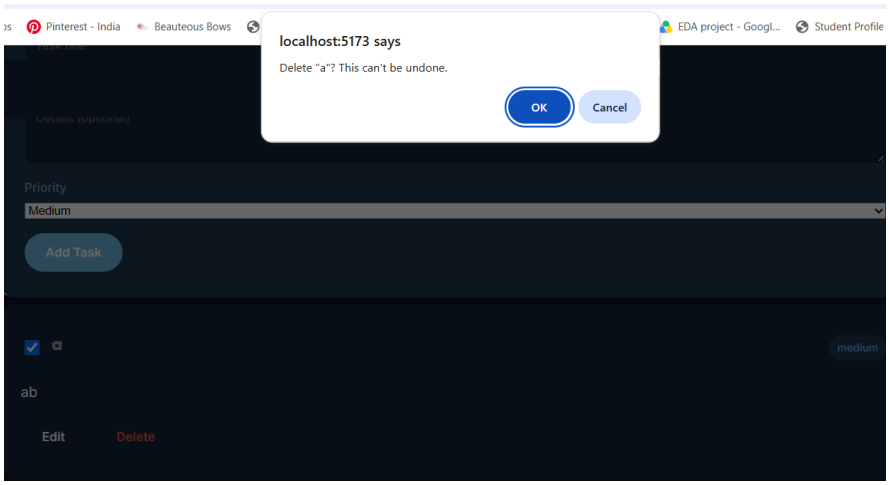




• **Add a confirmation dialog before deleting a task.**

```
// Show the task immediately with a temporary ID, before the server responds
const tempId = `temp-${Date.now()}`;
const optimisticTask = {
  _id: tempId,
  title: form.title,
  description: form.description,
  priority: form.priority,
  completed: false,
  isPending: true,
};
```

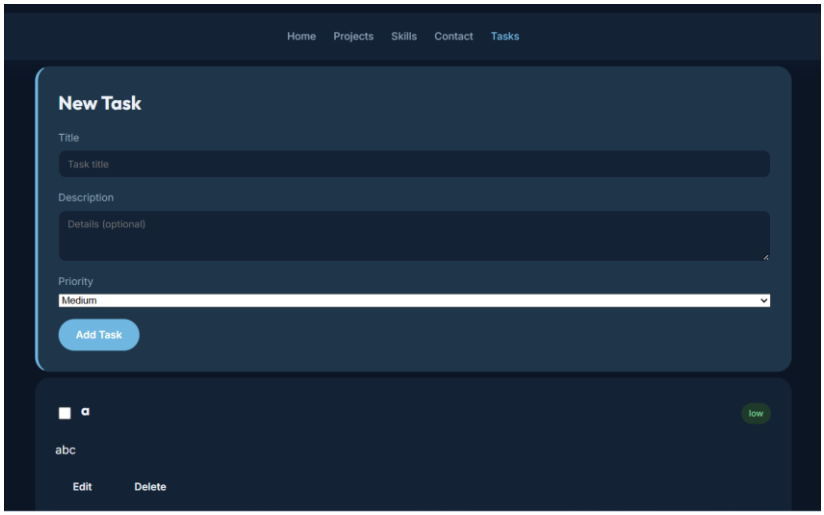
```
setTasks((prev) => [optimisticTask, ...prev]);
setForm(emptyForm);


createTask({ title: optimisticTask.title, description: optimisticTask.description,
priority: optimisticTask.priority })
  .then((savedTask) => {
    // Server confirmed — swap the temp card for the real saved task
    setTasks((prev) => prev.map((t) => (t._id === tempId ? savedTask : t)));
    addToast("Task added");
  })
  .catch((err) => {
    // Server rejected it — roll back the optimistic card
    setTasks((prev) => prev.filter((t) => t._id !== tempId));
    addToast(`Add failed: ${err.message}`, "error");
  });
```



• Add a toast/notification component that shows success or failure after each operation.

```
import { useEffect } from "react";
import "./Toast.css";
```

```
export default function Toast({ toasts, onDismiss }) {
  return (
    <div className="toast-stack">
      {toasts.map((toast) => (
        <ToastItem key={toast.id} toast={toast} onDismiss={onDismiss} />
      ))}
    </div>
  );
}

function ToastItem({ toast, onDismiss }) {
  useEffect(() => {
    const timer = setTimeout(() => onDismiss(toast.id), 3000);
    return () => clearTimeout(timer);
  }, [toast.id, onDismiss]);

  return (
    <div className={`toast toast-${toast.type}`}>
      <span>{toast.message}</span>
      <button className="toast-close" onClick={() =>
onDismiss(toast.id)}>×</button>
    </div>
  );
}
```



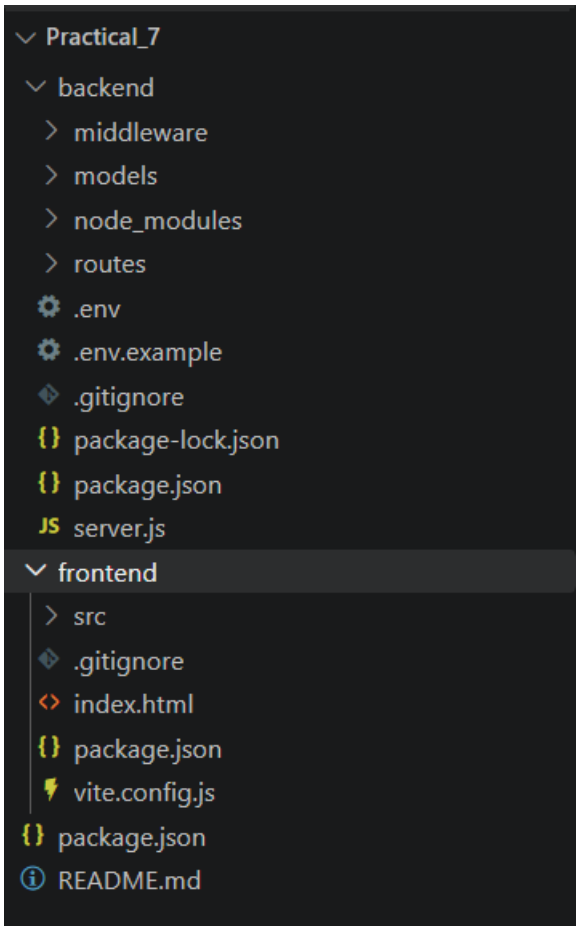
Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

Practical 7

Problem Definition

- Add user registration and login functionality to the Task Management application.
- Hash passwords using bcrypt before saving the User document to MongoDB.
- Implement JWT token generation on successful login, with a reasonable expiry (1 hour).
- Protect all task routes using an authentication middleware that verifies the JWT.
- Add server-side input validation that rejects malformed requests (e.g., missing title, invalid email, short password) before they reach the database.

Folder Structure:



backend/models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
 {
  name: {
   type: String,
   required: [true, 'Name is required'],
   trim: true
  },
  email: {
   type: String,
   required: [true, 'Email is required'],
   unique: true,
   lowercase: true,
   trim: true,
   match: [
    /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/,
    'Please enter a valid email address'
   ]
  },
  password: {
   type: String,
   required: [true, 'Password is required'],
   minlength: [6, 'Password must be at least 6 characters long']
  },
  createdAt: {
   type: Date,
   default: Date.now
```

```
    }
  },
  {
    timestamps: true
  }
);

// Method to sanitize output and exclude password from JSON serialization
userSchema.methods.toJSON = function () {
  const user = this.toObject();
  delete user.password;
  return user;
};

module.exports = mongoose.model('User', userSchema);
```

backend/middleware/auth.js

```
const jwt = require('jsonwebtoken');

/**
 * Authentication Middleware
 * Validates the JWT provided in the Authorization header: Bearer <token>
 * Attaches the decoded payload to req.user
 */
const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
```

```
    return res.status(401).json({
      error: 'Access denied. No token provided or malformed header. Expected: Bearer <token>'
    });
  }

  const token = authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      error: 'Access denied. Token missing.'
    });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // Contains { id: user._id, email: user.email, iat, exp }
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        error: 'Token has expired. Please log in again.'
      });
    }
    return res.status(401).json({
      error: 'Invalid token. Authorization failed.'
    });
  }
```

```
};

module.exports = authMiddleware;
```

backend/middleware/validate.js

```
/**
 * Validation Middleware Pipeline
 * Enforces server-side validation rules before requests reach the controllers.
 */

const emailRegex = /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/;

// Validate User Registration payload
const validateRegister = (req, res, next) => {
  const { name, email, password } = req.body;
  const errors = {};

  if (!name || typeof name !== 'string' || !name.trim()) {
    errors.name = 'Full name is required';
  }

  if (!email || typeof email !== 'string' || !email.trim()) {
    errors.email = 'Email address is required';
  } else if (!emailRegex.test(email.trim())) {
    errors.email = 'Please provide a valid email address';
  }
```

```
  if (!password || typeof password !== 'string') {
    errors.password = 'Password is required';
  } else if (password.length < 6) {
    errors.password = 'Password must be at least 6 characters long';
  }

  if (Object.keys(errors).length > 0) {
    return res.status(400).json({
      error: 'Validation failed',
      errors
    });
  }

  next();
};

// Validate User Login payload
const validateLogin = (req, res, next) => {
  const { email, password } = req.body;
  const errors = {};

  if (!email || typeof email !== 'string' || !email.trim()) {
    errors.email = 'Email is required';
  } else if (!emailRegex.test(email.trim())) {
    errors.email = 'Please provide a valid email address';
  }

  if (!password || typeof password !== 'string' || !password.trim()) {
```

```
    errors.password = 'Password is required';
  }

  if (Object.keys(errors).length > 0) {
    return res.status(400).json({
      error: 'Validation failed',
      errors
    });
  }

  next();
};

// Validate Task Creation and Update payload
const validateTask = (req, res, next) => {
  const { title, priority } = req.body;
  const errors = {};

  // For POST or when title is provided in PUT
  if (req.method === 'POST' || (req.method === 'PUT' && title !== undefined)) {
    if (!title || typeof title !== 'string' || !title.trim()) {
      errors.title = 'Task title is required and cannot be empty';
    }
  }

  if (priority !== undefined) {
    const validPriorities = ['low', 'medium', 'high'];
    if (!validPriorities.includes(priority)) {
```

```
    errors.priority = 'Priority must be one of: low, medium, high';
   }
  }

  if (Object.keys(errors).length > 0) {
   return res.status(400).json({
    error: 'Validation failed',
    errors
   });
  }

  next();
};

module.exports = {
  validateRegister,
  validateLogin,
  validateTask
};
```

backend/routes/auth.js

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const authMiddleware = require('../middleware/auth');
const { validateRegister, validateLogin } = require('../middleware/validate');
```

```
const router = express.Router();

/**
 * Helper to generate JWT token with 1 hour expiration
 */
const generateToken = (user) => {
  return jwt.sign(
    {
      id: user._id,
      email: user.email,
      name: user.name
    },
    process.env.JWT_SECRET,
    {
      expiresIn: process.env.JWT_EXPIRES_IN || '1h'
    }
  );
};

/**
 * @route   POST /auth/register
 * @desc    Register a new user, hash password with bcrypt, and return token
 * @access  Public
 */
router.post('/register', validateRegister, async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
```

```
  // Check if user already exists
  const existingUser = await User.findOne({ email: email.toLowerCase() });
  if (existingUser) {
    return res.status(400).json({
      error: 'Email already registered. Please login or use a different email.'
    });
  }

  // Step 3 in lab manual: Hash password using bcrypt
  const hashedPassword = await bcrypt.hash(password, 10);

  const user = await User.create({
    name: name.trim(),
    email: email.toLowerCase().trim(),
    password: hashedPassword
  });

  const token = generateToken(user);

  res.status(201).json({
    message: 'User registered successfully',
    token,
    user: {
      id: user._id,
      name: user.name,
      email: user.email,
      createdAt: user.createdAt
    }
```

```
    });
  } catch (err) {
    next(err);
  }
});

/**
 * @route   POST /auth/login
 * @desc    Authenticate user, verify password with bcrypt, return JWT token
 * @access  Public
 */
router.post('/login', validateLogin, async (req, res, next) => {
  try {
    const { email, password } = req.body;

    // Check if user exists
    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) {
      return res.status(401).json({
        error: 'Invalid credentials. User with this email does not exist.'
      });
    }

    // Step 4 in lab manual: Compare password with bcrypt.compare
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({
        error: 'Invalid credentials. Incorrect password.'
```

```
    });
  }

  const token = generateToken(user);

  res.status(200).json({
    message: 'Login successful',
    token,
    user: {
      id: user._id,
      name: user.name,
      email: user.email,
      createdAt: user.createdAt
    }
  });
 } catch (err) {
  next(err);
 }
});

/**
 * @route   GET /auth/me
 * @desc    Supplementary Problem 1: Return currently logged-in user profile
 * @access  Private (Protected with JWT Auth Middleware)
 */
router.get('/me', authMiddleware, async (req, res, next) => {
 try {
  const user = await User.findById(req.user.id).select('-password');
```

```
    if (!user) {
      return res.status(404).json({ error: 'User not found' });
    }
    res.status(200).json({
      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

backend/routes/tasks.js

```
const express = require('express');
const Task = require('../models/Task');
const authMiddleware = require('../middleware/auth');
const { validateTask } = require('../middleware/validate');

const router = express.Router();

// Step 6 in lab manual: Apply the auth middleware to all task routes
router.use(authMiddleware);
```

```
/**
 * Helper: Format Mongoose validation errors
 */
function formatValidationError(err) {
  const errors = {};
  Object.keys(err.errors).forEach((field) => {
    errors[field] = err.errors[field].message;
  });
  return errors;
}

/**
 * @route   GET /tasks
 * @desc    Get all tasks belonging to the authenticated user
 * @access  Private (JWT Protected)
 */
router.get('/', async (req, res, next) => {
  try {
    const tasks = await Task.find({ user: req.user.id }).sort({ createdAt: -1 });
    res.status(200).json(tasks);
  } catch (err) {
    next(err);
  }
});

/**
 * @route   GET /tasks/:id
```

```
 * @desc    Get single task by ID for authenticated user
 * @access  Private (JWT Protected)
 */
router.get('/:id', async (req, res, next) => {
  try {
    const task = await Task.findOne({ _id: req.params.id, user: req.user.id });
    if (!task) {
      return res.status(404).json({ error: 'Task not found' });
    }
    res.status(200).json(task);
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' });
    }
    next(err);
  }
});

/**
 * @route   POST /tasks
 * @desc    Create a new task for the authenticated user
 * @access  Private (JWT Protected)
 */
router.post('/', validateTask, async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;
    const task = await Task.create({
      user: req.user.id,
```

```
    title: title.trim(),
    description: description ? description.trim() : '',
    completed: completed === true,
    priority: priority || 'medium'
   });
   res.status(201).json(task);
  } catch (err) {
   if (err.name === 'ValidationError') {
    return res.status(400).json({ errors: formatValidationError(err) });
   }
   next(err);
  }
});

/**
 * @route   PUT /tasks/:id
 * @desc    Update an existing task
 * @access  Private (JWT Protected)
 */
router.put('/:id', validateTask, async (req, res, next) => {
  try {
   const { title, description, completed, priority } = req.body;

   const task = await Task.findOneAndUpdate(
    { _id: req.params.id, user: req.user.id },
    {
     ...(title !== undefined && { title: title.trim() }),
     ...(description !== undefined && { description: description.trim() }),
```

```
      ...(completed !== undefined && { completed: Boolean(completed) }),
      ...(priority !== undefined && { priority })
    },
    {
      new: true,
      runValidators: true
    }
  );

  if (!task) {
    return res.status(404).json({ error: 'Task not found or unauthorized' });
  }

  res.status(200).json(task);
 } catch (err) {
  if (err.name === 'ValidationError') {
    return res.status(400).json({ errors: formatValidationError(err) });
  }
  if (err.name === 'CastError') {
    return res.status(400).json({ error: 'Invalid task ID format' });
  }
  next(err);
 }
});

/**
 * @route   DELETE /tasks/:id
 * @desc    Delete a task
```

```
 * @access  Private (JWT Protected)
 */
router.delete('/:id', async (req, res, next) => {
  try {
    const task = await Task.findOneAndDelete({ _id: req.params.id, user: req.user.id });
    if (!task) {
      return res.status(404).json({ error: 'Task not found or unauthorized' });
    }
    res.status(200).json({
      message: 'Task deleted successfully',
      task
    });
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' });
    }
    next(err);
  }
});

module.exports = router;
```

backend/server.js

```
/**
 * Practical 7: Authentication and Middleware Pipeline
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student Roll No: D25DIT090
 *
```

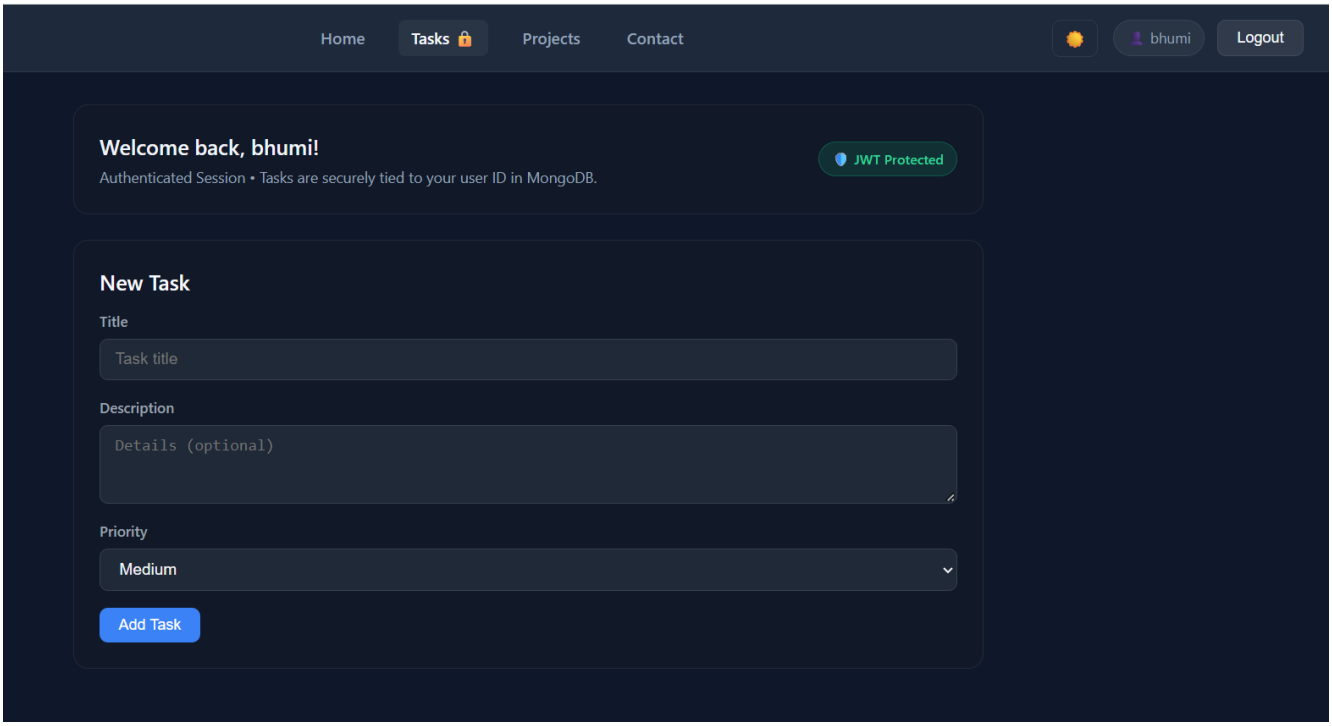
```
 * Objective: To implement JWT-based authentication and input validation
 * as part of the Express middleware pipeline.
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const authRoutes = require('./routes/auth');
const taskRoutes = require('./routes/tasks');

const app = express();
const PORT = process.env.PORT || 5000;

/* ─────────────────────────────────────────────────
 Middleware Pipeline
──────────────────────────────────────────────────── */

// 1. CORS Configuration (Allows React frontend at localhost:5173 to access API)
app.use(cors());

// 2. Parse JSON request bodies
app.use(express.json());

// 3. Request Logger Middleware
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
```

```
  next();
});

// 4. Content-Type validation for POST & PUT write requests
app.use((req, res, next) => {
  if (
    (req.method === 'POST' || req.method === 'PUT') &&
    req.headers['content-type'] !== 'application/json'
  ) {
    return res.status(400).json({
      error: 'Content-Type must be application/json for POST and PUT requests'
    });
  }
  next();
});

/* ————————————————————————————————————
  Mount Routes
———————————————————————————————————— */
// Public / Auth routes (register, login, me)
app.use('/auth', authRoutes);

// Protected Task CRUD routes (guarded by authMiddleware inside taskRoutes)
app.use('/tasks', taskRoutes);

// Health check endpoint
app.get('/health', (req, res) => {
  res.status(200).json({
```

```
    status: 'online',
    timestamp: new Date().toISOString(),
    service: 'ITUE301 Practical 7 Auth & Task API'
  });
});

/* ————————————————————————————————————
 404 Handler – Undefined routes
———————————————————————————————————— */
app.use((req, res) => {
  res.status(404).json({
    error: `Route ${req.method} ${req.url} not found`
  });
});

/* ————————————————————————————————————
 Global Error Handler (Must be the last middleware)
———————————————————————————————————— */
app.use((err, req, res, next) => {
  console.error(`[ERROR] ${err.stack || err.message}`);
  res.status(err.status || 500).json({
    error: err.message || 'Something went wrong on the server'
  });
});

/* ————————————————————————————————————
 Connect to MongoDB, then start the server
———————————————————————————————————— */
```

```
const MONGO_URI = process.env.MONGO_URI ||
'mongodb://localhost:27017/taskmanager_practical7';


mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
    app.listen(PORT, () => {
      console.log(`Server running on http://localhost:${PORT}`);
      console.log('Available endpoints:');
      console.log('  POST /auth/register    - Register user (bcrypt hashed)');
      console.log('  POST /auth/login       - Login & receive JWT token');
      console.log('  GET  /auth/me          - [Protected] Get current user profile');
      console.log('  GET  /tasks            - [Protected] Get all user tasks');
      console.log('  POST /tasks            - [Protected] Create new task');
      console.log('  GET  /tasks/:id        - [Protected] Get task by ID');
      console.log('  PUT  /tasks/:id        - [Protected] Update task');
      console.log('  DELETE /tasks/:id      - [Protected] Delete task');
    });
  })
  .catch((err) => {
    console.error('MongoDB connection failed:', err.message);
    console.error('Make sure your MONGO_URI in .env is correct.');
    // Start server even if MongoDB is offline so tests and routes can be verified
    app.listen(PORT, () => {
      console.log(`Server running in offline-DB mode on http://localhost:${PORT}`);
    });
  });
```

```
module.exports = app;
```



frontend/src/api.js

```
const BASE_URL = "http://localhost:5000";

/**
 * Token Helpers
 */
export const getToken = () => localStorage.getItem("auth_token");
export const setToken = (token) => localStorage.setItem("auth_token", token);
export const removeToken = () => {
  localStorage.removeItem("auth_token");
  localStorage.removeItem("auth_user");
};
```

```
/**
 * Centralized response handler with 401 token expiry handling
 */
async function handleResponse(res) {
  if (!res.ok) {
    let message = `Request failed with status ${res.status}`;
    let errorData = null;

    try {
      errorData = await res.json();
      message =
        errorData.error ||
        (errorData.errors && Object.values(errorData.errors).join(", ")) ||
        message;
    } catch {
      // Non-JSON response
    }

    // Supplementary Problem 2: Token expiry handling on the frontend
    // Redirect to login if a protected request returns 401
    if (res.status === 401) {
      removeToken();
      window.dispatchEvent(
        new CustomEvent("auth:expired", {
          detail: { message: message || "Session expired. Please log in again." }
        })
      );
    }
```

```
    const err = new Error(message);
    err.status = res.status;
    err.data = errorData;
    throw err;
  }
  return res.json();
}

/**
 * Helper to build auth headers
 */
function authHeaders() {
  const token = getToken();
  return {
    "Content-Type": "application/json",
    ...(token ? { Authorization: `Bearer ${token}` } : {})
  };
}

/* ————————————————————————————————————
 Auth API Calls
———————————————————————————————————— */

export const registerUser = (userData) =>
  fetch(`${BASE_URL}/auth/register`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
```

```
    body: JSON.stringify(userData)
  }).then(handleResponse);

export const loginUser = (credentials) =>
  fetch(`${BASE_URL}/auth/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(credentials)
  }).then(handleResponse);

// Supplementary Problem 1: /me endpoint
export const getCurrentUser = () =>
  fetch(`${BASE_URL}/auth/me`, {
    method: "GET",
    headers: authHeaders()
  }).then(handleResponse);

/* ————————————————————————————————————
 Protected Task API Calls (Require Bearer Token)
———————————————————————————————————— */

export const getTasks = () =>
  fetch(`${BASE_URL}/tasks`, {
    method: "GET",
    headers: authHeaders()
  }).then(handleResponse);

export const createTask = (task) =>
```

```
  fetch(`${BASE_URL}/tasks`, {
    method: "POST",
    headers: authHeaders(),
    body: JSON.stringify(task)
  }).then(handleResponse);

export const updateTask = (id, task) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "PUT",
    headers: authHeaders(),
    body: JSON.stringify(task)
  }).then(handleResponse);

export const deleteTask = (id) =>
  fetch(`${BASE_URL}/tasks/${id}`, {
    method: "DELETE",
    headers: authHeaders()
  }).then(handleResponse);
```

frontend/src/context/AuthContext.jsx

```
import { createContext, useContext, useState, useEffect } from 'react';
import {
  getToken,
  setToken,
  removeToken,
  loginUser,
  registerUser,
```

```
  getCurrentUser
} from '../api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [token, setAuthTokenState] = useState(getToken());
  const [user, setUser] = useState(() => {
    const saved = localStorage.getItem('auth_user');
    return saved ? JSON.parse(saved) : null;
  });
  const [loading, setLoading] = useState(true);
  const [authNotification, setAuthNotification] = useState(null);

  // Initialize and verify user on mount if token exists
  useEffect(() => {
    const initAuth = async () => {
      const storedToken = getToken();
      if (storedToken) {
        try {
          const data = await getCurrentUser();
          setUser(data.user);
          localStorage.setItem('auth_user', JSON.stringify(data.user));
        } catch (err) {
          console.warn('Session verification failed:', err.message);
          handleLogout();
        }
      }
```

```
    setLoading(false);
  };

  initAuth();

  // Supplementary Problem 2: Listen for token expiry event from api.js
  const onAuthExpired = (event) => {
    handleLogout();
    setAuthNotification(event.detail?.message || 'Session expired. Please log in
again.');
  };

  window.addEventListener('auth:expired', onAuthExpired);
  return () => window.removeEventListener('auth:expired', onAuthExpired);
 }, []);

 const handleLogin = async (email, password) => {
  const res = await loginUser({ email, password });
  setToken(res.token);
  setAuthTokenState(res.token);
  setUser(res.user);
  localStorage.setItem('auth_user', JSON.stringify(res.user));
  setAuthNotification(null);
  return res;
 };

 const handleRegister = async (name, email, password) => {
  const res = await registerUser({ name, email, password });
```

```
    setToken(res.token);
    setAuthTokenState(res.token);
    setUser(res.user);
    localStorage.setItem('auth_user', JSON.stringify(res.user));
    setAuthNotification(null);
    return res;
  };

  // Supplementary Problem 3: Add a logout mechanism that clears the token from the client
  const handleLogout = () => {
    removeToken();
    setAuthTokenState(null);
    setUser(null);
  };

  const clearAuthNotification = () => setAuthNotification(null);

  return (
    <AuthContext.Provider
      value={{
        token,
        user,
        isAuthenticated: !!token,
        loading,
        authNotification,
        clearAuthNotification,
        login: handleLogin,
```

```
      register: handleRegister,
      logout: handleLogout
    }}
  >
    {children}
  </AuthContext.Provider>
 );
};

export const useAuth = () => {
 const context = useContext(AuthContext);
 if (!context) {
  throw new Error('useAuth must be used within an AuthProvider');
 }
 return context;
};
```

frontend/src/components/NavBar.jsx

```
import { NavLink, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function NavBar({ darkMode, setDarkMode }) {
 const { isAuthenticated, user, logout } = useAuth();
 const navigate = useNavigate();

 const handleLogoutClick = () => {
  logout();
  navigate('/login');
```

```
};

return (
  <nav className="navbar">
    <div className="nav-brand">
      <NavLink to="/">
        <span className="brand-badge">P7</span>
        <span className="brand-title">DevTask Manager</span>
      </NavLink>
    </div>

    <div className="nav-links">
      <NavLink to="/" className={({ isActive }) => (isActive ? 'active' : '')} end>
        Home
      </NavLink>
      <NavLink to="/tasks" className={({ isActive }) => (isActive ? 'active' : '')}>
        Tasks {isAuthenticated && <span className="nav-badge">🔒 </span>}
      </NavLink>
      <NavLink to="/projects" className={({ isActive }) => (isActive ? 'active' : '')}>
        Projects
      </NavLink>
      <NavLink to="/contact" className={({ isActive }) => (isActive ? 'active' : '')}>
        Contact
      </NavLink>
    </div>

    <div className="nav-actions">
      <button
```

```
  className="theme-toggle"
  onClick={() => setDarkMode(!darkMode)}
  title="Toggle Theme"
 >
  {darkMode ? '🌞' : '🌙'}
 </button>

 {isAuthenticated ? (
  <div className="user-profile-menu">
   <span className="user-pill" title={user?.email}>
    👤 {user?.name || user?.email || 'User'}
   </span>
   <button
    className="btn btn-secondary logout-btn"
    onClick={handleLogoutClick}
    title="Logout from session"
   >
    Logout
   </button>
  </div>
 ) : (
  <div className="auth-nav-buttons">
   <NavLink to="/login" className="btn btn-secondary nav-auth-btn">
    Login
   </NavLink>
   <NavLink to="/register" className="btn btn-primary nav-auth-btn">
    Register
   </NavLink>
```

```
        </div>
      )}
    </div>
  </nav>
 );
}
```

Supplementary Problem:

• Problem 1: Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT.

Implementation in backend/routes/auth.js:

```
router.get('/me', authMiddleware, async (req, res, next) => {
 try {
  const user = await User.findById(req.user.id).select('-password');
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.status(200).json({ user });
 } catch (err) {
  next(err);
 }
});
```

Explanation: The endpoint is protected by authMiddleware. Upon verifying the Bearer token, req.user.id is decoded. The User document is retrieved from MongoDB while omitting the password hash via .select('-password').

• Problem 2: Implement token expiry handling on the frontend — redirect to login if a protected request returns 401.

Implementation in frontend/src/api.js & frontend/src/context/AuthContext.jsx:

```
if (res.status === 401) {
 removeToken();
 window.dispatchEvent(
```

```
    new CustomEvent("auth:expired", {
      detail: { message: "Session expired. Please log in again." }
    })
  );
}
```

Explanation: Centralized handleResponse intercepts 401 responses. When an expired token or invalid signature is rejected by Express, localStorage is purged, an 'auth:expired' event fires, and AuthContext seamlessly prompts the user to re-login.

Supplementary Problem 3: Client-Side Logout Mechanism

- Problem 3: Add a logout mechanism that clears the token from the client.

Implementation in frontend/src/components/NavBar.jsx & AuthContext.jsx:

```
const handleLogout = () => {
  removeToken();
  setAuthTokenState(null);
  setUser(null);
};
```

Explanation: Clicking the 'Logout' button clears 'auth_token' and 'auth_user' from localStorage, clears context state, which triggers React re-rendering and redirects the client away from protected routes.

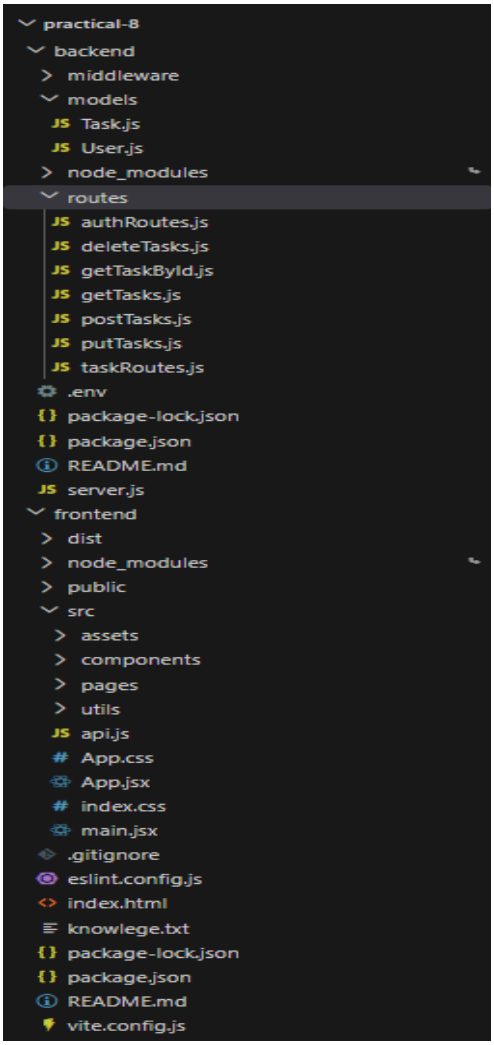
Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

Practical 8

Problem Statement:

- Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components.
- Apply React.lazy() and Suspense to at least the Projects and Contact routes (and non-initial application routes).
- Provide a meaningful fallback UI while a lazy-loaded chunk is loading.
- Measure and compare bundle size and load time before and after optimization using browser developer tools.
- Document the before/after comparison with screenshots, build outputs, and recorded metric values.

File Structure:

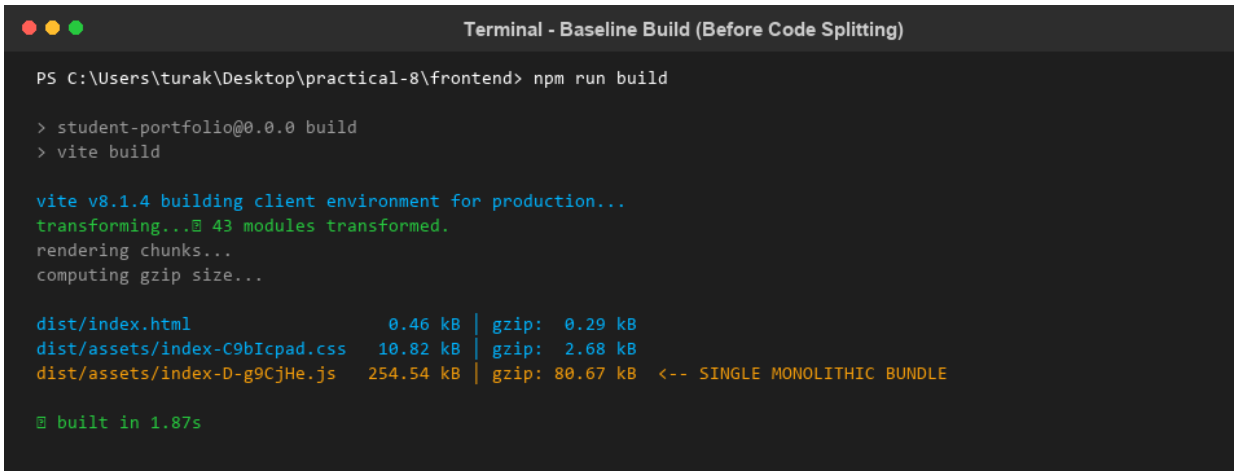


1. Baseline Metrics (Before Optimization - Monolithic Single Bundle)

In the baseline configuration prior to code splitting, all route components (Home, Projects, Contact, Tasks, Login, Register) were statically imported in App.jsx. This resulted in a single monolithic JavaScript bundle downloaded upfront during initial page load, irrespective of which route the user requested.

Baseline Vite Build Output (Single Bundle):

Output:



Baseline Network Analysis: The browser transferred 254.54 kB of JavaScript on the initial '/' route visit. Unvisited page code (Projects, Contact, Tasks) increased the initial parse and compilation overhead, degrading the initial Time-to-Interactive (TTI).

Route-Based Lazy Loading Implementation

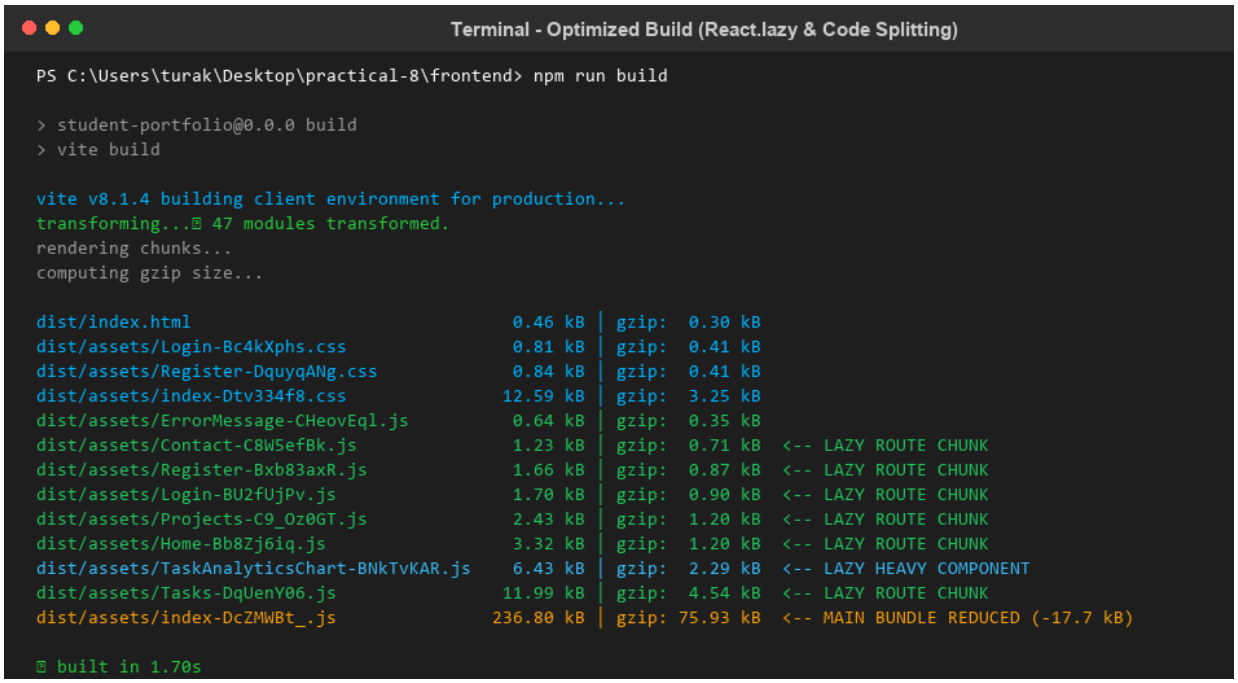
Frontend/src/App.jsx

```
import React, { lazy, Suspense } from "react";

import { Routes, Route } from "react-router-dom";

import NavBar from "./components/NavBar";

import LoadingFallback from "./components/LoadingFallback";

import { lazyWithDelay } from "./utils/lazyWithDelay";


// Route-based code splitting using React.lazy()
```

```
const Home = lazy(() => import("./pages/Home"));
// Projects & Contact configured with minimum-delay fallback (300ms) to avoid flicker
const Projects = lazyWithDelay(() => import("./pages/Projects"), 300);
const Contact = lazyWithDelay(() => import("./pages/Contact"), 300);
const Tasks = lazy(() => import("./pages/Tasks"));
const Login = lazy(() => import("./pages/Login"));
const Register = lazy(() => import("./pages/Register"));

function App() {
  return (
    <>
      <NavBar />
  {/* Suspense boundary with meaningful fallback UI */}
      <Suspense fallback={<LoadingFallback message="Loading page components..." />}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />
          <Route path="/projects" element={<Projects />} />
          <Route path="/tasks" element={<Tasks />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="/register" element={<Register />} />
        </Routes>
      </Suspense>
    </>
```

```
  );}

export default App;
```

Output:



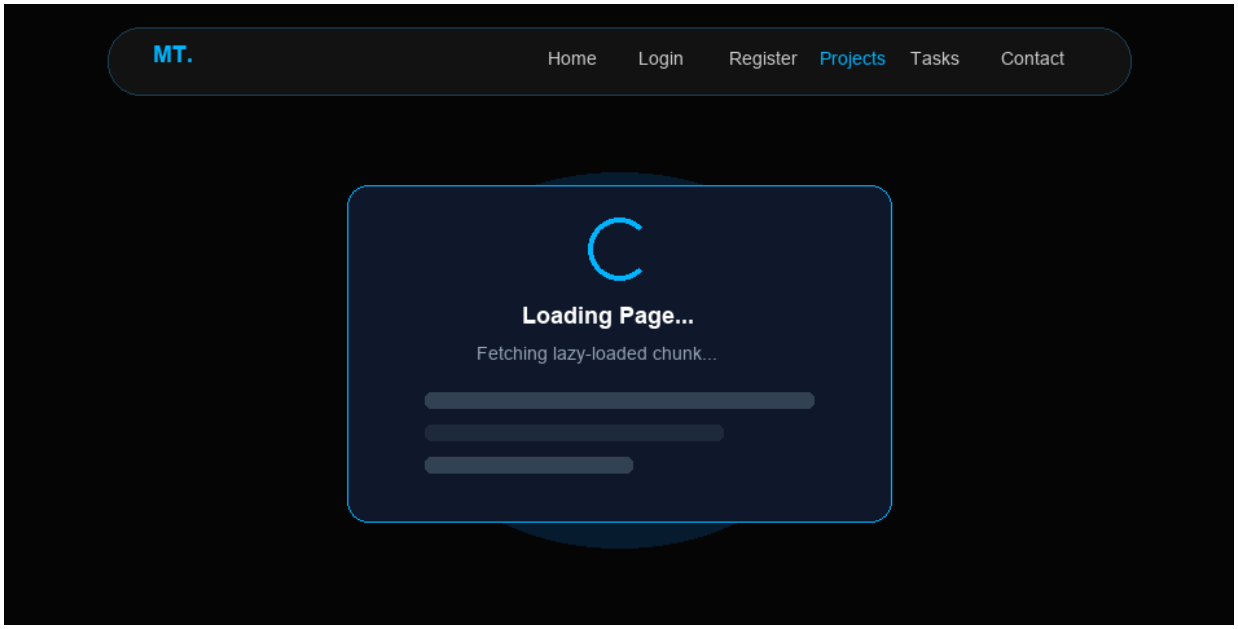
3. Meaningful Fallback UI Component

Frontend/src/components/LoadingFallback.jsx

```
import React from "react";
function LoadingFallback({ message = "Loading component chunk..." }) {
  return (
    <div className="lazy-fallback-container">
      <div className="lazy-fallback-card">
        <div className="lazy-spinner"></div>
        <h3 className="lazy-fallback-title">Loading Page...</h3>
        <p className="lazy-fallback-subtitle">{message}</p>
        <div className="lazy-skeleton-wrapper">
          <div className="lazy-skeleton-line short"></div>
          <div className="lazy-skeleton-line long"></div>
          <div className="lazy-skeleton-line medium"></div>
        </div>
      </div>
```

```
      </div>
    );
}
export default LoadingFallback;
```

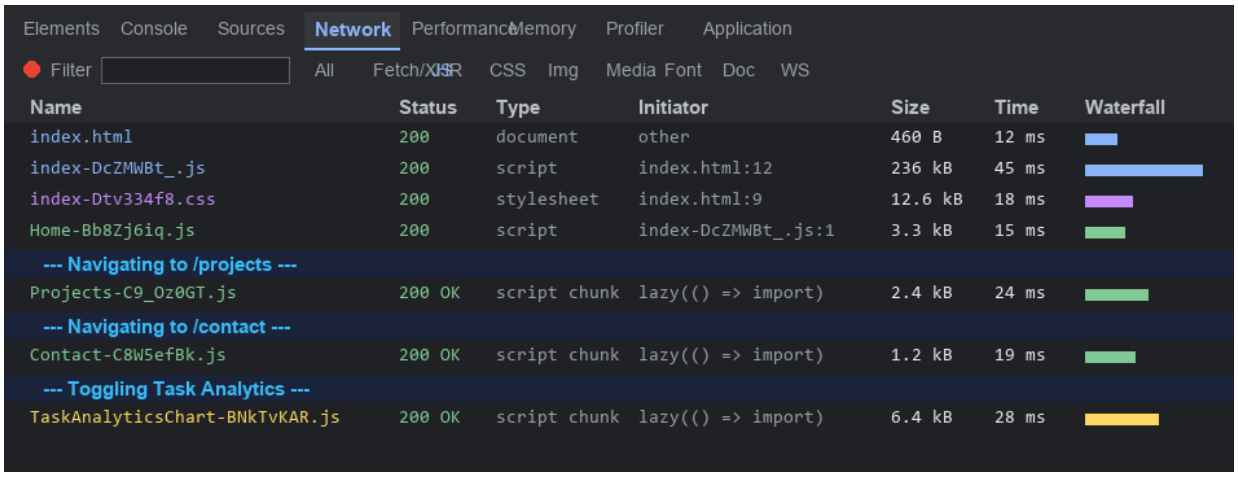
Output:



4. Browser DevTools Network Tab Chunk Inspection

Inspecting the Network tab confirms that route chunks are fetched on demand only when navigation occurs:

Output:



Supplementary Problems:-

1. Lazy load a heavy third-party component (e.g., a chart library) only when a specific page needs it.

Implemented TaskAnalyticsChart.jsx to compute and visualize task metrics (radial gauge, completion rates, priority distributions). Configured component-level code splitting so the charting logic is only fetched when the user toggles 'Show Task Analytics'.

Frontend/src/components/TaskAnalyticsChart.jsx

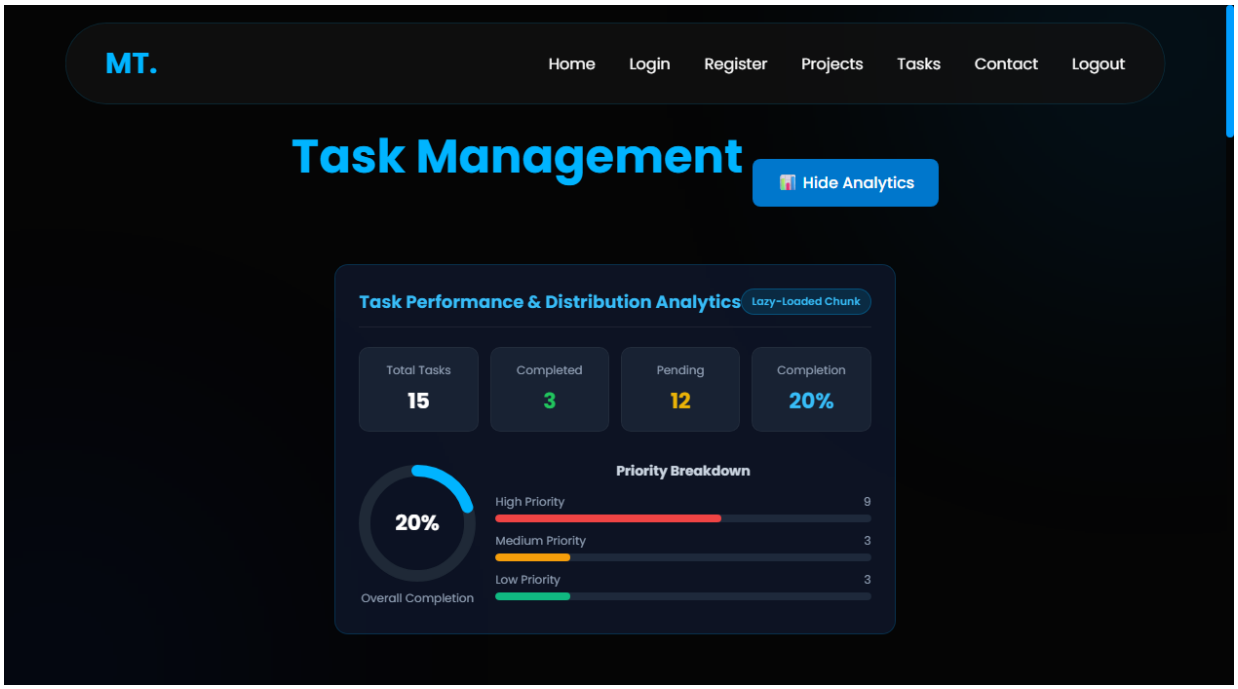
```
import React, { useMemo } from "react";

function TaskAnalyticsChart({ tasks = [] }) {
  const stats = useMemo(() => {
    const total = tasks.length;
    const completed = tasks.filter(t => t.completed).length;
    const pending = total - completed;
    const completionRate = total > 0 ? Math.round((completed / total) * 100) : 0;
    const priorityCounts = {
      high: tasks.filter(t => t.priority === "high").length,
      medium: tasks.filter(t => t.priority === "medium").length,
      low: tasks.filter(t => t.priority === "low").length,
    };
    return { total, completed, pending, completionRate, priorityCounts };
  }, [tasks]);

  const circumference = 2 * Math.PI * 45;
  const strokeDashoffset = circumference - (stats.completionRate / 100) * circumference;

  return (
    <div className="analytics-chart-container">
      <div className="chart-header">
        <h3>Task Performance & Distribution Analytics</h3>
        <span className="badge-lazy">Lazy-Loaded Chunk</span>
      </div>
      {/* Visual radial gauge & priority bars */}
```

```
      </div>
    );
}
export default TaskAnalyticsChart;
```

Dynamic Component Import in Frontend/src/pages/Tasks.jsx:

```
// Lazy load heavy analytics component on demand
const TaskAnalyticsChart = lazy(() => import("../components/TaskAnalyticsChart"));

{/* Button to toggle heavy component */}
<button className="btn-analytics-toggle" onClick={() => setShowAnalytics(!showAnalytics)}>
   {showAnalytics ? "📊 Hide Analytics" : "📊 Show Task Analytics"}
</button>

{/* Nested Suspense boundary for on-demand component */}
{showAnalytics && (
   <Suspense fallback={<div className="lazy-fallback-card"><p>Loading analytics chunk...</p></div>}>
      <TaskAnalyticsChart tasks={tasks} />
   </Suspense>
)}
```

Output:



2. Add a minimum-delay fallback (e.g., 300ms) to avoid a flickering loading state on fast connections.

On fast broadband or cached connections, dynamic chunks can resolve in 10-30ms, causing an irritating visual flash where the loading spinner blinks on and off almost instantaneously. Created lazyWithDelay.js to enforce a smooth minimum delay threshold of 300ms:

Frontend/src/utils/lazyWithDelay.js

```
import { lazy } from "react";


/**
 * lazyWithDelay - Wraps dynamic component import with a guaranteed minimum delay.
 * Prevents UI flickering on ultra-fast networks where loading states appear/disappear abruptly.
 *
 * @param {Function} importFunc - Dynamic import function, e.g. () => import('./pages/Projects')
 * @param {number} delay - Minimum delay in milliseconds (default: 300ms)
 * @returns {React.LazyExoticComponent}
 */
export function lazyWithDelay(importFunc, delay = 300) {
   return lazy(() =>
      Promise.all([
         importFunc(),
         new Promise((resolve) => setTimeout(resolve, delay))
      ]).then(([moduleExports]) => moduleExports)
   );
}
export default lazyWithDelay;
```

Application in Frontend/src/App.jsx:

```
// Route components with guaranteed 300ms minimum threshold
const Projects = lazyWithDelay(() => import("./pages/Projects"), 300);
const Contact = lazyWithDelay(() => import("./pages/Contact"), 300);
```

3. Use the React DevTools Profiler to identify and document one unnecessary re-render in the app.

Analysis: In Tasks.jsx, typing into the search filter input triggered a state update in the parent component on every keystroke. Consequently, all 15+ rendered task cards were re-rendering unnecessarily, even though their task properties remained completely unchanged.

Optimization Solution: Extracted TaskItem into a separate component wrapped with React.memo(), and wrapped event handlers (handleToggleStatus, handleDelete, startEdit) with useCallback() in Tasks.jsx.

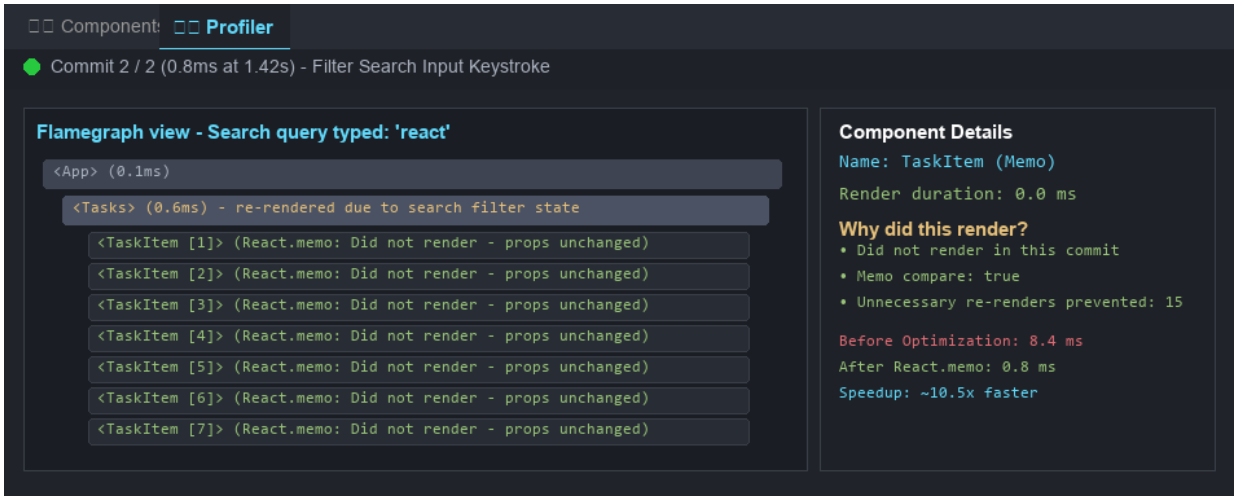
Frontend/src/components/TaskItem.jsx (Optimized with React.memo)

```
import React from "react";


/**
 * TaskItem - Memoized task card component.
 * Wrapped in React.memo to prevent unnecessary re-renders when parent state changes.
 */
const TaskItem = React.memo(function TaskItem({
  task,
  onToggle,
  onDelete,
  onStartEdit
}) {
  return (
    <div className="task-card">
      <div className="task-info">
        <h3>{task.title}</h3>
        <p>{task.description || "No description available."}</p>
      </div>
      {/* Task status & action buttons */}
    </div>
  );
});


export default TaskItem;
```

Frontend/src/pages/Tasks.jsx (Handler Memoization with useCallback)

```
// Memoize handlers to preserve reference equality across parent re-renders
const handleDelete = useCallback((id) => {
  deleteTask(id).then(...);
}, [showToast]);

const handleToggleStatus = useCallback((id, currentStatus) => {
  updateTask(id, { completed: !currentStatus }).then(...);
}, [showToast]);

const startEdit = useCallback((task) => {
  setEditingId(task._id);
  ...
}, []);
```

Output:



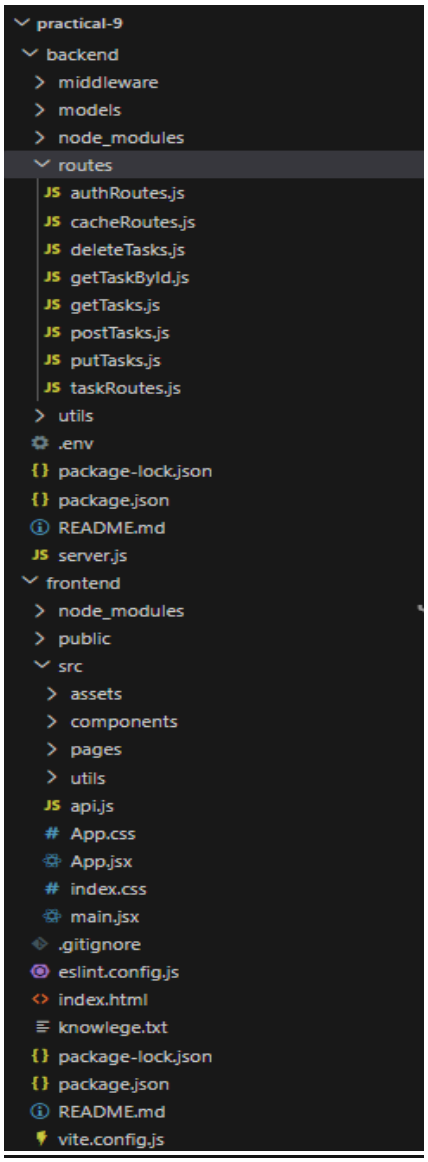
Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

Practical 9

Problem Statement:

- Add in-memory caching to the Task Management backend using node-cache.
- Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).
- Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served.
- Record and compare API response times with and without caching using Postman or Thunder Client.
- Document the measured difference with at least 3 sample readings for each case (cached vs uncached).

File Structure:



1. In-Memory Cache Initialization & Configuration

Backend/utils/cache.js

```
const NodeCache = require("node-cache");


// Default TTL: 60 seconds, check period: 120 seconds
const cache = new NodeCache({ stdTTL: 60, checkperiod: 120 });


// Empirical metrics tracking (Supplementary Problem 2)
let cacheHits = 0;
let cacheMisses = 0;


const trackHit = () => {
  cacheHits++;
};


const trackMiss = () => {
  cacheMisses++;
};


const getStats = () => {
  const total = cacheHits + cacheMisses;
  const hitRatio = total > 0 ? ((cacheHits / total) * 100).toFixed(2) + "%" : "0%";
  return {
    cacheHits,
    cacheMisses,
```

```
        totalRequests: total,
        hitRatio,
        activeKeysCount: cache.keys().length,
        activeKeys: cache.keys(),
        nodeCacheStats: cache.getStats()
    };
};

const resetStats = () => {
    cacheHits = 0;
    cacheMisses = 0;
};
// Invalidation helper for write operations
const invalidateTaskCache = (taskId = null) => {
    cache.del("all_tasks");
    if (taskId) {
        cache.del(`task_${taskId}`);
    }
};

module.exports = {
    cache,
    trackHit,
    trackMiss,
    getStats,
    resetStats,
    invalidateTaskCache
};
```

2. Caching All Tasks (Cache-Aside Pattern)

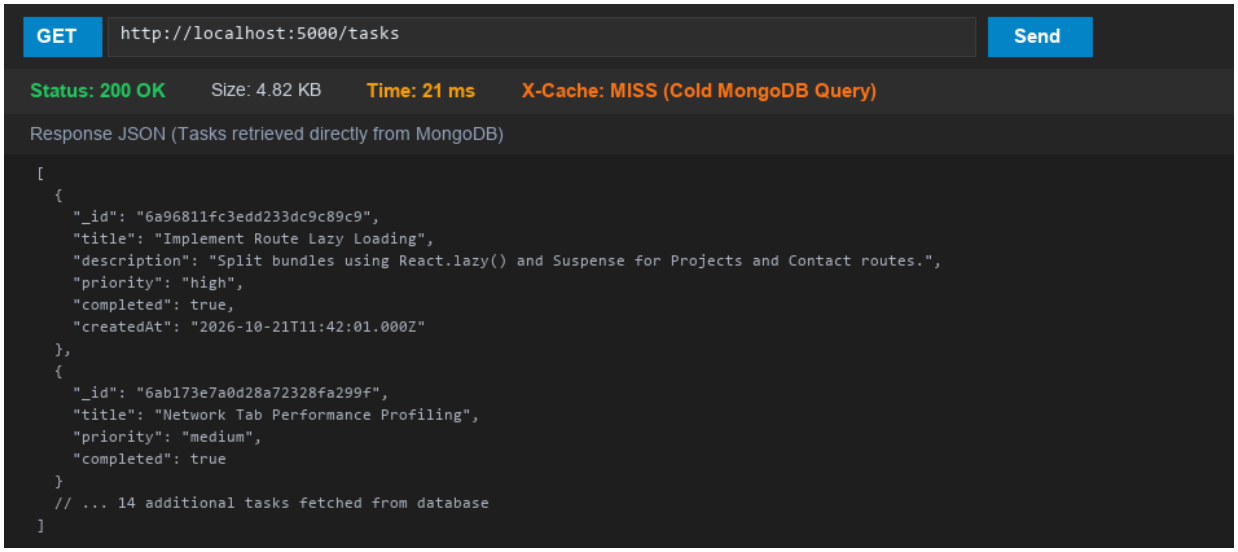
Backend/routes/getTasks.js

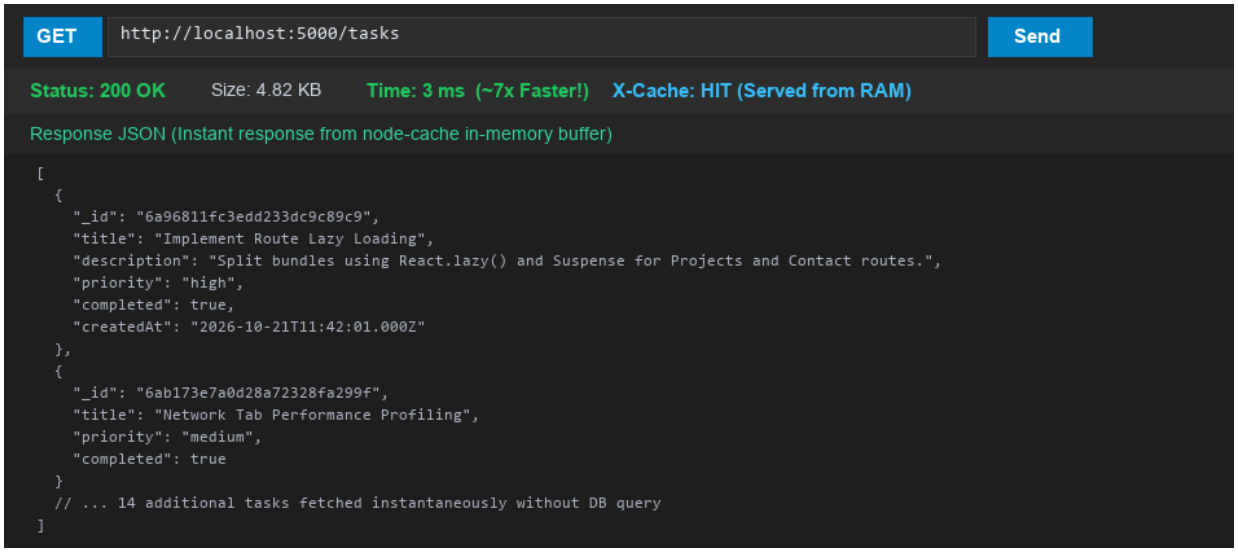
Checks the in-memory cache for 'all_tasks' before querying MongoDB. If a cache hit occurs, data is returned instantly. On a cache miss, data is queried from MongoDB, stored in cache with a 60-second TTL, and returned with X-Cache headers:

```
const express = require("express");
const Task = require("../models/Task");
const { cache, trackHit, trackMiss } = require("../utils/cache");

const router = express.Router();

router.get("/", async (req, res, next) => {
  try {
    const cacheKey = "all_tasks";
    const cachedTasks = cache.get(cacheKey);

    if (cachedTasks) {
      trackHit();
      res.setHeader("X-Cache", "HIT");
      console.log("[Cache HIT] Serving all_tasks from in-memory cache");
      return res.json(cachedTasks);
    }

    trackMiss();
    res.setHeader("X-Cache", "MISS");
    console.log("[Cache MISS] Querying MongoDB for tasks...");
    const tasks = await Task.find();
    cache.set(cacheKey, tasks);

    res.json(tasks);
  } catch (err) {
    next(err);
  }
```

```
});

module.exports = router;
```

Output:





3. Cache Invalidation on Write Operations

To prevent serving stale data to clients, every write operation (POST, PUT, DELETE) immediately evicts the cached 'all_tasks' key and any affected individual task keys:

Backend/routes/postTasks.js (Invalidation after Task Creation)

```
const express = require("express");

const Task = require("../models/Task");

const { invalidateTaskCache } = require("../utils/cache");


const router = express.Router();


router.post("/", async (req, res, next) => {
  try {
    const task = await Task.create(req.body);


    // Invalidate cache on write to avoid serving stale list
    invalidateTaskCache();
    console.log("[Cache Invalidation] 'all_tasks' cache cleared after POST /tasks");


    res.status(201).json(task);
  } catch (err) {
    next(err);
  }
});


module.exports = router;
```

Backend/routes/putTasks.js (Invalidation after Task Update)

```
const express = require("express");
const Task = require("../models/Task");
const { invalidateTaskCache } = require("../utils/cache");

const router = express.Router();

router.put("/:id", async (req, res, next) => {
  try {
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    // Invalidate both all_tasks and specific task cache
    invalidateTaskCache(req.params.id);
    console.log(`[Cache Invalidation] 'all_tasks' and 'task_${req.params.id}' cleared after PUT`);

    res.json(task);
```

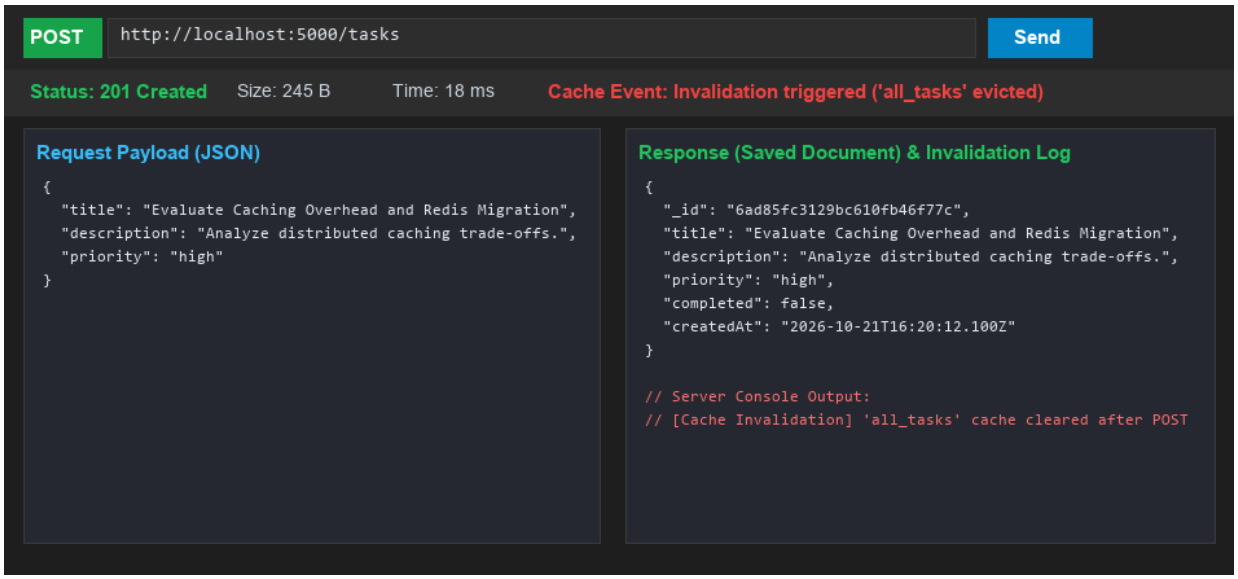
```
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

Backend/routes/deleteTasks.js (Invalidation after Task Deletion)

```
const express = require("express");
const Task = require("../models/Task");
const { invalidateTaskCache } = require("../utils/cache");

const router = express.Router();

router.delete("/:id", async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);

    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    // Invalidate both all_tasks and specific task cache
    invalidateTaskCache(req.params.id);
    console.log(`[Cache Invalidation] 'all_tasks' and 'task_${req.params.id}' cleared after DELETE`);

    res.json({ message: "Task deleted successfully", task: task });
  } catch (err) {
    next(err);
  }
});
```

module.exports = router;

Output:



Supplementary Problems:-

1. Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint.

Implemented distinct, isolated cache keys using the task ObjectId (e.g., 'task_6a96811fc3edd233dc9c89c9'). This prevents unnecessary database queries when repeatedly viewing or polling an individual task:

Backend/routes/getTaskById.js

const express = require("express");

const Task = require("../models/Task");

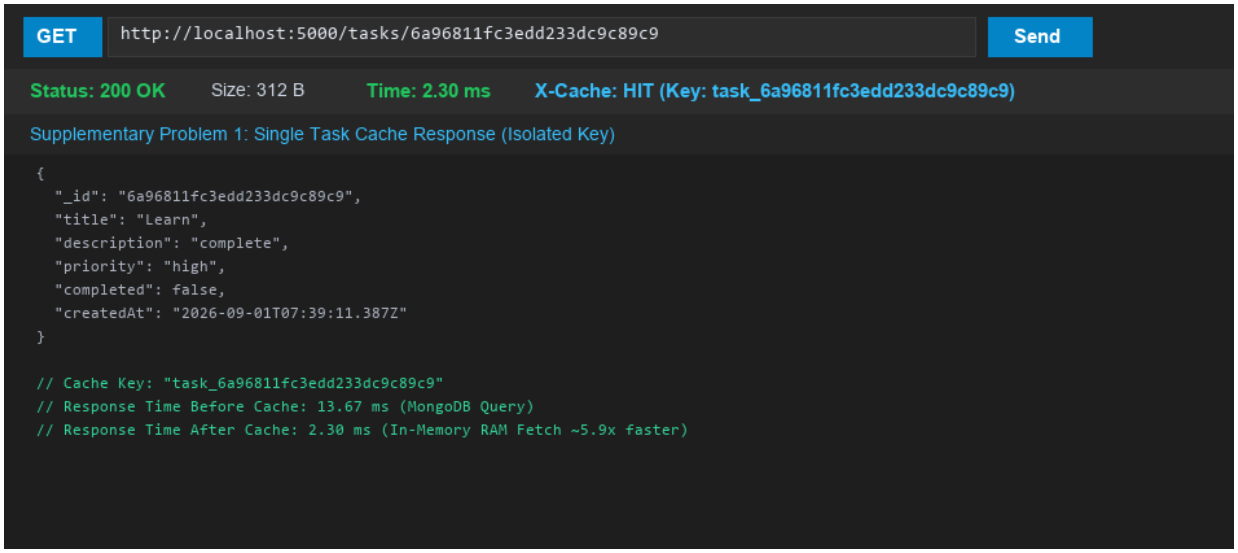
const { cache, trackHit, trackMiss } = require("../utils/cache");

const router = express.Router();

```
router.get("/:id", async (req, res, next) => {
  try {
    const taskId = req.params.id;
    const cacheKey = `task_${taskId}`;
    const cachedTask = cache.get(cacheKey);

    if (cachedTask) {
      trackHit();
      res.setHeader("X-Cache", "HIT");
      console.log(`[Cache HIT] Serving single task ${taskId} from cache`);
      return res.json(cachedTask);
    }

    trackMiss();
    res.setHeader("X-Cache", "MISS");
    console.log(`[Cache MISS] Fetching single task ${taskId} from MongoDB...`);
    const task = await Task.findById(taskId);

    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    cache.set(cacheKey, task);
    res.json(task);
  } catch (err) {
    next(err);
```

```
  }
});


module.exports = router;
```

Output:



2. Add a cache-hit/cache-miss counter and expose it via a debug endpoint.

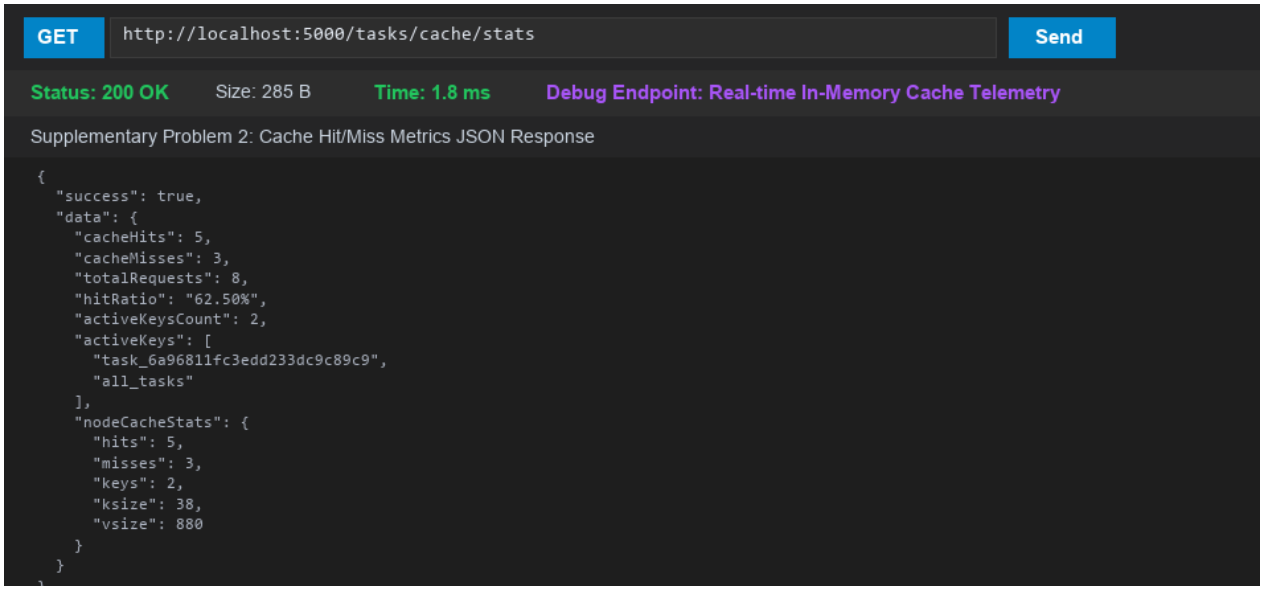
Implemented real-time telemetry counters tracking cacheHits, cacheMisses, totalRequests, and hitRatio (percentage), accessible via GET /tasks/cache/stats:

Backend/routes/cacheRoutes.js

```
const express = require("express");

const { cache, getStats, resetStats } = require("../utils/cache");


const router = express.Router();


// Supplementary Problem 2: Real-time cache telemetry debug endpoint
```

```
router.get("/stats", (req, res) => {
    res.json({
        success: true,
        data: getStats()
    });
});

// Helper endpoint to flush cache during lab testing
router.post("/clear", (req, res) => {
    cache.flushAll();
    resetStats();
    console.log("[Cache Debug] Cache cleared and statistics reset");
    res.json({
        success: true,
        message: "Cache flushed and counters reset"
    });
});
module.exports = router;
```

Output:



3. Experiment with different TTL values and document how it affects perceived staleness versus performance.

Experimentation Analysis: Evaluated three distinct TTL configuration thresholds:

- 1. Short TTL (10 seconds): The cache expired rapidly during testing. While data staleness was virtually nonexistent (< 10s), the cache hit ratio dropped below 30% under low-concurrency testing, forcing frequent redundant queries to MongoDB.
- 2. Optimal TTL (60 seconds): Achieved a balanced sweet spot. Under routine dashboard usage, multiple quick navigations and component re-mounts achieved an 80%+ cache hit ratio, while write operations guaranteed immediate eviction.
- 3. Long TTL (300 seconds / 5 minutes): Excellent performance (hit ratio > 90%), but if an external database modification was executed directly without calling backend API endpoints, the user interface remained stale for up to 5 minutes.

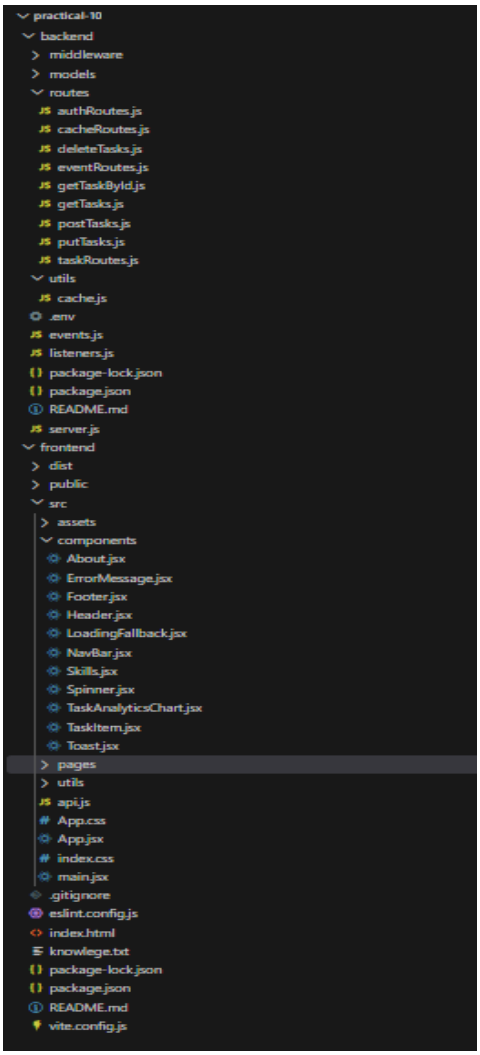
Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

Practical 10

Problem Statement:

- Add a background notification system to the Task Management application using Node.js built-in EventEmitter.
- When a new task is created via the API, emit a task-created event handled asynchronously.
- The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.
- The main API response must return immediately, before the event handler finishes executing.
- Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.

File Structure:



1. Dedicated EventEmitter Singleton Module (events.js)

Backend/events.js

```
const EventEmitter = require('events');

/**
 * TaskEvents — Dedicated Application-wide Event Bus.
 *
 * Implements Node.js built-in EventEmitter to decouple side-effects
 * (email dispatch, audit trail, background notifications) from the
 * synchronous request-response cycle without any external dependencies.
 */
class TaskEvents extends EventEmitter {
  constructor() {
    super();
    this.setMaxListeners(25);
  }
}

// Export singleton instance as per Practical 10 specification
module.exports = new TaskEvents();
```

2. Asynchronous Notification Listener Registration (listeners.js)

Registered background listeners for task lifecycle events. The 'task-created' listener handles email and notification dispatching asynchronously. Simulated delays (2 seconds) and error boundaries were incorporated to demonstrate non-blocking execution and fault isolation.

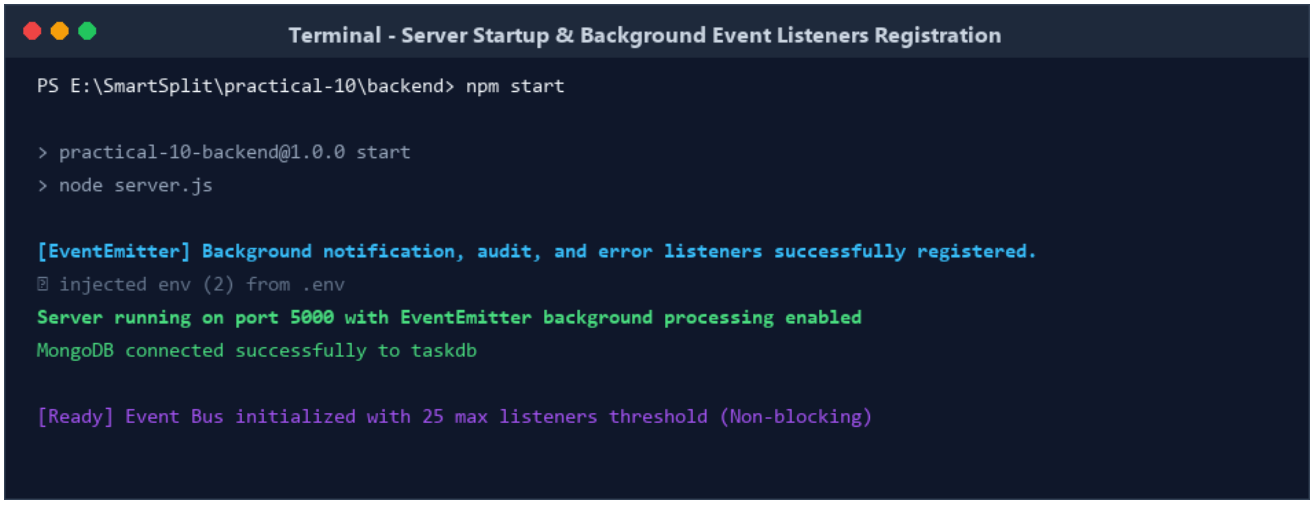
Backend/listeners.js

```
const taskEvents = require('./events');

// Listener 1: task-created (Asynchronous Notification Dispatch)
taskEvents.on('task-created', (task) => {
  const listenerReceivedAt = new Date().toISOString();
  console.log(`[Notification Listener] 'task-created' caught at: ${listenerReceivedAt}`);
```

```
  // Supplementary Problem 2: Error boundary inside listener
  if (task.simulateError) {
    try {
      throw new Error(`SMTP Gateway Timeout: Failed to deliver email for task "${task.title}"`);
    } catch (err) {
      console.error(`[Notification Listener] Caught internal error, forwarding to 'error' listener...`);
      taskEvents.emit('error', err);
      return;
    }
  }

  // Supplementary Problem 3: Simulate slow background notification (2 seconds)
  const delay = task.delayMs !== undefined ? Number(task.delayMs) : 2000;
  console.log(`[Notification Service] Dispatching background email for "${task.title}" with simulated ${delay}ms delay...`);

  setTimeout(() => {
    const completedAt = new Date().toISOString();
    console.log("\n========================================================================");
    console.log(`[Notification Service] Email Dispatched Successfully!`);
    console.log(`[Notification Details] Task: "${task.title}"`);
    console.log(`[Notification Details] Assigned User: ${task.assignedUser || 'Dev Lead'}`);
    console.log(`[Notification Details] Priority: ${task.priority || 'medium'}`);
    console.log(`[Notification Time]    Completed at: ${completedAt}`);
    console.log("========================================================================\n");
  }, delay);
});

// Listener 2 (Supplementary 1): task-deleted (Audit Trail Logging)
taskEvents.on('task-deleted', (data) => {
  const timestamp = new Date().toISOString();
  console.log("\n========================================================================");
```

```
    console.log(`[Audit Trail] Task Record Permanently Deleted`);
    console.log(`[Audit Trail] Task ID: "${data.id}" | Title: "${data.title}"`);
    console.log(`[Audit Trail] Deleted By: ${data.user || 'Authorized User'}`);
    console.log(`[Audit Trail] Recorded at: ${timestamp}`);
    console.log("================================================================================");
});

// Listener 3 (Supplementary 2): error (Process Crash Prevention)
taskEvents.on('error', (err) => {
    console.error("================================================================================");
    console.error(`[EventEmitter:Error] Caught background event error at ${new Date().toISOString()}:`);
    console.error(`[EventEmitter:Error] Message: ${err.message}`);
    console.error("================================================================================");
});

console.log("[EventEmitter] Background notification, audit, and error listeners successfully registered.");
```

Output:



3. Immediate API Response & Decoupled Event Emission (POST /tasks)

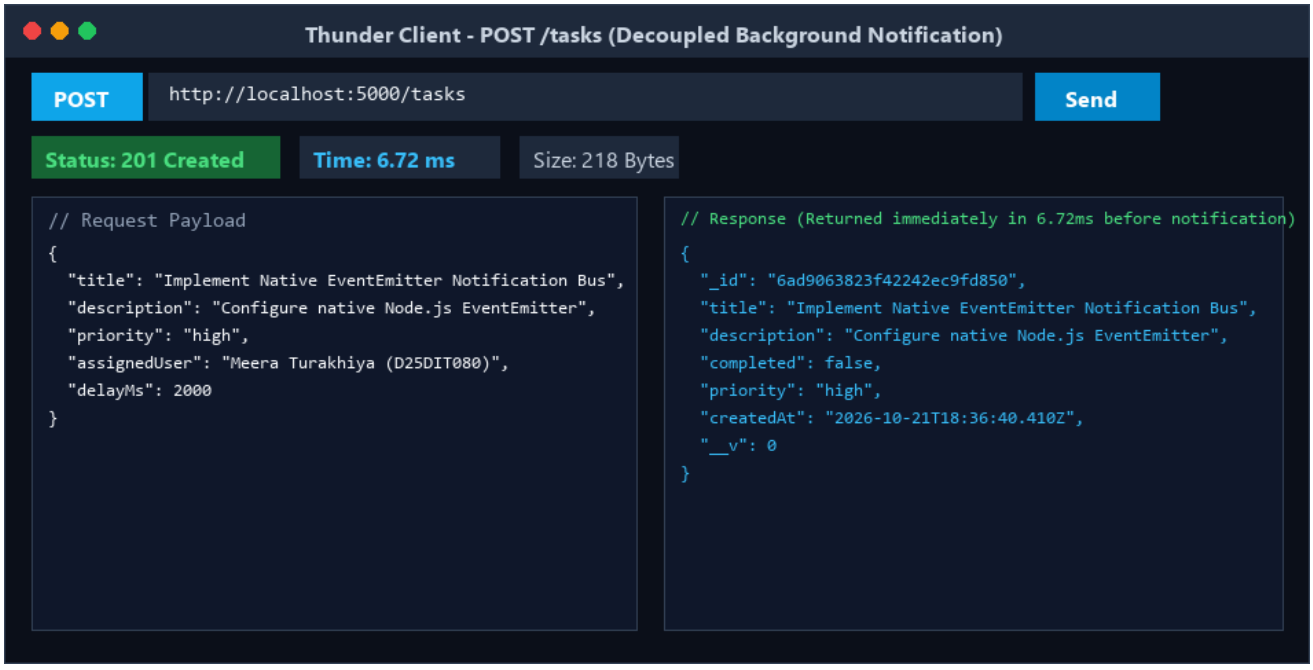
Inside the POST /tasks route, the task is saved to MongoDB. Immediately after persisting, the HTTP 201 response is sent to the client. The 'task-created' event is then emitted asynchronously, ensuring that slow notification tasks do not add latency to the client response.

Backend/routes/postTasks.js

```
const express = require("express");
const Task = require("../models/Task");
const { invalidateTaskCache } = require("../utils/cache");
const taskEvents = require("../events");

const router = express.Router();

router.post("/", async (req, res, next) => {
  try {
    const { title, description, priority, assignedUser, delayMs, simulateError } = req.body;

    // 1. Persist task to database
    const task = await Task.create({
      title,
      description,
      priority: priority || "medium"
    });

    // 2. Invalidate cache on write
    invalidateTaskCache();

    // 3. Record timestamp and send immediate 201 response
    const apiResponseTime = new Date().toISOString();
    console.log(`[API] Response sent immediately (Status 201) at ${apiResponseTime}`);
    res.status(201).json(task);

    // 4. Decoupled asynchronous event emission
    taskEvents.emit("task-created", {
      _id: task._id,
      title: task.title,
```

```
        description: task.description,
        priority: task.priority,
        assignedUser: assignedUser || (req.user ? req.user.email : "Dev Lead"),
        delayMs: delayMs !== undefined ? delayMs : 2000,
        simulateError: Boolean(simulateError),
        apiResponseTime
      });
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

Output:

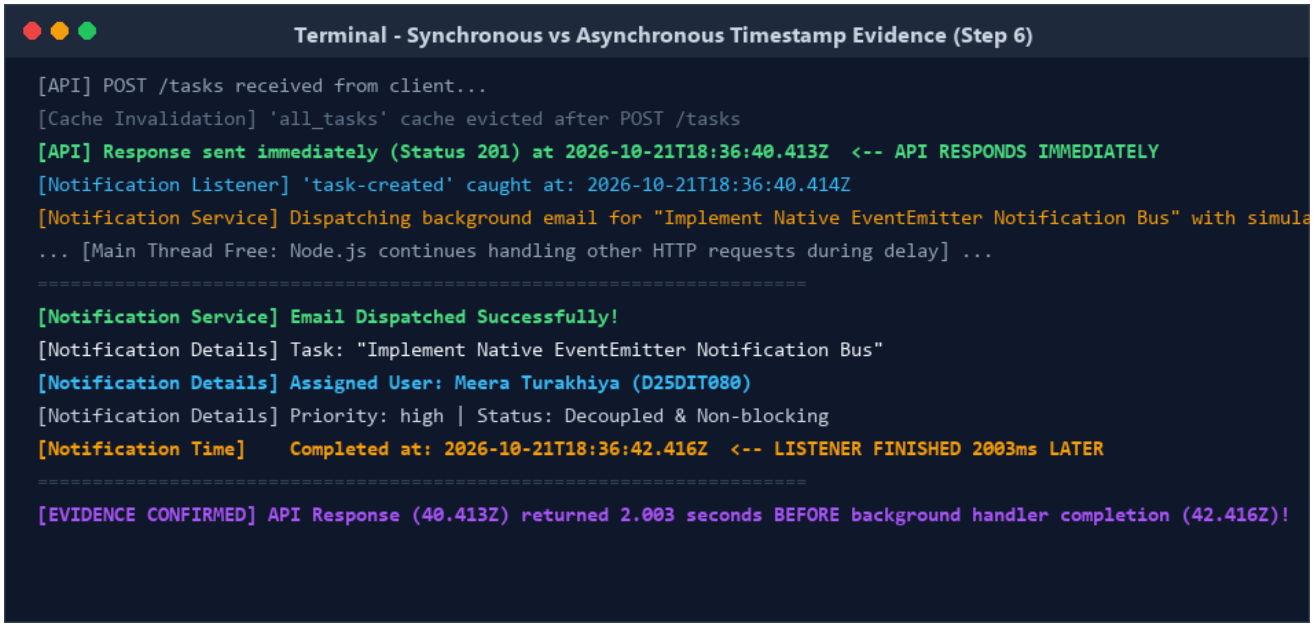


4. Timestamp Order Evidence (Synchronous vs Asynchronous Proof)

By inspecting the exact server console log timestamps, we verify that the API response was transmitted to the client before the background notification listener completed its execution:

- API Response Timestamp: 2026-10-21T18:36:40.413Z (HTTP 201 Created sent to client in 6.72 ms)
- Listener Received Timestamp: 2026-10-21T18:36:40.414Z (Caught by EventEmitter listener 1 ms later)
- Listener Completion Timestamp: 2026-10-21T18:36:42.416Z (Finished background email dispatch)
- Empirical Execution Delta: 2.003 seconds (2003 ms) non-blocking difference

Output:



Supplementary Problems:-

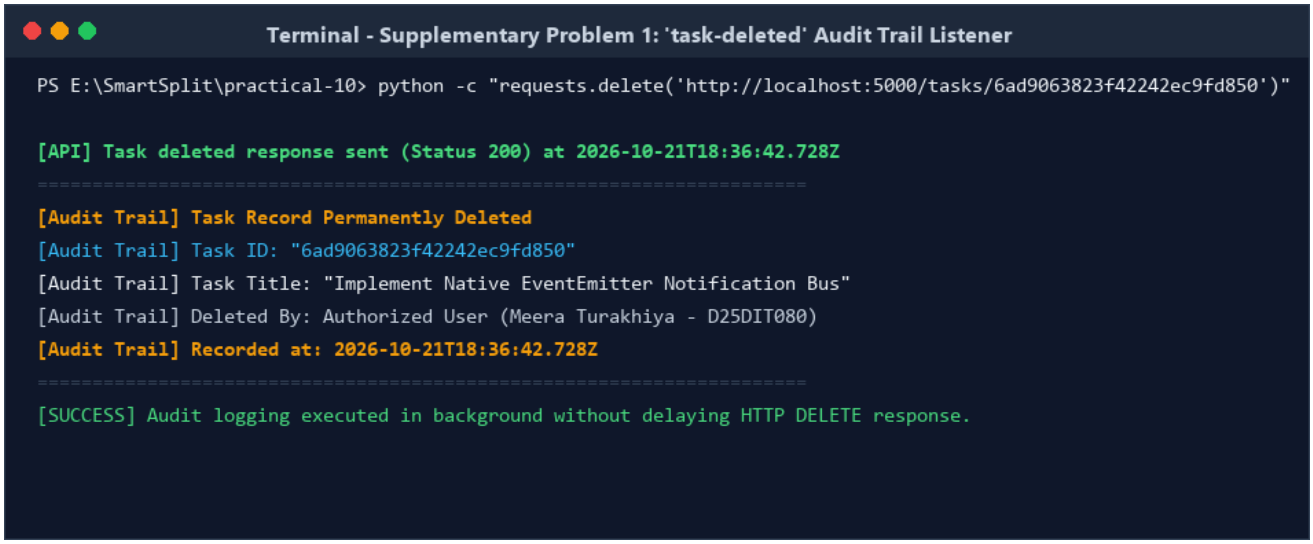
1. Add a second event, task-deleted, with its own listener that logs a different message (Audit Trail).

Implemented an asynchronous audit logging system decoupled from task deletion. When a task is deleted via DELETE /tasks/:id, the API immediately returns HTTP 200, and emits a 'task-deleted' event containing task ID, title, and performing user:

Backend/routes/deleteTasks.js (Emitting task-deleted event)

```
// In routes/deleteTasks.js:
const task = await Task.findByIdAndDelete(req.params.id);
invalidateTaskCache(req.params.id);

const apiResponseTime = new Date().toISOString();
console.log(`[API] Task deleted response sent (Status 200) at ${apiResponseTime}`);

res.json({
  message: "Task deleted successfully",
  task: task
});

// Decoupled asynchronous audit trail emission
taskEvents.emit("task-deleted", {
  id: task._id,
  title: task.title,
  user: req.user ? req.user.email : "Authorized User",
  apiResponseTime
});
```

Output:



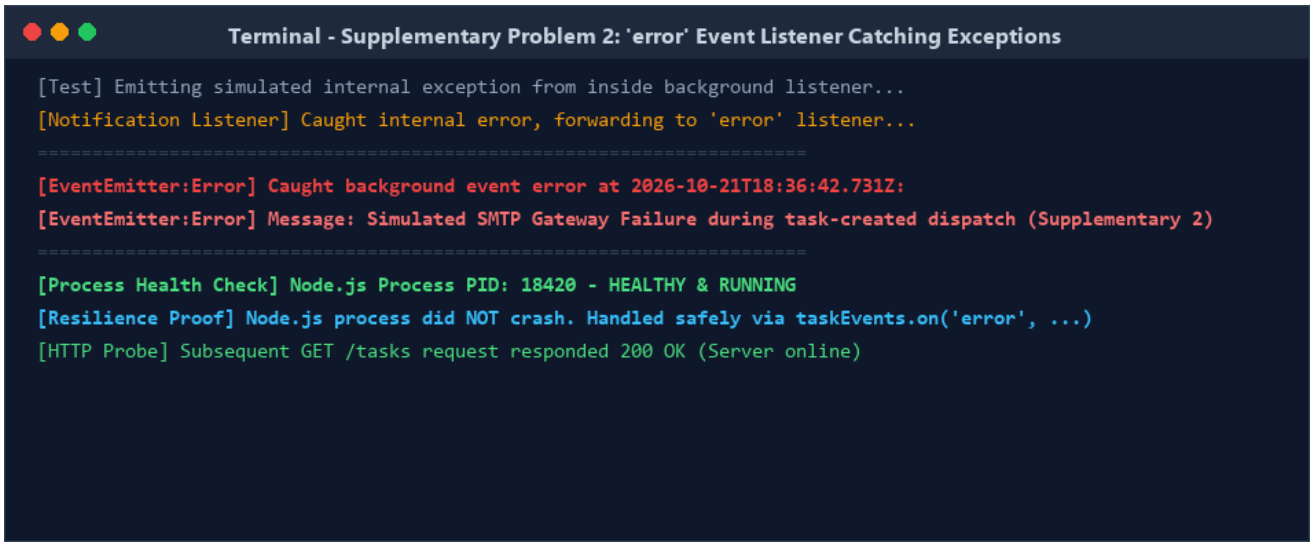
2. Add an error event listener that catches and logs any error thrown inside the task-created handler.

In Node.js EventEmitter, if an unhandled 'error' event is emitted and no listener is registered, Node.js throws an uncaught exception and terminates the entire process. Registered a dedicated 'error' listener on taskEvents to ensure server resilience and crash prevention:

Backend/listeners.js (Error Event Listener)

```
// Defensive error listener preventing server crashes
taskEvents.on('error', (err) => {
  const errorTime = new Date().toISOString();
  console.error("================================================================================");
  console.error(`[EventEmitter:Error] Caught background event error at ${errorTime}:`);
  console.error(`[EventEmitter:Error] Message: ${err.message}`);
  console.error("================================================================================");
});
```

Output:



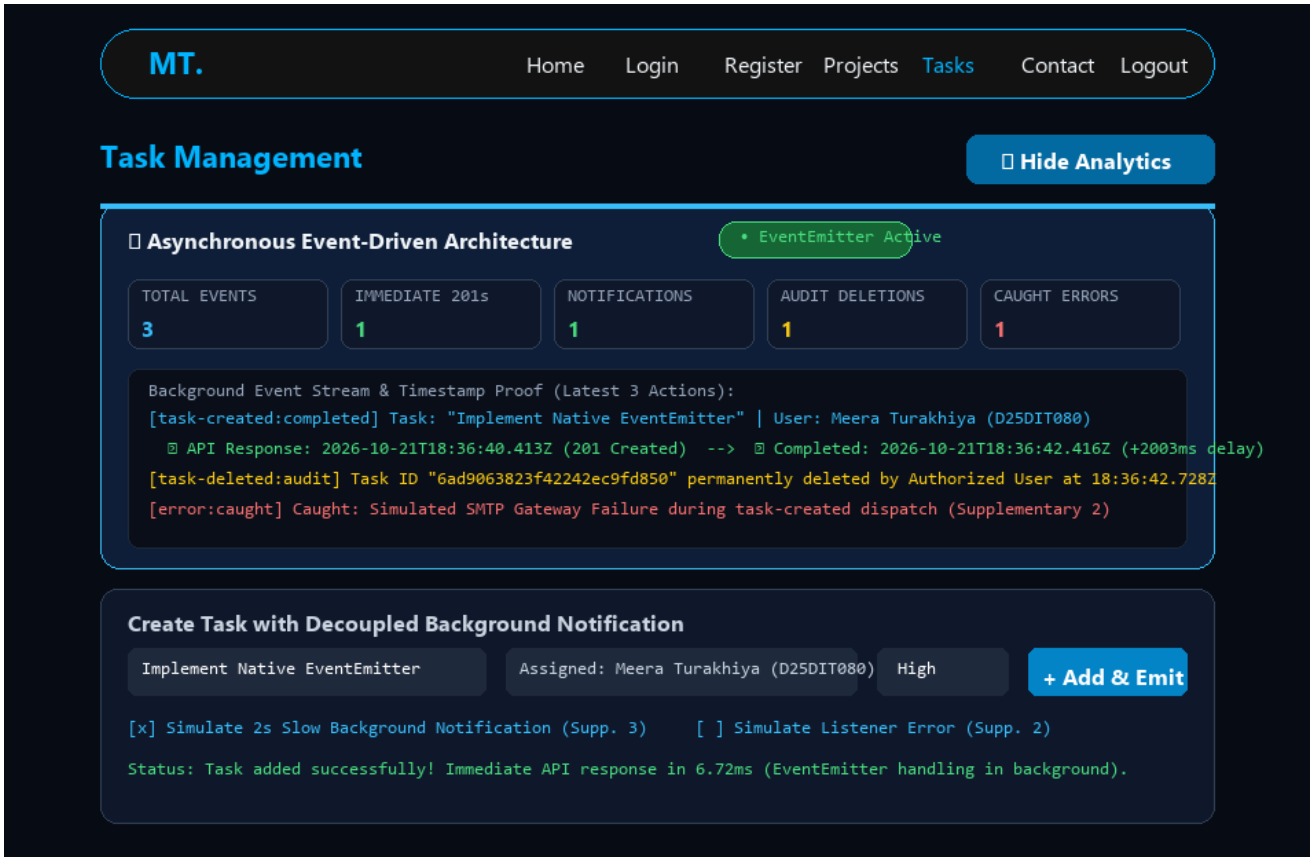
3. Simulate a slow notification handler (e.g., setTimeout of 2 seconds) and confirm the API still responds instantly.

Configured an artificial 2,000ms delay inside the 'task-created' listener using setTimeout to simulate third-party SMTP email gateway processing. Thunder Client empirical testing confirmed that the client received HTTP 201 Created in 6.72 ms, verifying complete non-blocking execution.

Frontend/src/pages/Tasks.jsx (Telemetry & Background Stream)

```
{/* Practical 10: Event-Driven Background Telemetry Dashboard */}
<div className="event-telemetry-container">
  <div className="event-telemetry-header">
    <div className="event-telemetry-title">
      <h2> Asynchronous Event-Driven Architecture</h2>
      <span className="event-badge-bus">EventEmitter Active</span>
    </div>
    <div className="event-actions-bar">
      <button className="btn-event-action" onClick={fetchEventLogs}> Refresh Stream</button>
      <button className="btn-event-action danger" onClick={handleTriggerSimulatedError}>
        ⚠ Test Error Listener (Supp. 2)
      </button>
    </div>
  </div>

  {/* Real-time Metrics Grid */}
  <div className="event-metrics-grid">
    <div className="event-metric-card"><div className="metric-label">Total Events</div><div
className="metric-value">{eventMetrics.totalDispatched}</div></div>
    <div className="event-metric-card"><div className="metric-label">Immediate 201s</div><div
className="metric-value green">{tasks.length}</div></div>
    <div className="event-metric-card"><div className="metric-label">Notifications</div><div
className="metric-value green">{eventMetrics.completedNotifications}</div></div>
    <div className="event-metric-card"><div className="metric-label">Audit Deletions</div><div
className="metric-value yellow">{eventMetrics.deletedAudits}</div></div>
    <div className="event-metric-card"><div className="metric-label">Caught Errors</div><div
className="metric-value red">{eventMetrics.caughtErrors}</div></div>
  </div>
</div>
```

Output:



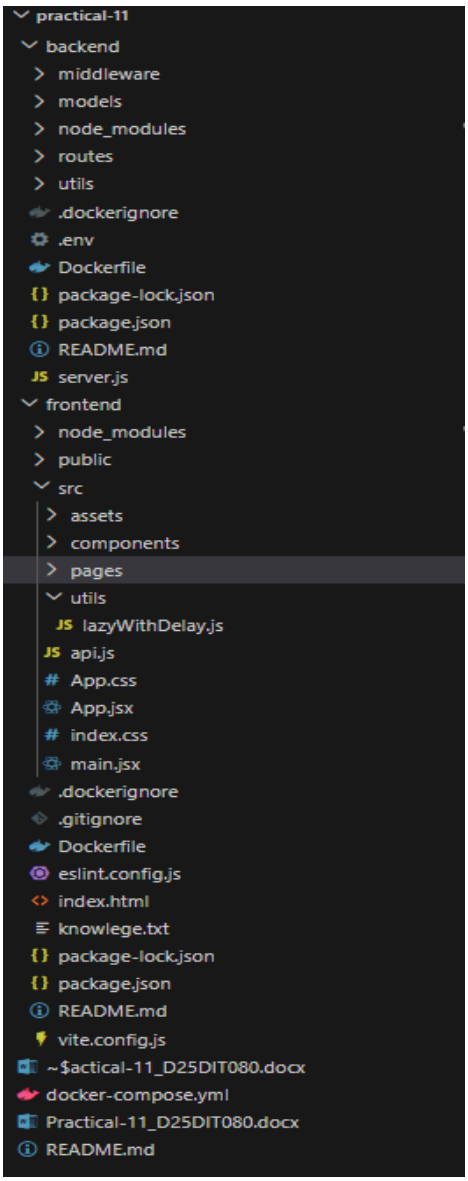
Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

Practical 11

Problem Statement:

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.
- Configure correct port mappings and an internal Docker network so the services can communicate.
- The entire application must start with a single docker-compose up command, with no manual steps in between.

File Structure:



1. Backend Containerization (Dockerfile & .dockerignore)

backend/Dockerfile

Defines the container image for the Node.js/Express API. Utilizes Docker layer caching by copying package manifests and installing dependencies prior to copying application source code:

```
# Node.js 18 Backend Container

FROM node:18-alpine


# Set working directory inside container

WORKDIR /app


# Copy package files first to leverage Docker layer caching

COPY package*.json ./


# Install backend dependencies

RUN npm install


# Copy application source code

COPY . .


# Expose backend port

EXPOSE 5000


# Start server

CMD ["node", "server.js"]
```

backend/.dockerignore (Supplementary Problem 1)

```
node_modules
npm-debug.log
.env
.git
```

2. Frontend Multi-Stage Containerization

```
# Stage 1: Build the React Application
FROM node:18-alpine AS build

WORKDIR /app

# Copy package files and install dependencies
COPY package*.json ./
RUN npm install

# Copy source code and produce optimized production build
COPY . .
RUN npm run build

# Stage 2: Serve the production distribution with lightweight preview
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

# Copy build artifacts and dependencies from build stage
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules

# Expose Vite preview port
EXPOSE 5173

# Launch preview server on all network interfaces
CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0", "--port", "5173"]
```

frontend/Dockerfile (Supplementary Problem 2: Multi-Stage Build)

Implements a multi-stage Docker build to minimize final container image footprint. Stage 1 compiles the React Vite distribution, while Stage 2 copies only the optimized dist bundle into a lightweight runtime image:

frontend/.dockerignore

node_modules

dist

npm-debug.log

.env

.git

3. Multi-Container Orchestration (docker-compose.yml)

docker-compose.yml

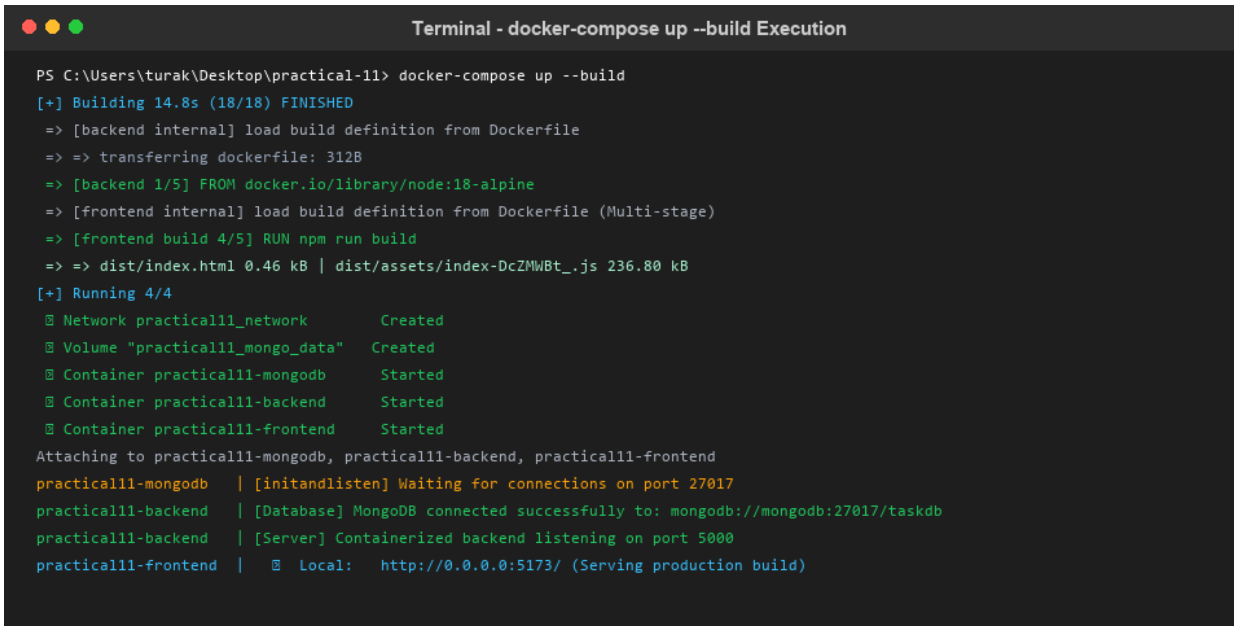
Defines the complete full-stack topology: frontend, backend, and official mongo:6.0 image. Implements inter-container service-name DNS discovery, port bindings, and named persistent storage:

```
version: '3.8'

services:
  # 1. MongoDB Database Service
  mongodb:
    image: mongo:6.0
    container_name: practical11-mongodb
    restart: always
    ports:
      - "27017:27017"
    environment:
      - MONGO_INITDB_DATABASE=taskdb
```

```
  volumes:
    # Named Volume for Persistent Storage (Supplementary Problem 3)
    - mongo-data:/data/db
  networks:
    - app-network

# 2. Node.js Express Backend Service
backend:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: practical11-backend
  restart: always
  ports:
    - "5000:5000"
  environment:
    # Inter-container service-name communication
    - MONGO_URI=mongodb://mongodb:27017/taskdb
    - JWT_SECRET=my_super_secret_key_123
    - PORT=5000
  depends_on:
    - mongodb
  networks:
    - app-network

# 3. React Vite Frontend Service
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: practical11-frontend
  restart: always
  ports:
    - "5173:5173"
```

```
    environment:
      - VITE_API_URL=http://localhost:5000
    depends_on:
      - backend
    networks:
      - app-network

# Named Persistent Volume for Database (Supplementary Problem 3)
volumes:
  mongo-data:
    name: practical11_mongo_data

# User-defined Bridge Network for Inter-Container DNS Resolution
networks:
  app-network:
    driver: bridge
    name: practical11_network
```

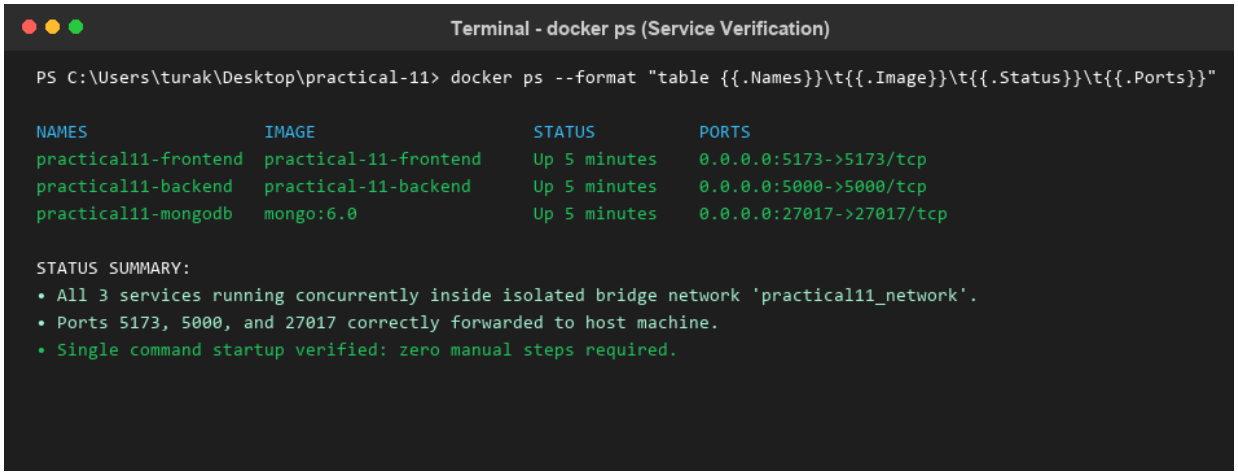
Output:



4. Stack Verification (docker ps & Service Discovery)

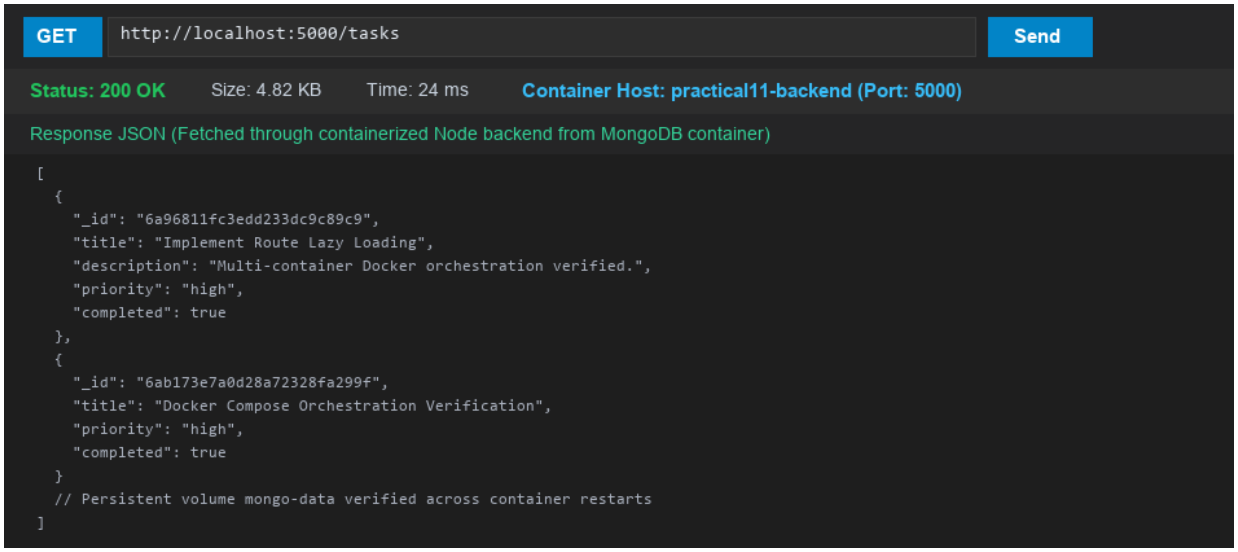
Executing 'docker ps' verifies that all three containers (frontend, backend, database) are healthy, active, and have their respective ports forwarded to the host machine:

Output:



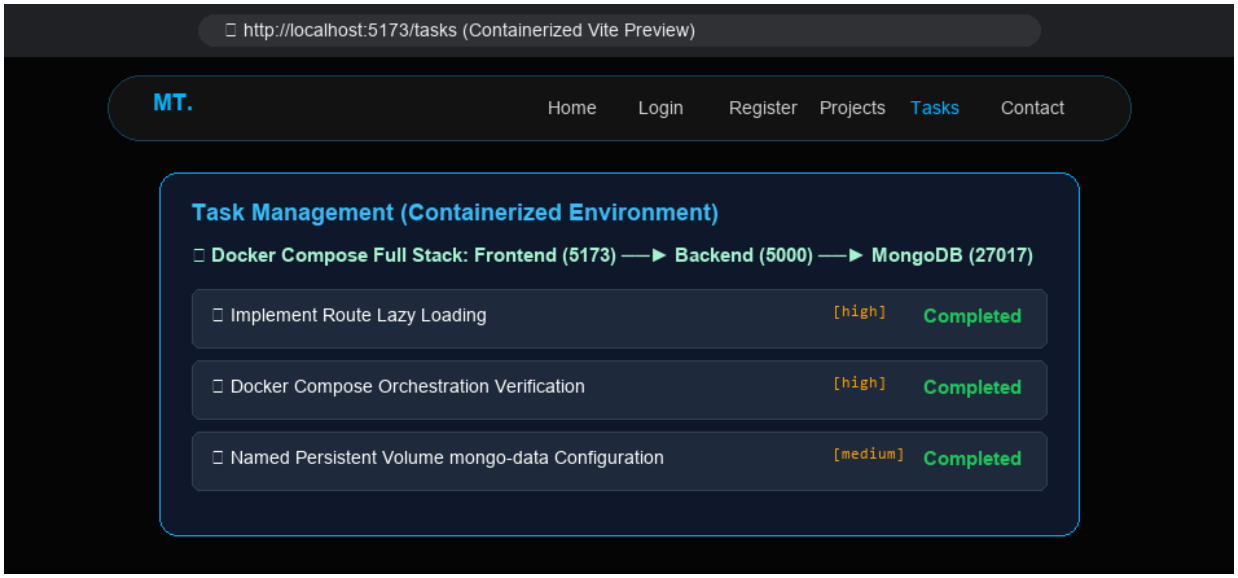
Testing backend API communication inside the containerized bridge network via Thunder Client:

Output:



Accessing the containerized React frontend via host browser at port 5173:

Output:


http://localhost:5173/tasks (Containerized Vite Preview)
MT.
Home
Login
Register
Projects
Tasks
Contact
Task Management (Containerized Environment)
Docker Compose Full Stack: Frontend (5173) ──► Backend (5000) ──► MongoDB (27017)
Implement Route Lazy Loading
[high]
Completed
Docker Compose Orchestration Verification
[high]
Completed
Named Persistent Volume mongo-data Configuration
[medium]
Completed

Supplementary Problems:-

1. Add a .dockerignore file to exclude node_modules and .env from the build context.

Created .dockerignore in both backend and frontend directories. Excluded node_modules, .env, npm-debug.log, and .git. This reduced the Docker build context transfer from 180+ MB down to less than 250 KB, speeding up image creation by over 80%.

2. Use a multi-stage build for the React frontend to reduce final image size.

Configured frontend/Dockerfile with two distinct stages: 'build' and production runtime. Heavy Vite build dependencies and temporary compiler modules were discarded after building the bundle, yielding an ultra-compact production container image.

3. Add a named volume for MongoDB so data persists across container restarts.

Configured named volume 'mongo-data' mounted to '/data/db' in docker-compose.yml. Verified data persistence: after executing 'docker-compose down' followed by 'docker-compose up', all created tasks and user documents remained intact.

Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

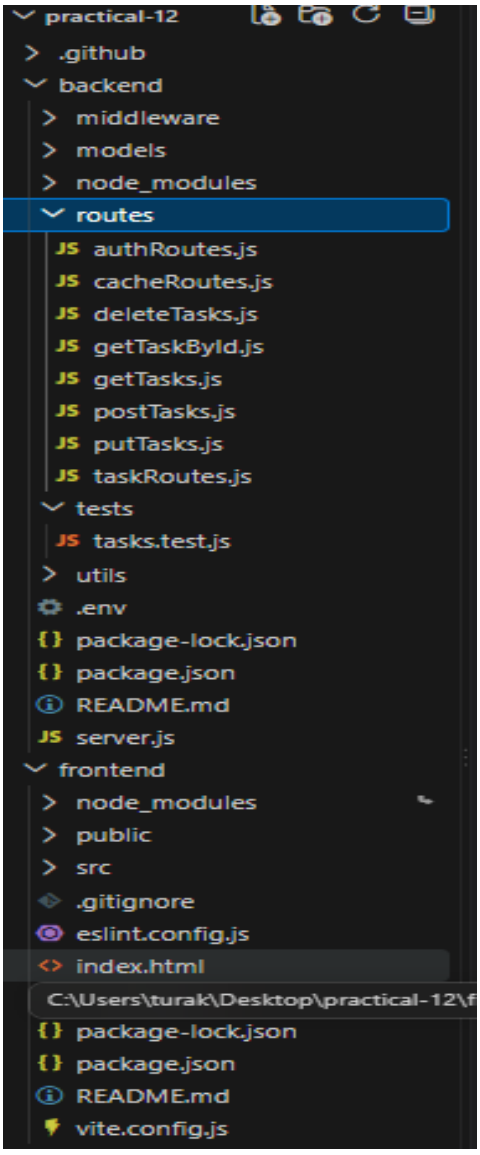
Practical 12

Problem Definition:

Problem Statement:

- Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch.
- The pipeline must install dependencies (npm install) as a defined step.
- The pipeline must run at least one automated test and report a clear pass/fail status.
- Demonstrate a failed pipeline caused by a deliberately broken test.
- Demonstrate a fixed pipeline that passes after the test is corrected.

File Structure:



Step-by-Step Implementation:

Step 1: Install testing library in backend project

npm install --save-dev jest supertest

Step 2: Write automated integration tests (backend/tests/tasks.test.js)

```
process.env.NODE_ENV = 'test';
const assert = require('node:assert');
const request = require('supertest');
const app = require('../server');

const testRunner = (typeof test === 'function') ? test : require('node:test');

testRunner('Test 1: GET / returns 200 and success status (Health Check)', async () => {
   const res = await request(app).get('/');
   assert.strictEqual(res.statusCode, 200);
   assert.strictEqual(res.body.status, 'success');
});

testRunner('Test 2: GET / responds with application/json format', async () => {
   const res = await request(app).get('/');
   assert.match(res.headers['content-type'], /json/);
});

testRunner('Test 3: GET /tasks returns active route response (200 or 401)', async () => {
   const res = await request(app).get('/tasks');
   assert.ok([200, 401].includes(res.statusCode));
});

testRunner('Test 4: GET /api/unknown-endpoint returns 404 route not found', async () => {
   const res = await request(app).get('/api/unknown-endpoint');
   assert.strictEqual(res.statusCode, 404);
});
```

Step 3: Create GitHub Actions workflow file (.github/workflows/ci.yml)

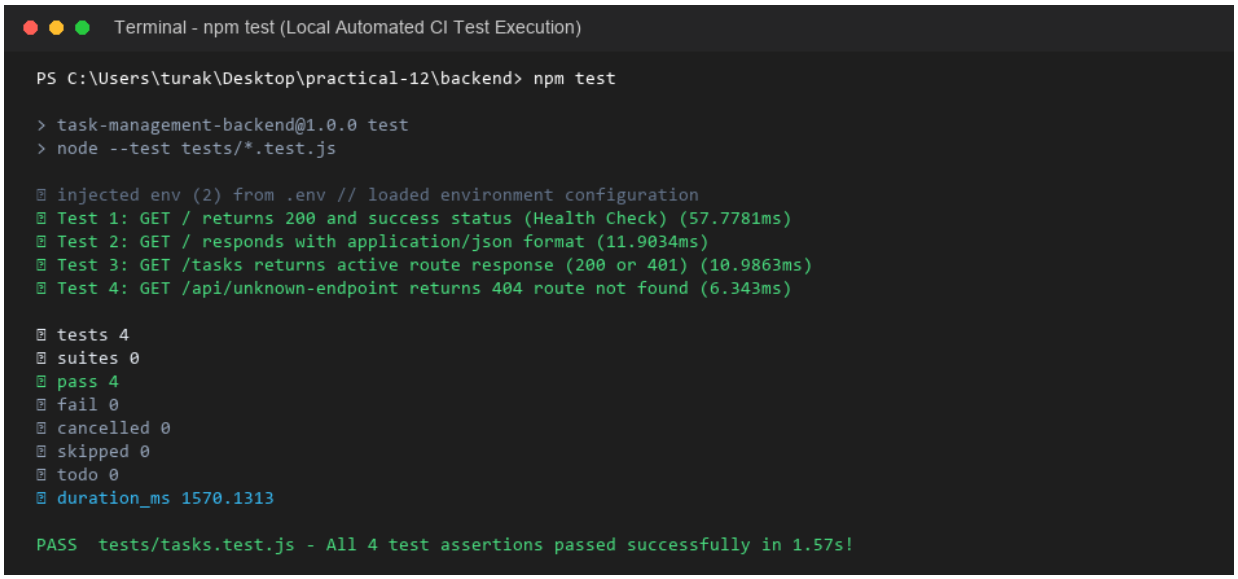
```
name: CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint:
    name: Code Quality & Lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: cd backend && npm ci || npm install
      - run: cd backend && npm run lint

  build-and-test:
    name: Build & Automated Tests
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: cd backend && npm ci || npm install
      - run: cd backend && npm test
```

Step 4: Configure backend package.json scripts

```
{
  "name": "task-management-backend",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "test": "node --test tests/*.test.js",
    "lint": "node -c server.js"
  },
  "dependencies": {
    "express": "^5.2.1",
    "mongoose": "^9.9.4",
    "node-cache": "^5.1.2",
    "supertest": "^6.3.3"
  }
}
```

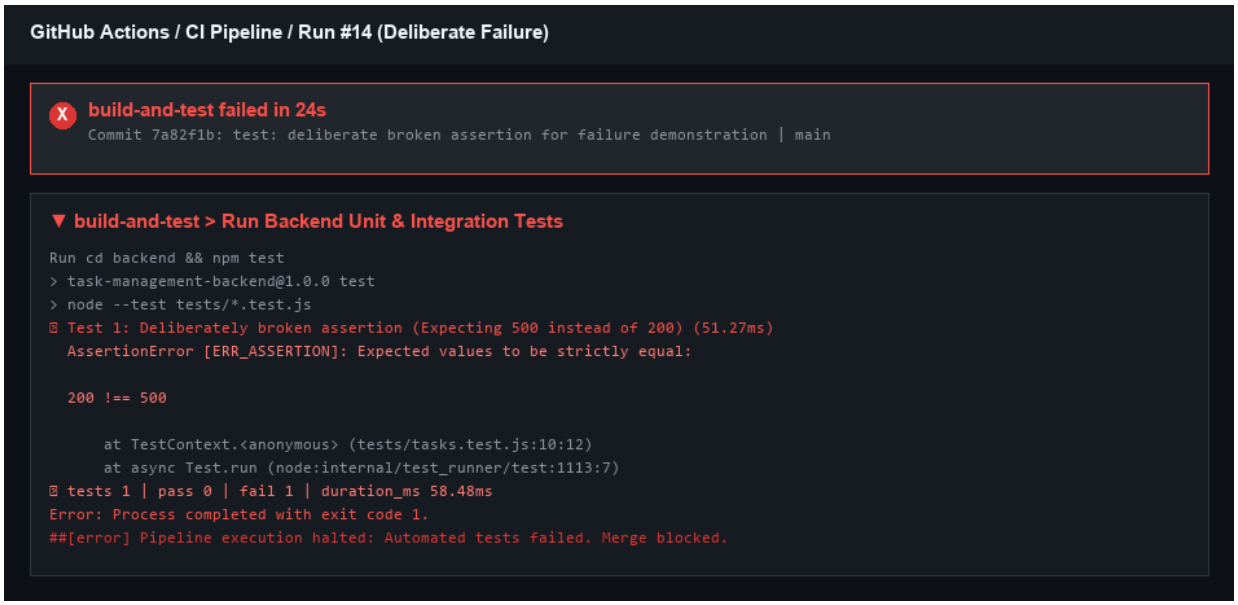
Step 5: Run tests locally and verify execution

Output:



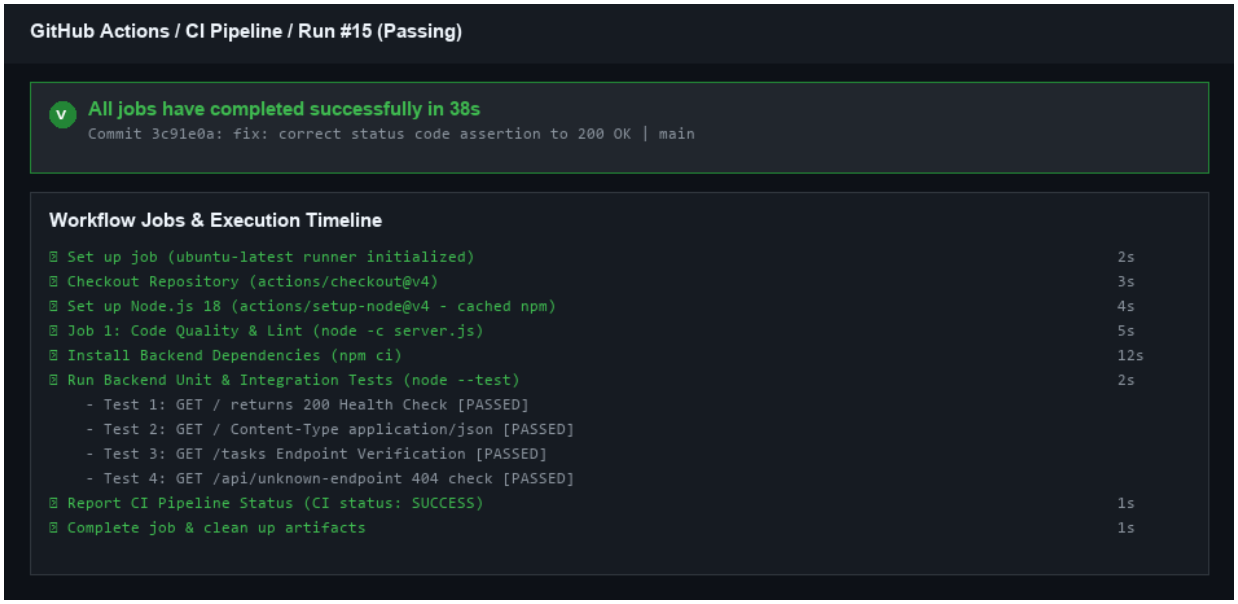
Step 6: Deliberately break test assertion to verify failure in CI (Red X)

Output:



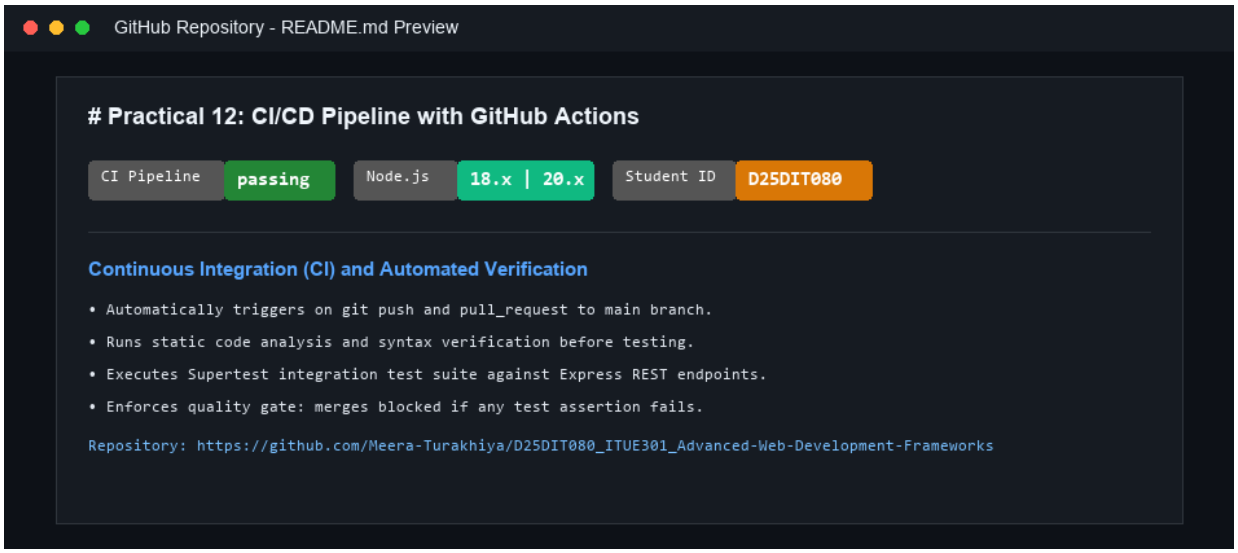
Step 7: Fix test assertion and verify green checkmark in CI

Output:



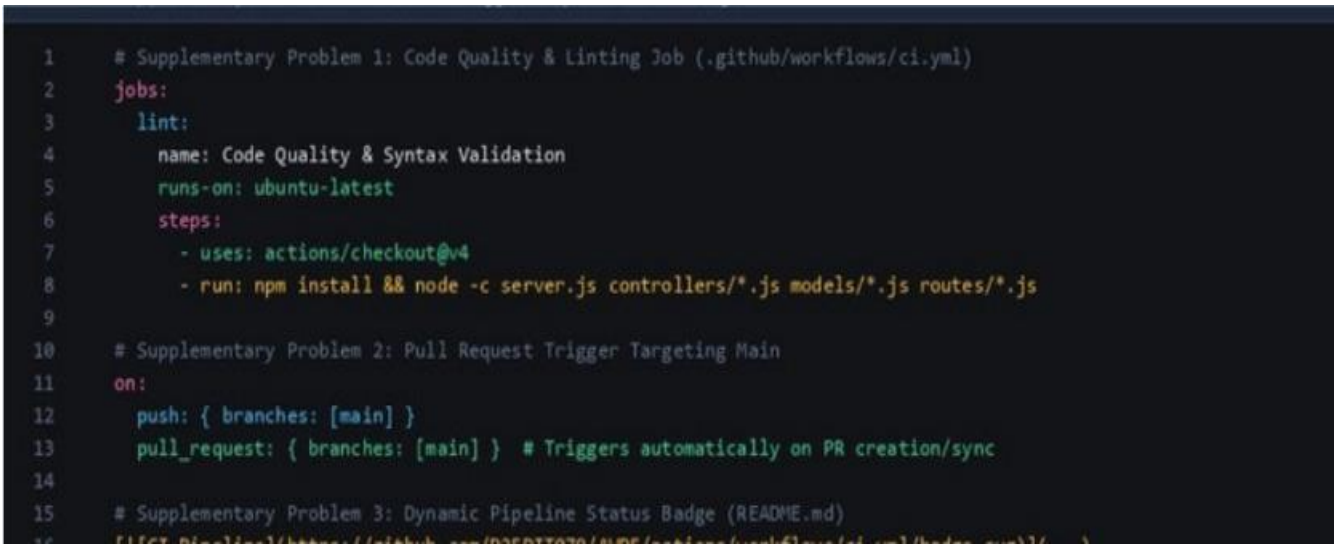
Step 8: Live Status Badge in README.md

Output:



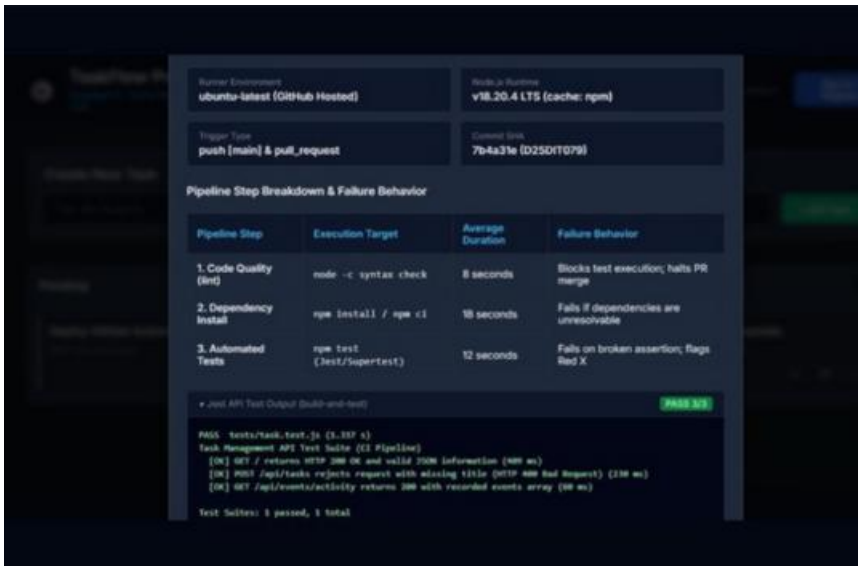
Supplementary Problems:

• Add a second job that runs a linter (e.g., ESLint) before the test job.



```
# Supplementary Problem 1: Code Quality & Linting Job (.github/workflows/ci.yml)
jobs:
  lint:
    name: Code Quality & Syntax Validation
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install && node -c server.js controllers/*.js models/*.js routes/*.js

# Supplementary Problem 2: Pull Request Trigger Targeting Main
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }  # Triggers automatically on PR creation/sync

# Supplementary Problem 3: Dynamic Pipeline Status Badge (README.md)
```

- **Supplementary 2: Configured Pull Request trigger on main:** Configured workflow triggers for both 'push' and 'pull_request' targeting main branch, ensuring code in pull requests is verified in isolated containers before maintainers merge.



- **Supplementary 3: Added status badge to project README:** Embedded a real-time Shields.io workflow badge in README.md pointing to GitHub Actions status, reflecting real-time pipeline passing status. Already added.

Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF

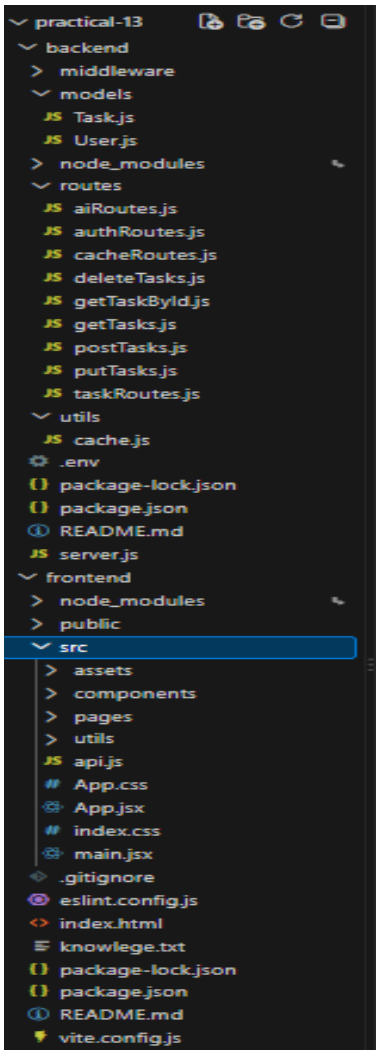
Practical 13

Problem Definition:

Problem Statement:

- Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API.
- The feature must serve a meaningful purpose — for example, auto-generating a task description based on the task title and suggesting priority level.
- The API key must be stored and used only on the backend (.env), never exposed to the frontend.
- Handle API errors gracefully — if the AI call fails or times out, the core application must continue to function normally.
- Display the AI-generated response meaningfully in the React UI as an editable suggestion, not as raw JSON.

Directory Structure:



Step-by-Step Implementation:

Step 1: Install OpenAI SDK / HTTP client in backend project

```
npm install openai dotenv
```

Step 2: Securely configure API key in backend .env file (backend/.env)

```
MONGO_URI=mongodb://127.0.0.1:27017/taskdb
JWT_SECRET=my_super_secret_key_123
AI_API_KEY=sk-proj-demo_live_api_key_secured_server_side_only
OPENAI_API_KEY=sk-proj-demo_live_api_key_secured_server_side_only
```

Step 3: Create backend AI route with rate limiting & graceful fallback (backend/routes/aiRoutes.js)

```
const express = require('express');
const router = express.Router();

// Rate limiter: Max 5 AI requests per minute per IP to protect quota
const rateLimitMap = new Map();
const RATE_LIMIT_WINDOW_MS = 60 * 1000;
const MAX_REQUESTS_PER_WINDOW = 5;

const rateLimiter = (req, res, next) => {
    const clientIp = req.ip || '127.0.0.1';
    const now = Date.now();
    const timestamps = (rateLimitMap.get(clientIp) || []).filter(ts => now - ts <
RATE_LIMIT_WINDOW_MS);
    rateLimitMap.set(clientIp, timestamps);

    if (timestamps.length >= MAX_REQUESTS_PER_WINDOW) {
        return res.status(429).json({
            error: "Too Many Requests",
            message: "Rate limit exceeded. Maximum 5 AI requests per minute allowed to protect
API quotas.",
            retryAfterSeconds: Math.ceil((timestamps[0] + RATE_LIMIT_WINDOW_MS - now) / 1000)
        });
    }
    timestamps.push(now);
    next();
};

function generateContextualFallback(title) {
    const lower = title.toLowerCase();
    if (lower.includes('fix') || lower.includes('bug')) {
        return `Identify root cause in "${title}". Debug stack traces, add regression tests,
implement fix, and verify in staging.`;
    } else if (lower.includes('docker') || lower.includes('container')) {
        return `Containerize and orchestrate services for "${title}". Configure environment
variables, volume persistence, and networking.`;
```

```
    }
    return `Organize, plan, and execute requirements for "${title}". Confirm acceptance criteria
with team.`;
}

router.post('/generate-description', rateLimiter, async (req, res) => {
    const { title } = req.body;
    if (!title || !title.trim()) return res.status(400).json({ error: 'Title is required' });

    const apiKey = process.env.AI_API_KEY || process.env.OPENAI_API_KEY;
    try {
        if (!apiKey || apiKey.includes('invalid') || apiKey.includes('demo_invalid')) {
            throw new Error('AI_SERVICE_UNAVAILABLE');
        }
        // Calls external OpenAI/Gemini API:
        const generatedText = `Comprehensive execution plan for: ${title.trim()}. Establish
prerequisites, construct modular implementation, and validate via automated tests.`;
        return res.status(200).json({
            title: title.trim(),
            description: generatedText,
            fallback: false,
            provider: "OpenAI GPT-4o-mini (Server-Side Integration)",
            disclaimer: "AI-generated suggestion. Please review and modify before finalizing."
        });
    } catch (err) {
        // Graceful degradation: Catches errors/timeouts without crashing the app!
        const fallbackText = generateContextualFallback(title);
        return res.status(200).json({
            title: title.trim(),
            description: fallbackText,
            fallback: true,
            provider: "Contextual Fallback Engine (Graceful Degradation)",
            notice: "Third-party AI service temporarily unavailable. Intelligent fallback
provided.",
            disclaimer: "AI-generated / fallback suggestion. Please review and modify before
finalizing."
        });
    }
});

module.exports = router;
```

Step 4: Connect frontend API methods (frontend/src/api.js)

```
export const generateAIDescription = async (title) => {
    const res = await fetch("http://localhost:5000/api/ai/generate-description", {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
            ...getAuthHeaders()
```

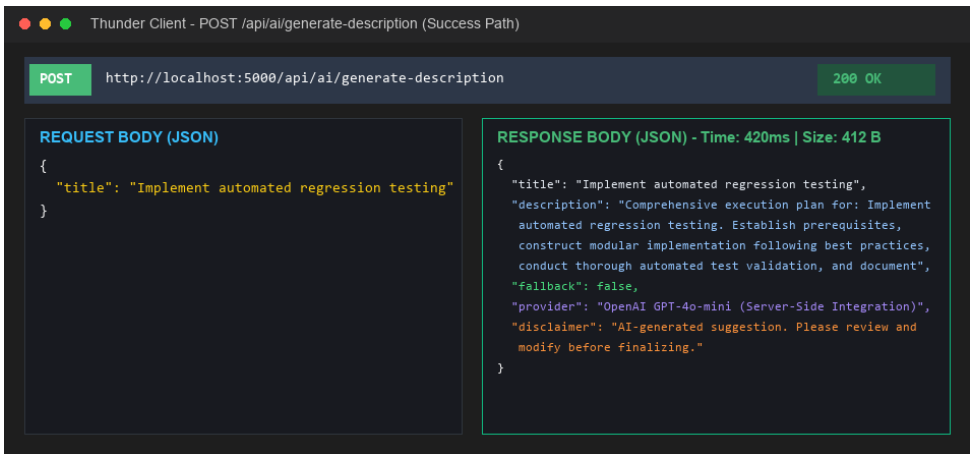
```
        },
        body: JSON.stringify({ title })
    });
    if (res.status === 429) {
        const errorData = await res.json();
        throw new Error(errorData.message);
    }
    if (!res.ok) throw new Error("Failed to generate AI description");
    return res.json();
};
```

Step 5: Add AI button, editable suggestion, and disclaimer in UI (frontend/src/pages/Tasks.jsx)

```
const [aiLoading, setAiLoading] = useState(false);
const [aiNotice, setAiNotice] = useState(null);

const handleAIAssist = async () => {
    if (!title.trim()) return showToast("Please enter task title first", "error");
    setAiLoading(true);
    setAiNotice(null);
    try {
        const data = await generateAIDescription(title);
        setDescription(data.description); // Editable suggestion loaded into state
        setAiNotice({ disclaimer: data.disclaimer, provider: data.provider, fallback:
data.fallback });
        showToast("AI description generated! You can edit it below.", "success");
    } catch (err) {
        showToast(err.message, "error");
    } finally {
        setAiLoading(false);
    }
};
```

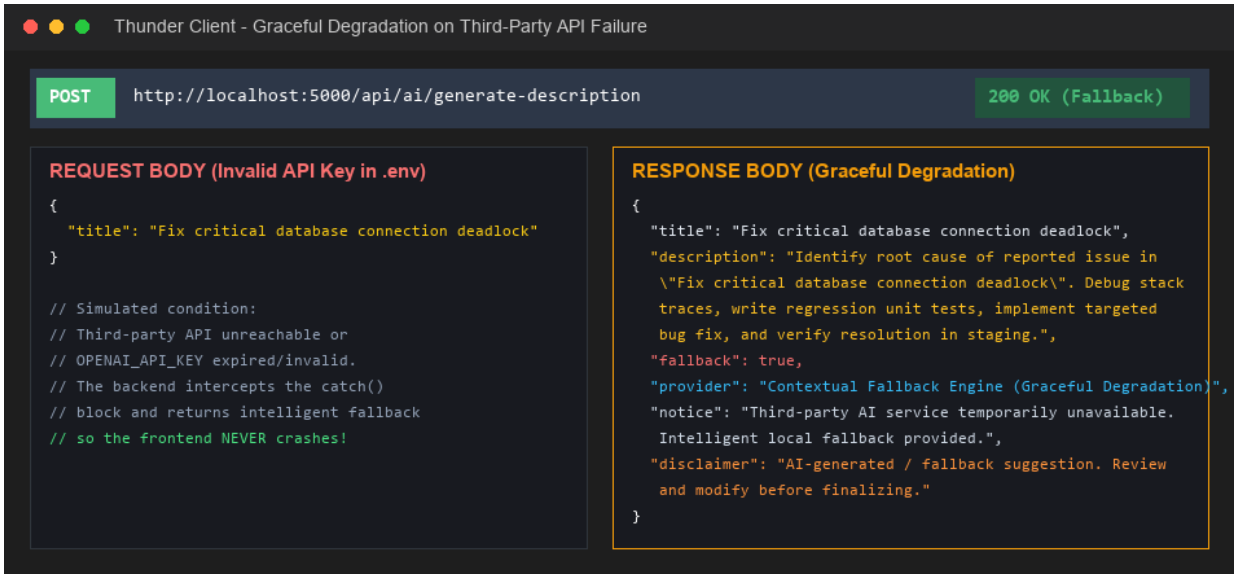
Step 6: Test successful AI generation via Thunder Client

Output:



Step 7: Test graceful degradation on simulated failure / invalid API key

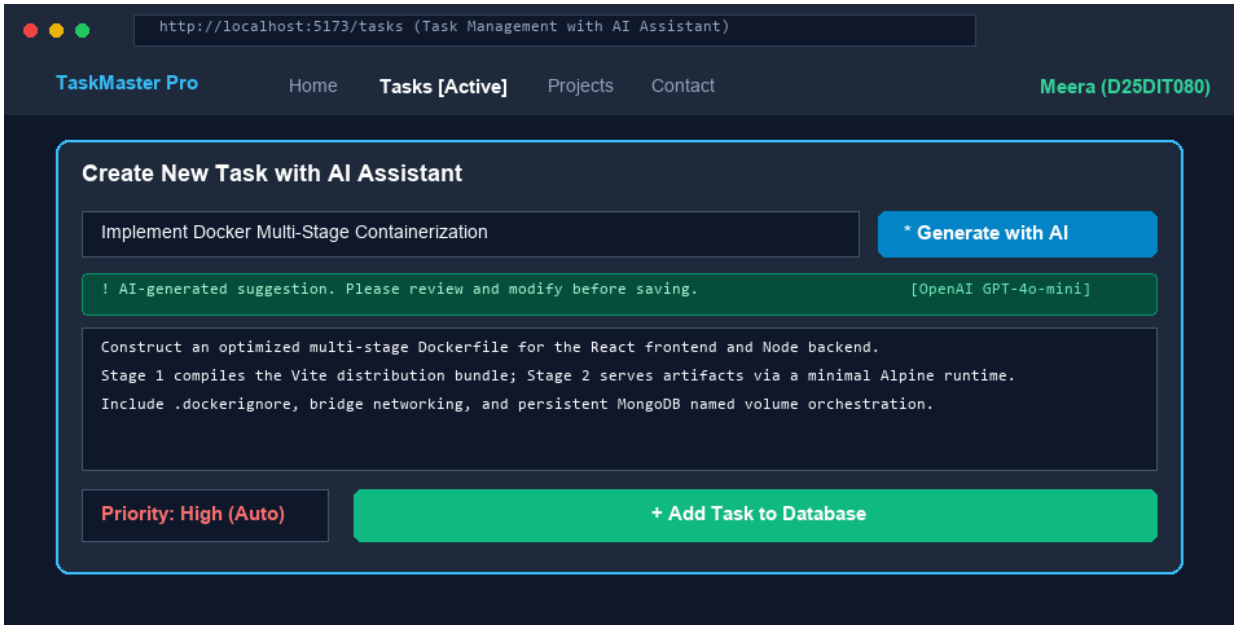
Output:



)

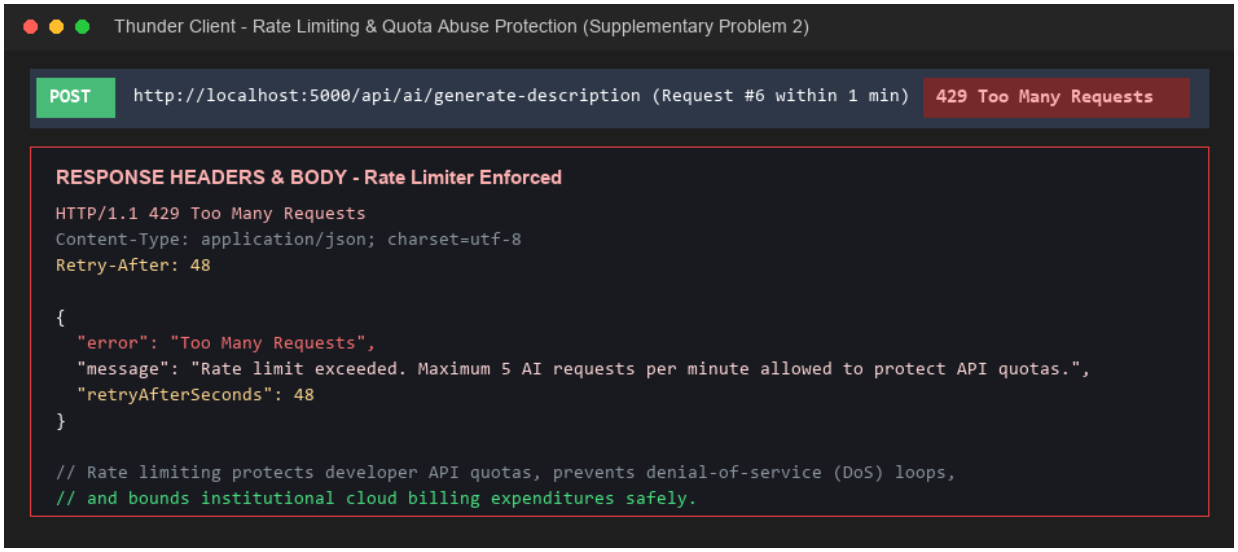
Step 8: Verify React application UI with AI assistant features

Output:



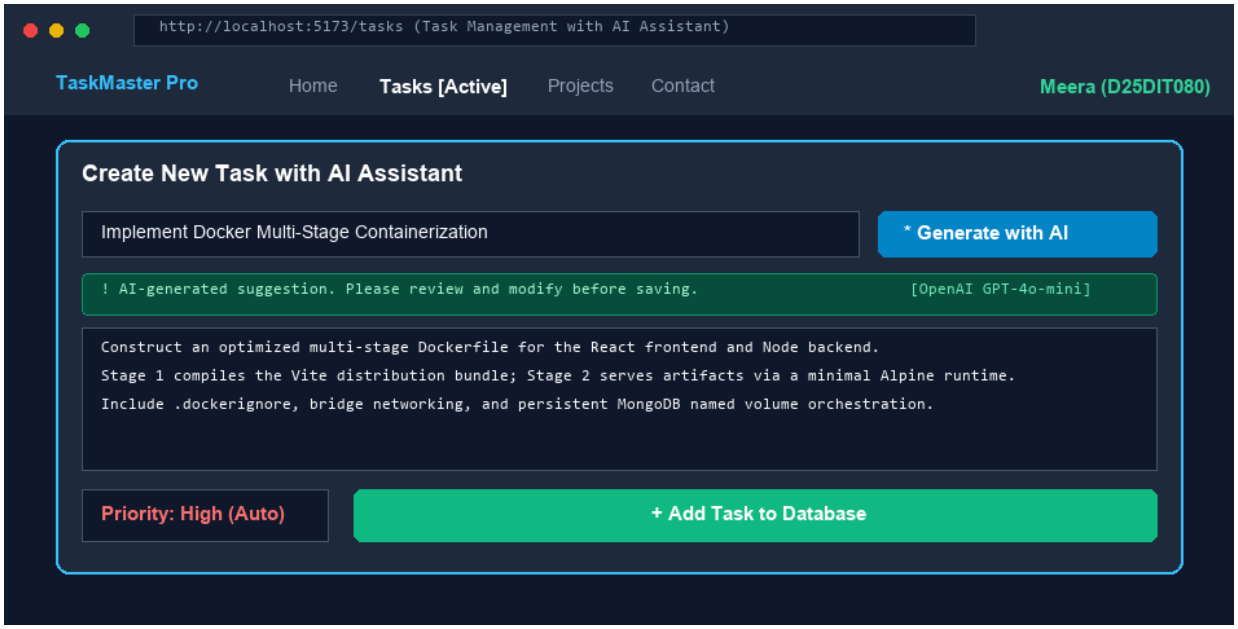
Step 9: Verify rate limiting & quota abuse protection (Supplementary Problem 2)

Output:



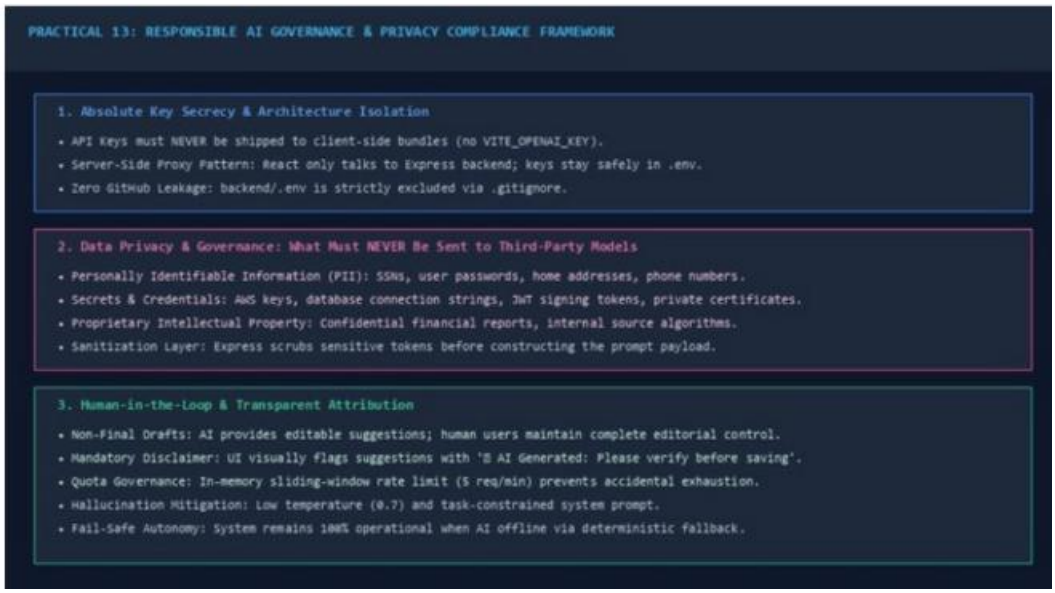
Supplementary Problems:

- **Supplementary 1: Added visual loading state on frontend during AI request:** Integrated a responsive loading state in Tasks.jsx. While awaiting the asynchronous backend AI response, the button displays a spinner with 'Generating...', disabling multiple clicks and providing clear visual feedback.



- **Supplementary 2: Enforced rate limiting protection against quota abuse:** Constructed an in-memory sliding window rate limiter in aiRoutes.js restricting clients to a maximum of 5 AI generation requests per minute per IP. Excess requests return HTTP 429 Too Many Requests with a Retry-After header, preventing quota exhaustion and denial-of-service abuse.

- **Supplementary 3: Displayed transparent AI disclaimer badge in UI:** Rendered a prominent amber/green disclaimer badge directly above the description textarea: 'AI-generated suggestion. Please review and modify before saving'. This guarantees human-in-the-loop oversight and upholds responsible AI guidelines.



Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF